C语言篇--白洪欢老师答疑精华

001 .C文件扩展名看不到，求助！！！！！
002 求值x&&(y=5),x&&z&&(y=1),y
003 关于TC调试过程
004 考试系统中的填空题怎样将给出的部分程序复制到tc中?
005 Null pointer assignment是什么意思?
006 数组作为参数的问题解答
007 %d输入整数后，不能gets()的问题
008 关于函数定义
009 为什么gets不了a?
010 int main()就意味着main()应该有返回值!
011 机考时注意每次repeat要重新对一些变量赋值
012 二维字符数组的赋值与输入问题
013 逗号表达式、strcmp、++k、数组名赋值
014 模拟题“十六进制字符转换” 问题及解答
015 建立和编译多个文件构成的程序及全局变量跨文件共享
016 定义在fun()与main()之间的全局变量
017 floating points format not linked
018 关于TC的调试过程演示(有声版)
019 偶做的05年答案(大家帮忙改下)
020 机考题中字符输入相关问题及解答
021 怎样把十进制转化为二进制呢?
022 表达式等价、指针的指针、移位
023 自动类型转换、typedef、异或
024 关于printf的%d输出格式问题
025 一个问题while(s++) putchar(++s); 很懊恼??
026 sizeof()问题求教!
027 五个问题(sizeof、void、异或、移位等)及解答
028 问个C语言笔试题:字符指针数组与指针的指针
029 关于打开文件时的文件路径
030 求问:结构指针
031 提问:关于结构
032 help: x%=x++/--y
033 问题:for(c=getchar(); getchar()!='#'; c=getchar())
034 问逻辑运算
035 请问：typedef int NUM[100];

036 问题：#define FB(a,b) (ab+1)
037 结构指针指向字符指针问题
038 关于%u?
039 二分法的例子程序
040 变量定义 int p[5], a[5] 及%s输入
041 问动态创建二维数组的问题
042 c语言中两个浮点数能比较大小么？
043 问题解答：void main()与int main()的区别
044 问个关于 指针与数组的问题
045 #和的游戏~
046 int (a)[80]这是不是指向二维数组的写法?
047 问小白老师，如何区分Shift等功能键的左、右?
048 问 TC 编译模式以及 VGA 12h 直接写屏
049 %lx、指针变量定义时赋值、#define F(x)
050 再问野指针的问题
051 上机练习15题:大写转小写，小写转大写
052 指数形式的常数是什么类型？如何理解0?0:1
053 今天上课关于计算宏定义FB(a,b)的题（请看第二题）
054 int p = (int)123 是什么意思?
055 一道与文件相关的题：最后一个数加了两遍
056 long b=a0x12345678; printf("%x\n", b);
057 switch中的break与循环中break的区别
058 一道比较复杂的指针题(3颗)
059 行指针与列指针的区别
060 字符串中查找子串问题?
061 if(i>j) if(a[i][j]==0)与i>j&&a[i][j]==0
062 指针或者数组元素调试
063 a[3]={'a','b','c'}，sizeof（a）=3
064 为什么5％－3等于2？ －5％3却等于-2
065 %%表示什么?
066 强制转换：低类型转换成高类型
067 关于全局变量
068 关于结构变量和指针赋值的问题
069 p是结构数组名，为什么p->k=2是正确的?
070 如何在TC里画图? initgraph怎么用?
071 区别:1. EOF和'\0'; 2.'\n'和数据输入
072 二维数组与指针的指针的区别
073 如何对结构中的字符指针成员进行赋值?
074 int a,b[3]; a=b; 这样对吗?

075 若p是结构指针，为啥p.pk是错的?
076 arr是char二维数组,printf("%s\n",arr)是啥意思?
077 左结合与右结合是指运算符还是变量?
078 如何重命名.txt为.c ?
079 关于 编译指令 操作符'#'
080 函数声明能写在主函数里面吗?
081 关于C语言指针的一些问题
082 char *a="abc",*b="edf";strcat(a,b);出问题
083 一道笔试题:char *a[2]={"one","two"},**p=a;
084 %d格式输入字母时，为什么循环停不下来?
085 p=&x++;为什么错?
086 char s[100]，scanf("%s",&s)有没有问题?
087 void p(int x,int y) {p=x;x=y;y=p;}
088 如果局部变量与全局变量同名
089 实参数组元素个数与形参数组元素个数不一致问题
090 全局变量未赋值时，它的初始值是0吗?
091 如何得到某个文件夹中的所有文件名?
092 如何判断键盘是否有键按下但不停止程序继续执行?
093 void 类型指针变量与其它类型指针变量的相互赋值问题
094 如何动态分配数组?
095 VC下strlwr函数参数的指针问题
096 40012题{double y=1;if(y/10==0.1)}已修复
097 上机题目汇总
098 函数声明在main()外面与里面有啥区别?
099 数组名为什么不能被赋值?
100 结构指针p->y=3.0为啥错(其中y是float)?
101 为什么不能用%f格式输出一个字符串?
102 FROG、SEAL、LION、LAMB构成defy()-08程序阅读1
103 求问08卷选择题第10题（有关strcmp）
104 关于数组指针是否就是指针的指针的探讨
105 表达式'A'+'B'的类型为什么是int?
106 复杂字符串求strlen()、sizeof()的自助解决方法
107 表达式优先级、结合性疑问：优先级高就先计算吗?
108 二维数组如何转成指针的指针?
109 如何防止scanf("%d",&x)时因输入字符而陷入死循环?
110 BMP Quick Show in 12h VGA mode
111 BMPShow in VESA Mode(102476824bit)
112 FractalTree in VESA Mode
113 MandelbrotFractal in Vesa Mode

114 (*p) ->str++中的++加在哪里？ (问题有更新)
115 关于strlen和sizeof
116 关于二阶指针，二维数组，数组指针
117 输入数字后要用getchar()?
118 int a=10,x=0;x=~a 问结果x等于多少
119 对于int *p；p可以赋值为二维数组名吗?
120 VC++ 6.0 精简版 & MSDN精简版
121 C语言：不用！求!x
122 分解质因数无需验证因数是否质数
123 C语言模拟考试答案（供参考）
124 char a[]="ABC\XYZ\007\xFF";求sizeof
125 C语言函数中不可以返回尸体的地址
126 为什么%e格式不可以输出一个字符串
127 求助：文本文件和二进制文件的区别弄不清啊
128 int8h时钟中断程序例子
129 迷宫虫系列
130 关于setjmp的问题
131 给大家的C大程提供点灵感哈~<C/C++ RPG游戏实现技术细节>
132 RPG游戏实现技术细节 第四章 模块动态载入
133 关于单精度浮点型做参数的问题
134 转义字符\ddd,\后面一定要3位吗?
135 08试卷，查找数组中是否有相同元素，双重循环跳出条件
136 部分位求反
137 若arr是char的二维数组，则printf("%s\n",arr);有错?
138 夹在函数之间的全局变量可以被该变量前面的函数访问吗?
139 用typedef定义函数指针
140 关于DosBoxTc的使用方法
141 老师，问一个有关于函数指针的问题~
142 数组指针和指针数组为什么可以一样进行p[i][j]操作
143 [C程]关于feof()函数不能正常判断文件结束位置的探讨
144 win8.1系统运行vc6.0的解决方法
145 [C大程]如何用010Editor搜索源代码中的函数?
146 [C大程]如何查库函数帮助信息?
147 [C大程]中断函数内切换到用户堆栈以便调用库函数的演示
148 关于printf()函数参数从右到左方向传递的问题
149 老师，该怎么理解这里的a和&a大小一样?
150 用dev-cpp进行C语言调试的方法（汇编与条件断点）

001 .C文件扩展名看不到，求助！！！！！

问：

编了个程序,保存后电脑里怎么没有.c的文件啊？只有个文本文档和个．bak的，还有个应用程序．还漏了个OBJ的

答：

你那个文本文档会不会就是.C文件?
如果你把.C文件关联到记事本的话,
那你看到的可能就是一个文本文档。

有些同学上传的.C文件名后面又多
出一个.C应该也是同样原因。

其实这个是大学计算机基础课上应该
已经讲过的常识。
资源管理器->工具->文件夹选项->查看->[√]隐藏已知文件类型的扩展名

把这个勾去掉就可以看到文件的扩展名了。

002 求值x&&(y=5),x&&z&&(y=1),y

问：

书上113页第6题的第7,8两个小题,求表达式的值.
初值情况为: x=5,y=5,z=0.
第7: x&&(y=5),x&&z&&(y=1),y
第8: x||(z=1),(z==1)||(x=0),x||(z=2),z
我做的答案是第7题:1,第八题:2.
可是如果编出来运行的话,答案却是第7题:5,第八题:2.
源程序为:

```
#include <stdio.h>
void main()
{
        int x=5,y=5,z=0;
        printf("%d,%d\n",(x&&(y=5),x&&z&&(y=1),y),(x||(z=1),(z==1)||(x=0),x||(z=2),z));
}
```

这到底是怎么回事呢?第7题中y在执行 x&&(y=5),x&&z&&(y=1),y的时候不是被赋值为1,然后才执行x&&(y=5),x&&z&&(y=1),y的么?为什么不是y=1呢?
如果说最后一项的表达式和前面的赋值没有关系,那为什么第八题运行出来是2呢???

答：

这个题目确实有点变态。没有以下3个一般书上不会讲到的知识是做不对的：

1. A || B 中，如果A真，则B不求值
2. A && B中，如果A假，则B不求值
3. C语言中参数传递次序是从右到左

```
#include <stdio.h>
void main()
{
 int x=5,y=5,z=0;
 printf("%d,%d\n",(x&&(y=5),x&&z&&(y=1),y),(x||(z=1),(z==1)||(x=0),x||(z=2),z));
}
```

根据3，我们应该先算后面那项：
(x||(z=1), (z==1) || (x=0), x ||(z=2), z)
真 不求值 假 x=0 假 z=2 2
所以，结果输出2，并且x=0, z=2。

再计算第1项：
(x && (y=5), x && z && (y=1), y)
假 不求值 假 不求值 不求值 5
所以，最后输出5，y的值保持不变。

003 关于TC调试过程

问：

照老师说的，按F8调试 可到这一步的时候，再按F8，亮条不继续向下走了，而是result值变化 怎么回事呢

答：

把光标移到return result;处，按F4。
就是Goto Cursor，意思是运行到光标处。

或者把光标移到return result;处，按Ctrl+F8，此处会出现一个红色的亮度，
表示设置一个断点。
然后Run->Run，程序运行到return处时会自动停下来。

004 考试系统中的填空题怎样将给出的部分程序复制到tc中?

问：

考试系统中的填空题怎样将给出的部分程序复制到tc中?

答:

方法1:

先复制， 然后在TC文件夹中新建一个.TXT文件,

粘贴

把.TXT文件名改成.C

TC中用File->Load功能调出此文件。

方法2:

在桌面上新建一个TC的快捷方式

复制考题,

在桌面上新建一个.TXT文件,

粘贴并保存

用鼠标点住这个新建的.TXT文件，并把它拖到TC快捷方式上面。

005 Null pointer assignment是什么意思?

问:

为什么运行时每次都是

NO

Null pointer assignment

这是什么意思啊

答:

空指针赋值的意思。

象以下程序会导致这个错误:

```
int x = 123;
int *p;
p = NULL;      // NULL表示值为0的地址。

*p = x;          // 企图把x赋值给地址为0的变量。
```

C语言规定，任何变量的地址都不可能为0，因此0地址称为空指针，空指针所指向的对象是不存在的，也不能对空指针指向的对象赋值。

006 数组作为参数的问题解答

问：

一个同学来信问的问题，他的信箱貌似有问题，没办法回复。发到这里。

问题： 我有一个问题。在定义子函数的时候，如果我已经在void main里面定义了一个数组a[7][7]，并且在主函数里面对这个数组输入了一串数。如果我想在 子函数里调用void main内的那个数组内的数，该怎么办?

答：

这个要用到数组作为参数传给子函数，例如：

```
int sum(int x[2][3]) {
    int i, j;
    int y=0;
    for(i=0; i<2; i++)
    {
        for(j=0; j<3; j++)
            y += x[i][j];
    }
    return y;
}

void main() {
    int a[2][3]={{1,2,3}, {4,5,6}};
    int y;
    y = sum(a);
    printf("%d", y);
}
```

007 %d输入整数后，不能gets()的问题

问：

情帮忙看下这个程序有什么问题，谢谢

运行结果很奇怪，输入m后就直接跳过gets（s）了

```
#include<stdio.h>
void strmcpy(char s[],char t[],int m)
{
   int i=0,j;
   for(j=m;t[j]!='\0';j++)
  {   s[i]=t[j];
      i++;
  }
}
main()
{
   int m;
   char s[80];
   char t[80];
   scanf("%d",&m);
   gets(t);
   strmcpy(s,t,m);
   puts(s);
}
```

但改变输入语句的次序后再试就对了，为什么呢？真奇怪

答：

因为输入%d时后面有回车，此回车进入到gets中了，

可以用以下两个方法解决：

1. 把gets(t)改成： scanf(" %s", t); /* 注意%s前面有一个空格 */
2. 在gets(t)前面插入以下一句：
   fflush(stdin);

008 关于函数定义

问：

您上课一定讲过，是我没好好听，交待完毕，进入正题

自定义函数的变量类型哪些地方要说明？ 自定义函数中？ main 中也有用到，这些变量还要说明么？

以下程序问题在于 自定义函数中

```
charstrmcpy(char s[100],char t[100],int m)
```

这一句 Expression syntax in funmction main 要说明不要说明的都说了，最保守的方式，换来的还是 错！

具体 p249 8

```
#include<stdio.h>
#include<string.h>
char strmcpy(char s[100],char t[100],int m);
void main ()
{
 char t[100],s[100];
 int m,i,j;
 scanf("%s",t);
 scanf("%d",&m);

 printf("%s",strmcpy(s,t,m));


 char strmcpy(char s[100],char t[100],int m)
 {
  char s[100],t[100];
  int m;
  i=m;j=0;
  while ((s[j]++=t[i]++)!='\n')
  ;
  return s;
 }
 }
```

答：

关键是函数strmcpy()的返回值有问题，不是char，而应该是char *。

因为返回一个数组相当于返回该数组首元素的地址。 这个本周课上还会讲到。

009 为什么gets不了a?

问：

```
#include<string.h>
#include<stdio.h>
void main()
{
   char a[100],b[100];
   int i,n;
   void strmcpy( char s[],char t[],int m);

   printf("Input n:");
   scanf("%d",&n);
   printf("Input two string:\n");

   gets(a);

   printf("input another:\n");
   gets(b);

   strmcpy(a,b,n);
   puts(a);
}

void strmcpy(char s[], char t[], int m)
{
   for(; t[m]!='\0'; m++)
     s[m]=t[m];
     s[m]=t[m];

}
```

我不可以输入a,为什么的?谢谢了..

答:

因为前面%d输入后有一个回车，此回车进入到第一个gets()中。

解决办法:

1. 把两个gets()都改成:
   `scanf(" %s", ...); //注意%s前面有一个空格`
2. 在gets()前增加以下语句:
   `flushall(); //表示清除缓冲区中的数据`

010 int main()就意味着main()应该有返回值!

问:

程序填空，不要改变与输入输出有关的语句。

输入一个正整数repeat (0<repeat<10)，做repeat次下列运算：

输入2个正整数a和n, 求a+aa+aaa+aa…a(n个a)之和。

要求定义并调用函数fn(a,n)，它的功能是返回aa…a(n个a)。例如，fn(3,2)的返回值是33。

例：括号内是说明 输入

2 (repeat=2)

2 3 (a=2, n=3)

8 5 (a=8, n=5)

输出

246 (2+22+222)

98760 (8+88+888+8888+88888)

```c
#include <stdio.h>
void main()
{
   int ri, repeat;
   int i, n;
   long a, sn;
   long fn(long a, int n);

   scanf("%d", &repeat);
   for(ri=1; ri<=repeat; ri++){
      scanf("%ld%d", &a, &n);
/*---------*/
      printf("%ld\n",sn);
   }
}
/*---------*/

// 我的程序-------------------------------------------------------------


#include <stdio.h>
#include <math.h>
 int main()
 {
 int repeat,ri,k,n,a;
 long sn=0,b;
 long fn(long a,int k);
  printf("enter =\n");scanf("%d",&repeat);
   for(ri=1;ri<=repeat;ri++){
  scanf("%ld %d",&a,&n);
  for(k=1;k<=n;k++){
  sn=sn+fn(a,k);}

  printf("%ld\n",sn);
  sn=0;  }
  getch();
  }
  long fn(long x,int y)
  {
  int h=0,j;
  long ss=0;
    for(j=1;j<=y;j++)
   ss=ss+x*(pow(10,j-1))
  ;
  return(ss);

  }
```

答:

你把它原先的void main()改成int main()就应该知道，

main()里面必须有return返回一个值，否则就是错的；

main()最后的getch()不要乱用，平时调试的时候是 可以的，但是提交的时候千万要去掉，否则判题的那 台机器会一值傻等着你的程序结束，然后你就死定了！

这个上课的时候已经分析过了，对于一些严格的编译器来说， int main()是一定要有返回值的，但是对于不太严格的编译器来说，可以不要返回值， 但是实际上你没有给返回值时，系统会自动返回一个随机值。对于我们这个考试系统来说，没有返回值应该也无事。 这个贴讨论的主要是另外一个错误。

011 机考时注意每次repeat要重新对一些变量赋值

问：

程序填空，不要改变与输入输出有关的语句。

输入一个正整数repeat (0<repeat<10)，做repeat次下列运算：

输入一行字符，统计其中单词的个数。各单词之间用空格分隔，空格数可以是多个。

例：括号内是说明

输入

2 (repeat=2)

Reold building room 123

Programming is fun

输出

4

3

```
#include <stdio.h>
int main( )
{
   int ri, repeat;
   int count=0;
   char c,word=' ';

   scanf("%d", &repeat);
   getchar();
   for(ri=1; ri<=repeat; ri++)
   {
while((c=getchar())!='\n')
{
 if(c!=' ' && word==' ')
 {count++;
 }
word=c;
}
     printf("%d\n", count);
   }
}
```

为什么提交总是答案错误???

答:

低级错误，每次repeat的时候要重新让 `count=0` ， 还有 `word=''`

012 二维字符数组的赋值与输入问题

问1:

问一下，数组中的元素能不能是一个数组？ 比如有五个字符串，如何进行排序？就是从大到小?

答1:

用char类型的二维数组或者char *类型的指针数组，例如:

```
char a[5][10]={"AAA", "BBBB", "CCCC", "DDDD"};
char *p[5]={"AAA", "BBBB", "CCCC", "DDDD"};
```

问2:

我看书本上是先对这个二维数组赋初值，那如果要输入五个字符串作为这个二维数组的元素，那应该怎么输入？又是如何将这五个字符串赋值给一个指针数组?

答2:

例如:

```
char s[5][10];
int i;
for(i=0; i<5; i++)
   gets(s [ i ]);
```

例如:

```
char s[5][10];
char * p[5];
int i;
for(i=0; i<5; i++)
   gets(s[ i ]);
for(i=0; i<5; i++)
   p[ i ] = s[ i ]; 或 p[ i ] = &s[ i ][0];
```

013 逗号表达式、strcmp、*++k、数组名赋值

问1:

1 若变量已正确定义，表达式 (j=3, j++) >

（因为非法，才不加吗，还是逗号表达式？) 的值是__?

A、3 B、 4 C、5 D、0

答1:

这个是逗号表达式，j++这个表达式的值是j的原值，所以结果是3。

问2:

表达式strcmp(“box”, “boss”) 的值是一个___。A

A、 正数 B、 负数 C、 0 D、 不确定的数

答2:

这个上课的时候讲过的，strcmp是对两个字符串的各个字符逐个做减法运算，一直到 差不等于0或者两个字符串全等为止。显然'x'- 's'是一个正数

问3：

```
int c[]={1, 3, 5};
int *k=c+1;
printf("d", *++k);
```

不懂*++k是什么意思，*k-->c[1],那么k是什么啊？不是说不能int *k;*k=&a[1]吗?

答3：

这个同样在上课的时候强调过好几遍了，

int *k=c+1 并不是把c+1赋值给*k，而是赋值给k。

int 与 * 结合在一起表示指向整数的指针类型。

++*k相当于*(++k)，这里++k的值等于k+1以后的值，即c+2，所以*++k的值就是5。

问4：

C、 `char str[10];str="string";` 为什么错?

答4：

唉，这个上课的时候不幸也讲过了，数组名相当于数组首元素的地址，是一个常数，不能被改变， 也不能被赋值的。

014 模拟题“十六进制字符转换” 问题及解答

问：

模拟题“十六进制字符转换” 问题及解答

答：

```
#include "stdio.h"
#define MAXLEN 80
int main( )
{  int i,k,j;
   long number;
   char ch,oldch,str[MAXLEN], num[MAXLEN];

   oldch=' ';
   while((ch=getchar())!='#'||oldch!='#'){
      k=0;
      while(ch!='#'&& k< MAXLEN-1){
             str[k++]=ch;
             ch=getchar();
      }
      oldch='#';      str[k]='\0';
           number=0;j=0;
        for(i=0;i<k;i++){
          if(str[i]>='a'&&str[i]<='f'||str[i]>='A'&&str[i]<='F'||str[i]>='0'&&str[i]<='9'
              num[j++]=str[i];
            /*   *num++=str[i];
                   *num=0;
                   指针为啥不对,显示lvalue required in function main?
                  答：因为num是数组名，是常数，不能对它进行++运算的。
             */
         num[j]='\0';
        for(i=0;num[i]!='\0';i++) {
           if(num[i]>='a'&&num[i]<='f') number=number*16+num[i]-'a'+10;
           if(num[i]>='A'&&num[i]<='F') number=number*16+num[i]-'A'+10;      /*这里死循环，
           if(num[i]>='0'&&num[i]<='9') number=number*16+num[i]-'0';
                  }
        }  <--------------关键在于你这个括号的位置错了!
           如果你这个}是对应前面那个for(i=0; i<k; i++)的话,
           那么两个嵌套的for用同一个i来控制，不死才怪呢。

      printf("%ld\n",number);
    }
}
```

015 建立和编译多个文件构成的程序及全局变量跨文件共享

问1：

建立和编译多个文件构成的程序及全局变量跨文件共享，显示 Error Unable to open dependency file 2. Press ESC. 请教是什么原因？该怎么解决?

答1：

应该是一个工程包含多个程序文件的问题。

在TC中按以下步骤做：

1. File->New
   输入以下两行 1.c 2.c 并保存，文件名设为1and2.prj
2. File->New 输入1.c文件的内容并保存
3. File->New 输入2.c文件的内容并保存
4. Project->Project Name 输入 1and2.prj
5. Compile->Build all 例如1.c的内容如下：

例如1.c的内容如下：

```
#include <stdio.h>
int g=100;
int f(int n);

void main()
{
    int x=123, y;
    y = f(x);
    printf("y=%d, g=%d\n", y, g);   /* 输出y=-123, g=101 */
}
```

2.c文件的内容如下：

```
extern int g; /* 这个是定义在1.c文件中的全局变量g，这里加上extern就可以引用它了 */
int f(int n)      /* 这个函数f可以在1.c文件中被调用 */
{
   g++;
   return -n;
}
```

问2：

这里不要用个文件包含吗？我是说#include <stdio.h>后面不要加#include “2.c”吗？变量那一节概念感觉好模糊的，求解！谢谢

答2：

包含的话，就是复制粘贴，相当于一个文件了，因此根本不需要用extern声明那个全局变量。 分成两个.c文件，再分别compile，然后link合并成一个exe时，其中一个文件中的 全局变量必须在另外一个文件中用extern声明才能引用。

016 定义在fun()与main()之间的全局变量

问1：

```
#include<stdio.h>
void main()
{
     int x=3,y=6,z=0;
     while(x++!=(y-=1)){
        z++;
        if(y<x) break;
     }
     printf("x=%d,y=%d,z=%d",x,y,z);
}
```

我觉得z=2,可结果x=5,y=4,z=1; 为什么z=1啊?

问2：

设程序中不再定义和声明变量，写出在函数fun中可以使用的所有变量

```
#include <stdio.h>
    void fun(int x)
{  static int y;
       ……
       return;
    }
    int z;
    void main( )
    {   int a,b;
        fun(a);
        ……
    }
```

A.x,y B.x,y,z C.a,b,y,z D.a,b,x,y,z

我选B,答案是A. Z不是全局变量吗,为什么不能在fun中使用?

问3：

请仔细阅读函数f1，然后在函数f2中填入正确的内容，使函数f1和函数f2有相同的功能。

```
int f1(char s[ ])
{  int k=0;
   while(s[k]!='\0')
      k++;
   return k;
}

int f2(char *s)
{  char *ss;
     ss=s  ;
   while(*s++!= '\0'  )
       ;
   return  s-ss-1 (11)   ;
}
```

11中我写的是s-ss+1,为什么答案是s-ss-1?

答:

你跟踪一下，循环结束时，s落在哪个位置？是'\0'处，还是'\0'之前，还是'\0'后面？ 然后就明白为什么答案是这个了。 到目前为止还没有学会跟踪的，到头来终究是没有好下场的，嘿嘿。

017 floating points format not linked

问:

写代码是遇到了这样的情况。。。。。。

如果是

```
#include<stdio.h>
struct student{
 int num;
 char name[20];
 float score;
};
struct student stud[10];

void main()
{
 int n,i;
 float ave=0;

 scanf("%d",&n);
 for(i=0;i<n;i++){
  scanf("%d%s%f",&stud[i].num,stud[i].name,&stud[i].score);
 }
 for(i=0;i<n;i++)
  ave+=stud[i].score;
 ave=ave/n;
 printf("The average score is:%f\n",ave);
}
```

编译没问题，但是在给name分量赋值时就会出错，系统显示：

scanf:floating formats not linked

Abnormal program termination

但是如果我把结构中的第三个分量改成int型，就一切正常了。 为什么啊？？？

答：

这个是TC的一个bug，你随便再定义一个函数，比如：

```
void f(void)
{
   float x;
   scanf("%f", &x);
}
```

然后把此函数定义放到main()之前，或者放到main()后面，但前面加上此函数原型即可。

这个问题的主要原因是TC为了尽量节约内存空间，所以如果它“觉得”你的程序中不需要输入小数的话，就不把小数输入的功能编译进去，但是有时它会“失手”，比如结构成员是小数，或者二维数组元素的小数等情况下，它会判断失误。

018 关于TC的调试过程演示(有声版)

问：

关于TC的调试过程演示(有声版)

答：

我早就放到“知识要点”里面了，有些同学就是不去看，上机的时候 找不到错误就瞎改程序，或者坐着等我过来解答。这么多人上机，我 转悠没有几圈，就下课了。 有些同学抱怨说演示速度太快，看不明白。我现在重新制作了一下， 是有声的，应该比较清楚了。 求人不如求已，请下载下来观看，一定要学会：http://blackwhite.8866.org/bhh/step.exe 点工具条或者点右键，可以全屏观看，而且进度条可以前后拉的。试一下就知道该怎么用了。

019 偶做的05年答案(大家帮忙改下)

问：

偶做的05年答案(大家帮忙改下)

一 1B 2C 3D 4B 5A 6A 7B 8C(对宏定义挺有把握的说，居然还是错了，大家也要看仔细啊。) 9C 10 D

二 1 24,24,c 2 ? 3 035 4 1,4,8, 5 41 6 x=4,y=5,z=1 7 3 8 double sum(double x,y) 形参类型应分别说明

三

1. k=6,x=0
2. 8 5 8 7 5 8 7 5 2 8 7 6 5 2
3. 54321
4. Happy

四

1. (1) 1.0*flag/k (2) flag=-flag
2. (3) FILE *fp (4) k2.txt (5) (ch=getchar())!='@' (6) fclose(fp) (7) "\c",ch
3. (8) n*1000(“左移三位”啥意思？) (9) m/=2 (10) j=i-1;j>=0;j++

五

1.
```c
#include<stdio.h>
int isright_triangle(float a,float b,float c)
{
   if(a+b>c&&b+c>a&&c+a>b)
        if((a*a+b*b-c*c)*(b*b+c*c-a*a)*(c*c+a*a-b*b))
               return 0;
        else   return 1;
   else
           return 0;
}
void main()
{
    float a,b,c;
    int x;
    printf("Input a,b,c\n");
    scanf("%f%f%f",&a,&b,&c);
    x=isright_triangle(a,b,c);
    if(x)
       printf("%f,%f,%f can form a rightangle.\n",a,b,c);
    else
       printf("%f,%f,%f can not form rightangle.\n",a,b,c);
}
```

2.
```
#include<stdio.h>
double f(char*p)
{
    double x;
    int flag=0;
    float i=10,j=1;
    if(*p<'0'||*p>'9')
        return 0.0;
    else x=*p-48;
    for(p++;*p;p++){
        if(*p>='0'&&*p<='9'){
              x=x*i+(*p-48)*j;
              if(flag==1)
                   j/=10;}
        else if(*p=='.'&&flag==0){
              i=1;
              j=0.1;
              flag=1;}
         else return x;
     }
    return x;
}
void main()
{
    char str[20];
    double s;

    printf("Input string:\n");
    gets(str);
      if(str[0]=='+')
           s=1.0*f(str+1);
      else if(str[0]=='-')
           s=-1.0*f(str+1);
      else s=f(str);
    printf("%f",s);
}
```

答:

020 机考题中字符输入相关问题及解答

问:

比如这道，现在情况不一样了，提交以后，他就死等在那里，并且我保证不是死循环。因为测 试都是对的阿。

```
#include <stdio.h>
int main( )
{
   int ri, repeat;
   int op1, op2, res;
   char operator;

   scanf("%d", &repeat);
   for(ri=1; ri<=repeat; ri++){
     scanf("%d", &op1);
     operator = getchar();
     while(operator!='=')
         {
           scanf("%d",&op2) ;
           if(operator=='+')  res=op1+op2;
           if(operator=='-')  res=op1-op2;
           if(operator=='*')  res=op1*op2;
           if(operator=='/')  res=op1/op2;
           op1=res;
           operator=getchar();

         }
/*---------*/
     printf("%d\n", res);
   }
}
```

答:

把

```
operator = getchar();
```

改成:

```
scanf(" %c", &operator);
```
是不错的选择。

注意%c前面有一个空格。

021 怎样把十进制转化为二进制呢?

问:

怎样把十进制转化为二进制呢?

答:

```
void dectobin(int n)
{
        if(n/2==0)
                printf("%d",n);
        else{
                dectobin(n/2);
                printf("%d",n%2);
        }
}
```

022 表达式等价、指针的指针、移位

问1：

如：1 .表达式！（x+1）等价于____.A X!=0 B X0 C X!=-1 D X-1 不懂是什么意思。

答1：

就是要求把任何值代入X中后，两个表达式的值应该相等。 所以答案显然是D。

问2：

还是不太懂 `int a[100],**p=a;` 是什么意思。能理解 `*p=a;<==>int *p;p=&a;` 而 `int **p=a;` 是否是 `int**p;p=&a;` ？

答2：

这个题目本身是错的。 int **p = a 语法错误。 如果写成这样是对的： int *a[100], **p=a; 上课讲过，指针数组名相当于指针的指针。

问3：

在笔试中，回车要不要写出来？用中文"回车"还是用回车的‘\n’

答3：

一般用以下这个符号表示回车： <↵

问4：

<< >> 移位运算中，是否对各种进制都是用。如1234左移2 位就是3400，还是123400。而在二进制中，移动后超出存储空间的就自动舍去。如1001 1101 移2位就是0111 0100？在书上没有找到相关的内容，希望老师讲讲。 此外，& ^ | 的意思不大明白。

答4：

移位是针对二进制的，如果要对十进制数进行移位，必须先把它们转化成二进制才行。 移位是有可能超过存储空间的，左移或右移的时候所有移出的部分都被丢弃。

1001 1101 左移2位后确实等于 0111 0100 关于& ^ |请看书上278页，或者看一下我主页上“知识要点”中的例子，这个我记得上课的 时候讲过的，而且在大学计算机基础课中也应该有相关的内容。

023 自动类型转换、typedef、异或

问1：

自动内型转换中，signed->unisigned,short->long,这是如何进行的。

例如：

```
signed int a=-10;
unsigned int b=12, c;
c=a+b;
```

计算机是如何计算a+b的？(unsigned)c又等于多少?

答1：

signed -> unsigned 与 short -> long都是对符号数进行位数扩充。

被转换的数如果是符号数就是符号扩充，被转换的数如果是非符号数就是非符号扩充。

符号扩充规则：扩充增加的各个位=原数的最高位

非符号扩充规则：扩充增加的各个位=0

例如：

```
    char a=-2;
    short int b;
    b = a;
扩充过程如下:
    -2 = 1111 1110 (二进制)
              ||
              \/
    -2 = 1111 1111 1111 1110
```

```
    char a=2;
    short int b;
    b = a;

2 = 0000 0010
      ||
      \/
2 = 0000 0000 0000 0010
```

假定下面例子中int是16位的：

```
signed int a=-10;
unsigned int b=12, c;
c=a+b;
```

此处a与b相加，符号数a服从非符号数b，所以结果如下：

```
 a = - 10 =  1111 1111 1111 0110 = 65526
              b =            0000 0000 0000 1100 = 12         +)
-------------------------------------------------------------------------------
              c =             0000 0000 0000 0010 = 2
```

问2：

`typedef int (*A[10])();` 是什么意思？特别是最后一个括号。

答2：

A是一个数组，共10个元素，每个元素都是一个指针，每个指针都指向一个函数，该函数无参数但返回 一个int。这里的()指是就是一个函数。

问3：

$b^{(a}b)$等价于$a^{b}b$? $(a^{b)}(b^{a}b)$等价于$a^{a}b^{b}b$?

答3：

异或运算支持交换律的。 所以上述两式是等价的。

024 关于printf的%d输出格式问题

问：

是不是在VC++中printf("%d",a);与printf("%ld",a);等价（其中a为长整型）？ 而在TC中printf("%d",a);语句对于短整型的a和一般整型的a输出结果一样， 对于长整型的a则不同?

答：

对的。理解正确。

其实%d用来输出int类型的值，因此在TC中就是输出short int的值， 在VC中就是输出long int的值。

另外，在VC中如果要输出short int的值，要用%hd格式。 h表示short，呵呵。

025 一个问题while(*s++) putchar(++s); 很懊恼??

问：

```
include<stdio.h>
void main()
{

  char ss[10]="12345",*s=ss;

     while(*s++)  putchar(*++s);
          if(*(s-2)=='''\0''')  printf("%d",2);

 }
```

结果怎么是 35 2_ (_ 表示光标的位置) (跟自己的计算的答案不一样)
把中的 `if(*(s-2)=='''\0''') printf("%d",2);` -2 改为-3后为 35 _ (跟自己的计算一样)
把 `if(*(s-2)=='''\0''') printf("%d",2);` 中的 -2 改为-1后为 35 2_ (跟自己的计算一样)
把 `if(*(s-2)=='''\0''') printf("%d",2);` 中的 -2 改为0后为 35 2_ (跟自己的计算一样)

把 `if(*(s-2)=='''\0''') printf("%d",2);` 中的 -2 改为+1后为 35 2_

为什么啊? -2的时候应该是 35 _???

答：

这是一个烂题目。答案应该是不确定的。

分析如下：

`char ss[10]="12345", *s=ss; while(*s++) putchar(*++s);`

假定"12345"后面的3个字符分别用ABC表示：

12345[A][B] [C]

第1次循环：

`while(*s++)` 后，s指向'2'

`putchar(*++s)` 后，s指向'3'

第2次循环：

`while(*s++)` 后，s指向'4'

`putchar(*++s)` 后，s指向'5'

第3次循环：

`while(*s++)` 后，s指向[A]

`putchar(*++s)` 后，s指向[B]

第4次循环：

`while(*s++)` 前，s指向[B]，此时如果[B]=='\0'则条件为假

`while(*s++)` 后，s指向[C]

关键在于[B]是否肯定为0。

对于非static存储类别的变量或者数组， 如果没有初始值，则它们的值是不确定的。

显然[B]与[C]的值是不确定的，[A]肯定为0。 虽然对于大多数编译器来说，它们会把没有 初始值的数组赋值为0，但这个不是C语言的 标准。 假定[B]==0，那么最后s指向[C]，所以， `*(s-2) == 0` 在第3次循环时会输出'\0'，屏幕表现为一个 空白或一个小方块。 假定[B]!=0，那么最后s指向哪里就不确定了。

026 sizeof()问题求教!

问1：

为什么sizeof("\nsum=%d\n")的值是九?

答1：

\n 1个
sum= 4个
%d 2个
\n 1个

\0 1个 +)

9个

问2：

下列程序的输出结果：

```
int a=1;
long b=a*ox12345678;
printf("%x\n",b);
```

答案有12345678或5678。请问5678怎么来的?

答2:

%x表示以16进制格式输出int类型的值。
在TC中int是16位，在VC中int是32位。
所以就有两种结果。
在TC只输出32位中的低16位即5678,
在VC中输出全部32位即12345678。
注意在内存中，位数大于8位的数都是按
低字节在前，高字节在后规则存放的。
0x12345678在内存中存放如下所示:
地址 值
1000 0x78
1001 0x56
1002 0x34
1003 0x12

027 五个问题(sizeof、void、异或、移位等)及解答

问1:

函数声明里void fact（void）；是不是说该函数没参数啊?

答1:

该函数无参数，并且无返回值

问2:

sizeof（）是不是只计算结构的位数大小?

答2:

sizeof()用来计算任何对象或者类型占用内存的字节数。 不是仅对结构求大小。

问3:

stdlib里的函数是不是都不要管啊，貌似去年试卷里有出现了这个讨厌的stdlib。

答3：

一般是不需要的。有些函数比如malloc()、free()、random()之类跟这个头文件有关。

问4：

异或运算中a=b是不是等价于a=ab?

答4：

对的

问5：

左移运算跟符号有关吗?

答5：

左移跟符号无关，但是右移与符号有关。
例如：
char a=0xFE; /* 相当于a=-2 /
a = a << 1; / 结果a = 0xFC; 相当于a=-4 */
移位过程如下：
a的二进制原值= 1111 1110
左移1位后的值= 1111 1100

例如：
char a=0xFC;
a = a >> 1; /* 结果a=0xFE; 相当于a=-2 */
移位过程如下：
a的二进制原值= 1111 1100
右移1位后的值= 1111 1110
注意：通常情况下(如正数或者非符号数)，右移1位意味着左边要补0，但是上例中左边是补1的。
这是因为对于负数来讲，补1可以保证移位后的值仍旧是负数，如果补0就会变成正数。
这些知识有点超过C语言的范围了，所以上课的时候我没有深入讲解。
在汇编语言中要涉及这些知识的。

028 问个C语言笔试题:字符指针数组与指针的指针

问：

9. 下列程序段的输出是_____。

```
char *a[2]={"one", "two"}, **p=a;
printf("%s,", *(p++)+1);
printf("%c\n", **p-1);
```

答：

输出ne,s

029 关于打开文件时的文件路径

问：

文件打开时if(fp=fopen("k2.txt",'r')==NULL){ .....} 如果题目告诉你了k2.txt是哪的文件要不要写路径阿，要写的话是用双斜杠“//”还是用单斜杠“/”表下一级目录呢

答：

如果有路径的话，要用\ 比如："C:\windows\abc.txt"

030 求问:结构指针

问：

13. 对于以下的变量定义，表达式_____是正确的。

```
struct node {
        char s[10];
        int k;
} p[4];
```

A．p->k=2 B．p[0].s=“abc” C．p[0]->k=2 D．p->s=‘a’ 答案为什么是A

答：

正确答案 确实是A。
B 中 s是数组名，不能被赋值，所以错。正确写法： strcpy(p[0].s, "abc");
C 中 p[0]是一个结构变量，后面要跟.不能跟->
D 中 s是数组名，同样不能被赋值。正确写法：*(p->s) = 'a'

A中，p是数组名，所以相当于 &p[0]，
p-> 相当于 (*&p[0]). 即 p[0].
因此 p->k 相当于 p[0].k

031 提问:关于结构

问:

对于以下的变量定义,表达式____是正确的.

```
struct node{
  char s[10];
  int k;
}p[4];
```

A.p->k=2 B.p[0].s="abc" C.p[0]->k=2 D.p->s='a'
答案是A,这个懂的,可是如果要把B改对的话,应该怎么改啊??是不是 `p[0].*s="abc"` ??

答:

p[0].s是数组名，不能直接赋值的。 上课讲过的，你再好好想一想。

"abc"相当于一个地址，地址可以赋值给*s吗?
正确用法:
strcpy(p[0].s, "abc");
或者
*p[0].s = 'a'; 或 p[0].s[0] = 'a';
*(p[0].s+1) = 'b'; p[0].s[1] = 'b';
*(p[0].s+2) = 'c'; p[0].s[2]='c';
*(p[0].s+3) = '\0'; p[0].s[3] = '\0';

听课不认真，要打PP。

032 help: x%=x++/--y

问:

3.设 `int x=17,y=5;` 执行语句 `x%=x++/--y` 后x的值为 。

在最后的一步中x取? 为什么?

答:

这个题跟以前讲过的 如下这种容易引起歧义的题差不多:
y = ++x + x++ + x++;

关键在于等于左边的x会不会被右边的x++影响。
受影响 与 不受影响 就会产生两种结果。
在TC中，经测试，等号的x被右边的x++影响，
导致x=18，最后18%4，结果是2。

希望这次考试不要出这种垃圾题。

033 问题:for(c=getchar(); getchar()!='#'; c=getchar())

问：

输入12345#后，写出下列程序的输出结果。

```
void main()
{
    char c;
         for(c=getchar(); getchar()!='#'; c=getchar())
          putchar(c);
}
```

答：

要注意到for循环的使用，相当于：

```
c=getchar();
while(getchar()!='#'){
    putchar(c);
    c=getchar();
    }
```

所以是13
第一步：c='1'
第二步：空读一个getchar()='2' 输出1 c='3'
第三步：空读一个getchar()='4' 输出3 c='5'
第四步：空读一个getchar()='#' 跳出循环

034 问逻辑运算

问1：

设c='m',x=5,y=5,z=0

1. !x-y!=!-x-y
2. x&&(y=5),x&&z&&(y=1),y
3. x||(z=1),(z==1)||(x=0),x||(z=2),z

答案分别是 1 5 2 要怎么做啊,还有逗号是用来干什么的呀

答1:

!x - y != !-x - y

0 - 5 != 0 - 5

结果是假，所以答案=0

x&&(y=5) , x&&z&&(y=1), y
5&&5 , 5 && 0 &&(y=1), y
1 , 此处前面一半5&&0=0所以后面一半(y=1)不执行, 5
最后的值=5

x || (z=1), (z==1)||(x=0), x||(z=2), z
5 不执行, 0 || 0 , 0 || 2 , 2
最后值=2

归纳一下:
如果&&前面表达式的值=0，则&&后面不执行;
如果||前面表达式的值=1，则&&后面不执行。

问2:

还有:

```
int x=-10,y=0;
if(x>=0)
if(x==0) y=1;
else y=-1;
printf("%d",y);
```

答案是0

答2:

```
int x=-10,y=0;
 if(x>=0)
 {
     if(x==0) y=1;
     else y=-1;
 }
 printf(&quot;%d&quot;,y);
```

这样你应该可以看清楚了吧?

035 请问：typedef int NUM[100];

问1：

请问：

`typedef int NUM[100]; NUM arrey[10];` 这算不算一个合法的C语句，是不是定义了一个数组，数组元素还是数组？如果不是，那应当怎么来定义一个元素是数组的数组?

答1：

这类问题其实很简单的,
只要把新类型后面所跟的东东 代入 原先类型出现的位置 即可：
int NUM[100]
||
||
/
int arrey[10][100];

问2：

顺便问下

用typedef定义一个指向函数的指针类型FP，该函数的返回值为整数。

答案： `typedef int (*FP)();`

这个答案是怎么回事呢？怎么用它定义别的变量呢?

答2：

这个答案是正确的。
在

int (*FP)();
中

FP是一个指针变量，它指向一个函数，该函数返回一个整数。
按先看右边再看左边的方法来读即可。

036 问题：#define FB(a,b) (a*b+1)

问：

8. 执行下列程序：

```
#define FB(a,b) (a*b+1)
int k=3;
k=FB(FB(k,k+1),k)-7;
```

后变量k的值是___C___。 A、7 B、24 C、27 D、33

答：

注意(a*b+1) 是加了括号的，但是a、b本身没有加括号。

FB(k,k+1) = (k * k+1 + 1)
FB(FB(k, k+1), k) = ((k * k+1 + 1) * k + 1)
= 11*3+1 = 34

所以答案是27

037 结构指针指向字符指针问题

问：

15. 对于以下的变量定义，表达式_B____不符合C语言语法。

```
struct node {
    int len;
        har *pk;
        } x = {2, "right"}, *p = &x;
```

A、p->pk B、*p.pk C、*p->pk D、*x.pk
为什么选B？我认为B、C、D都有问题嘛？

答：

此题不幸也回答过了。
请在提问前看精华区，或者搜索关键词。
还有，提问的标题一定要表达你想问的问题，
不要笼统地“问一个问题”之类。

C. *p->pk
注意 p->pk 先结合，是正确的。
所以 *p->pk 相当于
(p->pk)
*而pk本身是一个指针变量，那它前面加个*当然是正确的。

D. *x.pk*
同样 x.pk 先结合，是正确的。
*而pk是一个指针变量，它前面加个*同样是正确的。

A. p->pk
这个正确，就不用说了。

B. *p.pk
因为 p.pk 先结合，而p是结构指针，后面不能跟. 只能跟-> 所以是错的。

038 关于%u?

问：

请问%u到底十六进制还是十进制？ 书上好像是前后矛盾的……

答：

%u表示以十进制格式输出 unsigned int类型的值。
如果要用16进制格式输出，必须用%x与%X，同时注意
16进制格式输出时，没有前缀0x的。比如printf("%X", 32767)输出7FFF

比如：
unsigned int a = 65535;
printf("%u", a); 输出65535

int b = -1;
printf("%u", b); 输出65535
此处表示把-1看作非符号数，所以就是65535了。
其实16位符号数 -1 与 16位非符号数65535 在计算机中都是以以下二进制形式存在的：
1111 1111 1111 1111

039 二分法的例子程序

问：

我知道二分法的原理，但是不会编程 eg.编写函数 find(a,n,x)，采用二分法在数组a的前n个元素中查找x，如果找到，返回x在 数组a中的位置；如果没有找到，返回0。

答：

二分法要求数组a中所有元素已按小到大顺序排列。以下为例子：

```
int  find(int a[], int n, int x)
{
   int left, mid, right;
   left = 0;                   /* 首项下标 */
   right = n-1;              /* 末项下标 */
   while(left <= right)
   {
      mid = (left+right)/2; /* 中间项下标 */
      if(x == a[mid])
         return mid;             /* 相等就返回 */
      else if(x < a[mid])    /* 若在左边就把末项下标设成mid-1 */
         right = mid - 1;
      else if(x > a[mid])    /* 若在右边就把首项下标设成mid+1 */
         left = mid + 1;
   }
   return -1;                    /* 找不着就返回-1 */
}
```

040 变量定义 int p[5], *a[5] 及%s输入

问1：

10. 对于变量定义 int p[5], *a[5];，表达式_____是正确的。
    A．p=a B．p[0]=a C．(a+1)=p D．a[0]=2
    答案为什么是C呢?
    感觉B,D也都是对的.
    B是把a的地址赋值给变量p[0].
    D是把2作为地址赋值给a[0].

答1：

先要搞清楚int *a[5]定义了什么东东。
注意数组a中每个元素a[i]都是指针变量，类型是: int *，即可以把

任何一个int类型的变量的地址赋值给a[i]。
A. p=a 肯定错，因为p是数组名，不能被赋值
B. p[0]=a 肯定错，因为p[0]是一个int，a是数组名即地址，类型不匹配。
C. *(a+1)=p 正确，因为*(a+1)相当于a[1]。p相当于&p[0]。
D. a[0]=2 肯定错，因为a[0]是一个指针变量，不能赋值为一个整数2。

问2：

还有就是 `char *s; scanf("%s",s);` 是错的. `scanf("%d",s);` 应该是正确的吧? 输入的是s的地址吧?

答2：

这个问题上课的时候讲过了，scanf("%s", s)或者gets(s)
要求s指向一个明确的内存空间，如果s未知，输入内容就会
存放到地址不确定的内存空间中。

%d格式不能用来输入一个字符串，只能用来输入一个整数。

041 问动态创建二维数组的问题

问：

问一下小白, 如果我想开一个不定大小的二维数组不可以这样吗?

```
double **Graph=NULL;

*Graph=(double **)malloc( n * sizeof(double *));

for (i=0;i<n;i++){
   Graph[i]=(double *)malloc(n*sizeof (double));
}
```

结果总是不行...指针有问题...

答1：

```
#include <stdio.h>
#include<stdlib.h>
void main()
{
    int **a;
    int i,m,n;
    scanf("%d %d",&m,&n);
    *a=(int **)malloc(m*sizeof(int *));
    for (i=0;i<m;i++)
    a[i]=(int *)malloc(n*sizeof(int));
}
```

这个是网上搜来的ms也有问题，小白老师帮忙看一下

答2:

1楼显然不对，**是指针的指针了，不是二维……

要达到你的效果，只有用一维数组来模拟二维的数组，可以参考这里http://www.cc98.org/dispbbs.asp?boardID=23&ID=1370769&page=4

多位数组在内存上的存储实际上仍然是一维的。所以取值计算即可 而对于static变量，int型初始化为0

答3:

1 与 2都有一个同样的错误：野指针
int **a; 定义了一个指针的指针，
然后在没有对a本身赋值的情况下，却对*a进行赋值就是野指针。
(1) 假定每行列数是常数，则可以用数组指针来实现；
(2) 假定每行列数是变量，则需要用指针的指针来实现。

(1) 假定每行列数是4，行数n，则：

```
int  (*p)[4];
p =  (int (*)[4]) malloc(sizeof(int)*4*n);    /* p指向由4列构成的首行 */
p[0][0]=1;
```

(2)假定m行，n列，则：

```
int  **p;
int i;
p = (int **) malloc( sizeof(int *) * m );      /* p指向 m个指针中的首个 */
for (i = 0; i < m;  i ++)
{
   p [ i ]  =  (int *) malloc(sizeof(int) * n); /* m个指针中的每一个指针都指向一维数组的首列 */
}                                                /* 该一维数组由n列构成 */
p[0][0]=1;
```

042 c语言中两个浮点数能比较大小么?

问:

0.1和（0.02＋0.08）的结果好像是不一样大的?

答1:

确实，因为十进制转二进制时舍入会有小错误。 比较浮点数一般用(a-b)<.0000000001之类的方法

答2:

定义一个宏:

```
#define M 1e-8

#define ZERO(x) fabs(x) < M
```

然后判断是否是零只需要if(ZERO(a - b))

043 问题解答：void main()与int main()的区别

问1:

int main 和 void main 有何区别?

答1:

int main表示main要返回一个整数，但是 大多情况下，main函数最后都没有用return 返回一个整数，在这种情况下，系统会自动 返回一个随机的值，此值返回给操作系统后 也是自动被丢弃的，所以没有大碍。
void main表示main无任何返回值。

问2:

程序:

```
#include<stdio.h>
#include<math.h>
int main()
{
    int m, y, r, i;
        scanf("%f", &m, &y, &r, &i);
        i = m * pow(r, y);
        printf("%f\n", i);
}
```

运行之后老显示overflow,问题是出在哪里呢?

答2:

因为你企图用%f输入整数值。

044 问个关于 指针与数组的问题

问:

数组名到底代表了什么?

没事的时候，自己编了个例子，让自己一直很疑惑:

```
#include <iostream>
using namespace std;
void main()
{
    char a[10];
    char *b;
    b=a;
        cout<<"sizeof(a)="<<sizeof(a)<<endl; //1
        cout<<"sizeof(b)="<<sizeof(b)<<endl<<endl; //2

        cout<<"a="<<hex<<(long)a<<endl; //3
        cout<<"&a="<<hex<<&a<<endl<<endl; //4

        cout<<"b="<<hex<<(long)b<<endl; //5
        cout<<"&b="<<hex<<&b<<endl<<endl; //6

   //cout<<"&(&a)="<<hex<<&(&a)<<endl; //7
}
```

结果第1行输出10，第2行输出4;
第3行输出12FF74,第4行输出0012FF74;
第5行输出12FF74,第6行输出0012FF70;

第1，2行得出“数组名不完全是个指针常量”，至少类型上不是指针类型。
（那还有哪些其他意义？）

第3，4行说明 a作为一个数组名，a和&a在数值上相等。且a有内存的分配。
（我们知道一个数组的大小＝单个元素大小*元素个数,那么&a存放在哪？）

第5，6行说明 对于一般意义的指针，b和&b不相等。

第7行如果把//去掉（即也参与编译），则编译错误，当a是数组名时，
a和&a不是同个东西。（否则应该可以迭代的取a的地址）

怎么回事呢？谢谢各位大侠了:)

答：

```
cout<<"sizeof(a)="<<sizeof(a)<<endl; //1
cout<<"sizeof(b)="<<sizeof(b)<<endl<<endl; //2
```

用sizeof求长度时，如果对象是数组名，则求出的就是该数组整体占用内存的字节数， 而不会把该数组名当作指针来看求出指针的长度。但是，如果你把数组作为参数传 给某个函数，则该函数对传入的数组名求sizeof时得到的是指针的长度，因为数组 作参数实际传递的是数组的首地址，不是整个数组。 如果sizeof的对象是一个指针，则求出的长度就是该指针在内存中占用的字节数， 与该指针指向什么东东无关。

```
cout<<"a="<<hex<<(long)a<<endl; //3
cout<<"&a="<<hex<<&a<<endl<<endl; //4
```

3这里的a应该理解为数组a的首元素的地址，而不是数组本身，即a相当于&a[0];

4这里的a应该理解为数组a本身，于是&a就是数组a的地址。&a的值虽然与a或&a[0]相同， 但是类型不同，如果定义两个变量分别接收a与&a，则应该这样定义： char *p = a; char (*pa)[10] = &a; 也就是说，前者是字符指针，而后者是指向数组的指针。 另外，&a[0]及&a都是常数，在内存中并没有空间存放它们的。

```
cout<<"b="<<hex<<(long)b<<endl; //5
cout<<"&b="<<hex<<&b<<endl<<endl; //6
```

这里就容易区分了。指针变量本身的地址与指针变量的值即它所指向的对象的地址 当然不同。

```
//cout<<"&(&a)="<<hex<<&(&a)<<endl; //7
```

&a已经是一个地址值，即它是一个常数，所以再对它取地址是错误的。

045 #和*的游戏~

问：

文件下载

1.c里*走一步#也走一步

2.c里#隔整数秒（#define TIME 的值）一步

感觉算距离的函数限制比较大，地图只能这样：从上往下数，空行在偶数行，有障碍的行在奇数行，而且障碍行的通道只有一个字符宽

再就是2.c里，不知道怎么让#移动间隔小于1秒

ctstp()－－算两个坐标之间的步数，用的递归。 1．偶数行（也就是在弯里）的点，归纳成更近的奇数行的点，重新计算，结果+1； 2．两点在同行或差一行，返回的距离是横坐标差+0，1或2（由奇数/偶数行决定）； 3．差两行以上，寻找下面的点上一行所有的弯，对每个弯，拿它的坐标代替下面的点计算，得出结果+下面点到弯的距离，返回各个弯对应的最小值。 思路就是这样啦。后来把引起递归的条件放到前面，又加上第一个点高于第二个点时交换位置的步骤来减少判断，就成了这个函数。

答1：

赞一个， #会转弯去追* ，这个比较强。 粗略地看了一下你的程序，感觉不是新手所为。 有时间的话，给大伙解释一个函数ctstp()里的 思路。

答2：

这个~~~偶有一个想法~~~干脆不用什么time~~~在电脑的每两次移动之间插一个\a好了~~~满足大家的听觉享受~~~

小白和大家玩过Popcap的木乃伊吗？幻想游戏3.1里的~~~那个倒和现在的#*游戏比较接近~~~它的主要规则是~~~

回合制~~~没有豆子，目标是探险家走到出口~~~对方的移动有固定的规律（白色木乃伊优先从横向接近，红色木乃伊优先从纵向接近~~~等等）~~~我方的探险家走一格木乃伊走两格~~~主要思路是用墙把对方限制住~~~

046 int (*a)[80]这是不是指向二维数组的写法?

问：

int (*a)[80]这是不是指向二维数组的写法?

答：

*a是一个指向数组的指针，这个数组是一个有80个元素的int型的数组

例如：

```
int main()
{
  int (*a)[10];
  int b[10]={1,2,3,4,5,6,7,8,9,10};

  a=&b;
}
```

这样，(*a)就相当于b，(*a)[0]就相当于b[0]

047 问小白老师，如何区分Shift等功能键的左、右?

问：

键盘扫描码里面，左右Shift、Ctrl、Alt的扫描码一样的，好像不能区分。。。 像有些函数，如 bioskey()里，却能区分，它是怎么做到的呢?

答：

像左右shift键的码本来就是不同的，而象左右ctrl则通过有无扩充码来区分，
请看偶刚写的键盘中断程序：

```
#include <stdio.h>
#include <dos.h>
int stop=0;
void interrupt int_9h(void);
void main()
{
    void interrupt (*old_9h)(void);
    old_9h = getvect(9); /* save interrupt vector of 9 */
    setvect(9, int_9h); /* set interrupt vector of 9 */
    while(!stop);
    setvect(9, old_9h); /* recover interrupt vector of 9 */
}

void interrupt int_9h(void)
{
    unsigned char key;
    key = inportb(0x60);  /* read key code */
    outportb(0x61, inportb(0x61) | 0x80);
    outportb(0x61, inportb(0x61) & 0x7F); /* respond to keyboard */
    outportb(0x20, 0x20); /* End Of Interrupt */
    if( key == 0xE0 || key == 0xE1 )
       printf("KeyExtend=%02X ", key);
    else if( (key & 0x80) == 0x80 )
       printf("KeyUp=%02X\n", key);
    else
       printf("KeyDown=%02X ", key);
    if(key==0x81) /* Esc key is pressed and released */
       stop = 1;  /* Set the signal flag */
}
```

048 问 TC 编译模式以及 VGA 12h 直接写屏

问：

看了些介绍，好像说跟内存中数据段、代码段、堆、栈分配有关系，不同源文件的变量、指针、函数放在相同或不同的什么段里似懂非懂的
如果我的源程序不是各个文件分别编译的，而是一路 include 下来的，这个选项影响如何?
还有我试了下分别编译，编译都正常，有些模式下链接出错，说什么“fixup overflow...”，这个是什么原因？有的在 large 模式下能正常编译、链接、运行，但在 huge 能正常编译、链接，但是不能正常运行，为什么更大的反而不行了?
究竟应该选哪个模式呢?
还有源文件组织怎么样好，全部 include，还是分文件编译(这样声明跟实现要分开，不是很习惯的说)?

另外我在考虑究竟用哪个图形模式的问题。VGA 13h 虽然操作上比较理想，但是分辨率太小，我现在已经采用了 640*480，改起来也很麻烦。剩下的只有 VGA 12h 模式了，但是这个直接写屏好像很麻烦的样

子，网上找到的资料说得不清楚，例子都不是很典型，我摸索了一上午，还是搞不清楚：

```
void InitGraph()
{
    union REGS r;
    r.x.ax=0x0012;
    int86(0x10, &r, &r);   // 进入 VGA 12h 模式
    outportb(0x03ce, 5);  // 选择 5 号图形寄存器
    outportb(0x03cf, 0);  // 选择写模式 0
}

void PutPix(unsigned int nX, unsigned int nY, unsigned char byColor)
{
        outportb(0x03c4, 2);  // 选择2号时序寄存器
/*
        outportb(0x03c5, 0x0f);  //开放所有位平面
        g_pbyVideoBuffer[(nY<<6)+(nY<<4)+(nX>>3)]&=~(0x80>>(nX&0x07)); //清除目标点的四个位
*/
        outportb(0x03c5, byColor&0x0f);  // 根据颜色开放某些位平面
        g_pbyVideoBuffer[(nY<<6)+(nY<<4)+(nX>>3)]|=(0x80>>(nX&0x07));  //向目标点写入 1
}

int main()
{
    int i, j, k;

    InitGraph();

    for(i=10;i<14;i++)
        for(j=10;j<40;j++)
            PutPix(i, j, 10);


    for(i=10;i<18;i++)
        for(j=40;j<80;j++)
            PutPix(i, j, 14);


    bioskey(0);
    return 0;
}
```

可是跟我预期的不一样啊，只出现了一条竖线和两条竖线，而不是两个区域

我上面写点函数哪里不对?

答:

```
/* 相关原理资料在程序后面 */

#include <stdio.h>
#include <dos.h>
#include <bios.h>

void vga(void)
{
  _AX=0x0012;
  geninterrupt(0x10);
}
char far * video_addr(int x, int y, char *mask)
{
   char far *v = (char far *)0xA0000000;
   v += y*(640/8) + x/8;
   *mask = 1 << 7-(x%8);
   return v;
}

void put_pixel(int x, int y, int color)
{  /* copyright (c) Black White */
   /* email: iceman@zju.edu.cn  */
   char far *v, mask, c;
   v = video_addr(x, y, &mask);
   outportb(0x3CE, 8);    /* 8=bit mask register */
   outportb(0x3CF, mask); /* enable 1 bit to update */
   outportb(0x3CE, 5);    /* 5=mode register */
   outportb(0x3CF, 2);    /* 2=0000 0010 */
                          /* Bit1Bit0=write mode 2, Bit3=read mode 0 */
   outportb(0x3CE, 3);    /* 3=data rotate/function select register */
   outportb(0x3CF, 0);    /* Bit4Bit3=00 replace, Bit4Bit3=01 AND */
                                    /* Bit4Bit3=10 OR, Bit4Bit3=11 XOR */
   c = *v;                /* latch one byte from each bit plane */
   *v = color;
   outport(0x3CE, 0xFF08);/* default bit mask */
   outport(0x3CE, 0x0005);/* default mode register */
   outport(0x3CE, 0x0003);/* default function select */
}

main()
{
   int x, y;
   vga();
   for(y=240-50; y<240+50; y++)
   {
      for(x=320-50; x<320+50; x++)
      {
         put_pixel(x, y, 4); /* 4=red */
      }
   }
   bioskey(0);
```

```
}
```

049 %lx、指针变量定义时赋值、#define F(x)

问1：

请问小白老师在定义变量double a;以后在scanf和printf中都要使用%lx吗?

答1：

什么叫%lx？ %lx是来输入/输出long类型的16进制值的！！！

应该是%lf。输出的时候也可以用%f，但是我们自己编程的时候还是统一起来 不管输入/输出都用%lf。

问2：

还有两个99年卷子的问题

7．下列语句定义pf为指向float类型变量f的指针，___B__是正确的。

A、float f, *pf = f; B、float f, *pf = &f; C、float *pf = &f, f; D、float f, pf = f;

这个C不可吗，为什么呢?

答2：

因为C是pf定义在前，f在后面，前面的怎么可以引用后面的变量呢？ 当然是不可以滴!

问3：

8．表达式 sizeof(“key”) 的值是__D____。

A、1 B、2 C、3 D、4

答3：

这个太简单了，相当于问你"key"这个字符串所包括的元素个数，答案是4 如果改成strlen("key")的话，则答案是3。这个问题我记得上课讲过的。 而且象这类问题你完全可以自己编个程序试一下，自己动手，丰衣足食。

问4：

问题多起来了，再问一个

2．写出下列程序段的输出结果。

```
#define F(x)  x - 2
#define D(x)  x*F(x)
printf("%d,%d", D(3), D(D(3)));
```

答4：

这类问题你应该看书先，不看书来问明显是偷懒。

D(3) == 3 * F(3) == 3 * 3-2 == 9-2 == 7 注意展开F(x)的时候，你不要自作主张给它加上括号，没有括号的！！！ 后面那个你自己算吧，答案由电脑来告诉你即可。

050 再问野指针的问题

问：

今天上课白老师讲到用strcpy赋值是野指针，那么在struct里面定义了char *p以后get(p)是野指针吗？迷惑啊

答：

```
char *p;    //p 是野孩子
//-------------------------


char p[10];   //p 是好孩子
//-------------------------


char *p;  //这时候 p 是野孩子
p=(char *)malloc(10);   //p 成了好孩子了
//-------------------------


char *p;  //这时候 p 是野孩子
char a[10];  //a 是好孩子
p=a;   //p 成了好孩子了
//-------------------------


char *p;  //这时候 p 是野孩子
char a;
p=&a;   //p 成了好孩子了
```

051 上机练习15题:大写转小写，小写转大写

问:

题目是 程序填空，不要改变与输入输出有关的语句。

输入一个正整数repeat (0<repeat<10)，做repeat次下列运算： 输入一个字符，如果它是大写字母，输出相应的小写字母；如果它是小写字母，输出相应的大写字母；否则，原样输出。

例：括号内是说明 输入 3F=y (repeat=3, 输入的3个字符是'F', '=', 'y') 输出 f=Y

我编的是

```
#include <stdio.h>
#include<math.h>
int main( )
{
   int ri, repeat;
   char ch;

   scanf("%d", &repeat);
   fflush(stdin);
   for(ri=1; ri<=repeat; ri++){
      ch=getchar();
      if(ch>='A'&&ch<='Z')
         ch=ch-'A'+'a';
      if(ch>='a'&&ch<='z')
         ch=ch+'A'-'a';
      putchar(ch);
   }
}
```

在TC上能算出来的，但是交了后说“答案错误”
请问是啥原因~

答:

if -> else if

052 指数形式的常数是什么类型？如何理解0?0:1

问1:

书P91说指数的类型都是双精度浮点，但是实型不是有单精度和双精度吗?

答1:

小数形式的常数的类型默认就是double，不是float，例如3.14，314E-2都是double。 如果你要强调某个小数形式的常数是float的话，则要求后面加个F后缀，比如： 3.14F

问2：

书P91说指数的类型都是双精度浮点，但是实型不是有单精度和双精度吗?

答2：

? 前面是条件，0转化成条件就是假。
根据? : 运算符的规则，条件为真时答案为:前那个，
条件为假答案为:后面那个。所以，此表达式的值
是1。
关于数值如何转化为条件以及条件如何转为数值，
请参考主页“知识要点”。

053 今天上课关于计算宏定义FB(a,b)的题（请看第二题）

问1：

今天上课的一道例题
求输出结果：

```
#include<stdio.h>
#define FB(a,b)  a*b+1
main()
{
 int k=3;
 printf("%d\n",FB(k,k+1));
 printf("%d\n",FB(FB(k,k+1),k)-7);

}
```

运行结果： 11
7（不是27）
貌似不能先把FAB(k,k+1)=11 算出来，要完全带入才行(k*k+1+1*k+1-7=7)

如果 define FB(a,b) （a*b+1） 结果就是27

答1：

正解。

问2：

再问一个很困惑的问题

```
#include<stdio.h>
#include<math.h>
main()
{
    int num=123456,digit,i=1;

    digit=num%(int)pow(10,i);
    printf("%d\n",digit);

}
```

为什么输出是 -6 而不是 6

答2:

如果你这个题是在VC里运行的话，结果应该就是6。
在TC里，因为int是16位，而123456是long类型
的常数，所以在num=123456赋值时已经出问题，
不信你可以在赋值后立即printf出来看看。
当long类型赋值给short类型时，会造成数据高16位
丢失，仅保留低16位。
用计算器算一下，123456转化成16进制为：
0001 E240
保留低16位即num=0xE240。
此数肯定是负数，因为最高位为1。
它的相反数可以用凑凑法算出来是1DC0：
E240

1DC0 +)

0000
再用计算器算一下，0x1DC0转化成十进制是：
7616
所以，0xE240就是-7616。

054 int *p = (int *)123 是什么意思?

问:

```
int x,*p=&x; p=(int *)123;
```

这句话什么意思？p和x有没有什么变化？

答：

p=(int *)123; 表示把整数123转化成指针类型并赋值给p。
因此本来p指向x，现在p指向另外一个变量，该变量的
地址为123。x没有变化。

如果写成p=123是错误的，因为类型不一致。
不管把整数赋值给指针变量，还是把指针赋值
给一个整型变量都是错误的。除了以下情况：
指针变量=0;
它相当于
指针变量=NULL;

055 一道与文件相关的题：最后一个数加了两遍

问：

输入一个代表文件名的字符串，该文件存放了若干整数，将这些整数之和写入文件out.txt中。

比如：
文件in.txt中的内容
1 2 3 4 5
10 9 8 7 6

输入：
in.txt

输出：

创建out.txt文件，文件内容为：
55

该题如下,为什么老是输出61,将最后一个数加两遍

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
 FILE   *fp1,*fp2;
 int x,sum=0;;

 if ((fp1=fopen("in.txt","r"))==NULL)
 {       printf("File open error!\n");
    exit(0);
 }
if ((fp2=fopen("out.txt","w"))==NULL)
 {       printf("File open error!\n");
    exit(0);
 }
 while (!feof(fp1)){

           fscanf(fp1,"%d",&x);
           sum=sum+x;

 }
 fprintf(fp2,"%d",sum);
 fclose(fp1);
 fclose(fp2);
}
```

该题如下,为什么老是输出61,将最后一个数加两遍

答:

```
fscanf(fp1,"%d",&x);
```

改成

```
fscanf(fp1,"%d ",&x);
```

注意%d后面有一个空格。

056 long b=a*0x12345678; printf("%x\n", b);

问:

```
int a=1;
long b=a*0x12345678;
printf("%x\n", b);
```

答案是多少？为什么是5678或12345678
昨天还看到过有同学曾经提过同样的问题，今天怎么就找不到了

答：

这道题如果在VC下面运行的话，结果就是
12345678
因为在VC中，int就是long int，并且
%d、%x输出int类型的值就相当于
输出long int类型的值。
注意十六进制前缀0x是不会打印出来的。

如果在TC下面运行，结果就是5678。因为
TC的int实际上是short int，因此%d、%x
输出的是short int类型的值。但是b的值
是long类型的，于是printf()仅取0x12345678
中低16位打印出来。0x12345678在内存中是
以低8位在前，高8位在后顺序存放的，从内存中
取出之后，要颠倒一下：
地址 值
1000 0x78
1001 0x56
1002 0x34
1003 0x12
因此，取出地址1000及1001中包含的低16位
再颠倒一下就是0x5678。
在TC环境下，出这个题是超范围的。
因为，数据在内存中的存放格式并没有在课本中讲述。

057 switch中的break与循环中break的区别

问1：

以下不正确的字符常量是 A '%d' B 10

A是因为它有两个字节了，所以不对，是吗？ B为什么不正确?

答1：

你对A的理解正确；

B是10，它是一个整数类型的常数，不是字符， 所有字符常数都必须用单引号括起来的。

问2：

break语句跳出的是它所在的最大还是最小循环?

答2：

小循环。请参考我主页上“知识要点”里讲到如何用 break跳出两层循环。

问3：

switch里的break和以下这个程序的break有什么区别吗?

答3：

switch里的break与循环无关，只是表示某个case到此结束， 不要再继续执行到其它case里去。 循环(包括while、do...while、for)里的break都表示跳出当前所在的一层循环。

058 一道比较复杂的指针题(3颗*)

问：

```
char *c[] = {"ENTER", "NEW", "POINT", "FIRST"};
char **cp[]= {c+3, c+2, c+1, c};
char ***cpp = cp;
main()
{
    printf("%s", **++cpp);
    printf("%s", *--*++cpp);
    printf("%s", *cpp[-2]+3);
    printf("%s\n", cpp[-1][-1]+1);
}
```

我有点晕了

答：

059 行指针与列指针的区别

问：

char a[80][80],(*p)[80]; *

*p 与 (*p或p[0]) 没什么区别吧，记得某次听小白老师说 p 指向第一行，(*p或p[0]) 指向第一行的首元素的地址，但实际它们的值是一样的。

so p 指向第一行，(*p或p[0]) 指向第一行的首元素的地址什么意思吓？p难道不也是指向第一行首元素的地址么？

答：

char a[80][80],(*p)[80];
p = &a[0]; 或 p=a;

p指向a[0]行，是一个行指针，或者说是一维数组指针。
*p 就是a[0]行本身，转化一下，就是a[0]行首列的地址，即&a[0][0]。
概括一下：
p == &a[0]
*p == &a[0][0]
你说它们会是一回事吗？明显不是。

但是它们的值确实是相等的。因为对于某个一维数组来说，
比如这里的a[0]行，它的整行地址与它的首列地址是相等的，即
&a[0] 在数值上 与 &a[0][0]相等。

那么它们究竟有啥区别呢?
主要是引用它们的时候会有一个*号的差别：
比如你想要取得a[0][0]的值，并赋值给字符型变量c，则：
c = **p;
而不能写成：
c = *p;*
*因为*p == &a[0][0]，仍旧是一个指针，怎么可以赋值给c ？
换一句话说，p指向的是a[0]行，*p就是a[0]行本身，它是
不能赋值给一个字符变量c的，你想想，一行怎么能赋值给一个呢?
再转化一下，*p作为a[0]行本身等价于该行首列的地址，所以它只
能赋值给一个类型为char *的指针变量。

060 字符串中查找子串问题?

问：

从a字符串中找出与b字符串相同的子字符串的个数?
如abadsfab ab 答案是2
我是这样写滴，有两个，其中有不明白之处？？？
1：

```c
#include<stdio.h>
int fun(char *p,char *q);
void main()
{
    char p[80],q[10];
    gets(p);
    gets(q);
    printf("%d",fun(p,q));
    getch();
}
int fun(char *p,char *q)
{
    int count=0;
    char *temp=q,*i;
    while(*p)
    {
        i=p;
        if(*q==*i)
        {
            while(*q++==*i++);
        }
        if(!(*q))
            count++;
        p++;
        q=temp;
    }
    return count;
}
```

2:

```
#include<stdio.h>
int fun(char *p,char *q);
void main()
{
    char p[80],q[10];
    gets(p);
    gets(q);
    printf("%d",fun(p,q));
    getch();
}
int fun(char *p,char *q)
{
    int count=0;
    char *temp=q,*i;
    while(*p)
    {
        i=p;
        while(*q)
        {
            if(*q==*i)
            {
                q++;
                i++;
            }
            else
                break;
        }
        if(!(*q))
            count++;
        p++;
        q=temp;
    }
    return count;
}
```

第二个是正确的，第一个很奇怪啊，如果用tc的调试一步步进行下去，则能够出来正确的答案，而如果直接编译运行，则会出来错误的答案，想不通～～～

答：

```
while(*p)
    {
        i=p;
        if(*q==*i)
        {
            while(*++q==*++i);      /*这里改成这样好像更对一点，但还是不对.最大的问题，我用tc的f

        }
        if(!(*q))
            count++;
        p++;
        q=temp;
    }
```

061 if(i>j) if(a[i][j]==0)与i>j&&a[i][j]==0

问：

if(i>j)

```
if(a\[i][j]==0)
```

与if(i>j&&a[i][j]＝＝0)有什么区别

答：

如果后面没有else的话，是没有区别的。
如果有else的话，比如：
if(i>j)
if(a[i][j]==0)
(1)
else
(2)
else
(3)
则，只有在(1)这里，两者是等价的。

062 指针或者数组元素调试

问：

老师： 对于指针或者数组元素进行调试时，要看指针移到哪个位置了，或者相应数组元素的值，应该怎么调试啊？？

答：

如果是数组，比较好办，直接输入数组名即可。
如果是指针变量的话，有两种情况：
(1) char *p这种字符指针变量，直接输入p看到的是p指向的字符串，不是p本身的值。如果你想看p本身的值，可以输入
p, p
表示以指针形式显示该指针变量的值。
或者输入
p, x
表示以16进制形式显示p的值。
(2) int p这种整型指针，直接输入p看到的是p本身的值，如果你想看p指向的整数，则应该输入
p
如果你想看p、*(p+1)*、(p+2)、...共10个元素，则可以输入
*p, 10
如果你想以16进制形式观察这10个元素，则输入
*p, 10x

063 a[3]={'a','b','c'}，sizeof（a）=3

问：

`sizeof("hello")=6` ,因为要考虑‘\0’; 那么为什么 `a[3]={'a','b','c'}` ， `sizeof（a）=3` ， 为什么不用考虑‘\0’

答：

因为凡是双引号括起来的东东，最后肯定有一个隐藏的'\0'。 但是象 `{'a','b','c'}` 这种，你看到有一个，就一共有几个， 后面没有'\0'的。

064 为什么5%－3等于2? －5%3却等于-2

问：

为什么5%－3等于2呢？？？

但是－5%3却等于-2！

答：

告诉你一个诀窍：

你先对被除数与除数都求绝对值，
然后计算商，算出商后再根据原
先被除数及除数决定该商是正的
还是负数。
商有了，余数就好办了。

举例如下：
5 % -3 ==> 先算5 / 3 = 1
==> 真正的商是-1 ==>
5 - (-3 * -1) = 2为余数

-5 % 3 ==>先算5 / 3 = 1
==> 真正的商是-1 ==>
-5 - (3 * -1) = -2为余数

-5 % -3 ==>先算5 / 3 = 1
==>真正的商是1 ==>
-5 - (-3 * 1) = -2为余数

065 %%表示什么?

问：

语句printf(“%%d%d”, 123); 将输出： C

A) %123%d B) %%d123 C) %d123 D)上述语句语法有错

答：

对滴。
答案是C。
注意，双%号只在printf里面才会输出一个%号，
如果puts("%%")，则输出的仍旧是两个%号。
所以不能因为printf("%%")输出一个%号
就据此得出错误的结论："%%"等价于一个%号。
"%%"中包括的元素个数仍旧是3个：
'%' , '%', '\0'

而"\"则不一样，它包括的元素个数是2个：
'\', '\0'
其中'\'表示一个\符。

066 强制转换：低类型转换成高类型

问：

再问一个，将高类别的强制转换成低的类别数，会产生错误；那低的转高的呢?
long b=0x1234;而内存中恰好是
0x34
0x12
0x56
0x78
这样连续排列，会不会b=ox78561234呢?

答：

如果你已经定义了
long b=0x1234;
则表明它已经是一个32位数了，
内存中应该是这样排列的：
0x34
0x12
0x00
0x00
即它的高16位是0。

如果你希望出现你所猜测的结果，则应该这样写：

```
char a[4]={0x34, 0x12, 0x56, 0x78};
long int *p;
long int b;
p = (long int *)a;
b = *p;
```

此时，b=0x78561234;

如果你写成这样：

```
short int a=0x1234, b=0x7856;
long int x;
x = a;
```

则x的值为0x00001234，而不可能变成

0x78561234。
要是你一定要让x变成0x78561234，则
要这样写：
x = *(long int *)&a;

067 关于全局变量

问1：

如果全局变量被多次赋值，引用，那么最后其为最后一次变化后的值吗？还是看函数而定，即为在该函数内变化的最后的值?

答1：

全局变量的值决定于最后一次变化后的值。
比如：

1. main()使全局变量x=100;
2. main()调用函数f()时，此时全局变量的值=100，然后假定f()使x=1000;
3. main()调用函数g()时，此时全局变量的值=1000，然后再假定g()使x=1234;
4. 最后main()输出x的值就是1234

其实静态变量与全局变量的初始赋值都是 在程序刚开始运行时完成的。以后都不会 执行定义它们的那个语句。

对于局部静态变量，讲课的时候为了便于 与普通的局部变量联系起来，所以就讲成 它是在函数首次调用时被定义并赋值，以 后每次函数调用时不再被赋值。实质上并 不是这么回事，它其实在每次函数调用时都不赋值的，因为它是在整个程序刚开始 运行时就已经被定义并赋值了。

问2：

如果某包含了该全局变量赋值的函数被多次调用呢? 是不是只执行一次赋值?

答2：

多次赋值。 但都是对同一个 地址上的变量 赋值。 全局变量相当于一块 固定的内存块。这个变量一直在那存在，每次程序对它操作，都在那个内存块上操作

068 关于结构变量和指针赋值的问题

问1：

struct student s1 s2

对s1,s2的表达式正确的有
(a)s1=s2 (b) s1==s2

答案是a

我的理解:
s1和s2的本质是结构数组的初始地址.既然能够赋值,为不能够比较?

答1:

你的理解
“s1和s2的本质是结构数组的初始地址”
是不对滴。
s1与s2是两个结构变量，不是地址。
s1=s2相当于把s2的成员值全部复制给s1。
至于为什么不能if(s1==s2)就不知道了。
如果真能比的话，就意味着每个成员都要
逐一进行比较，一旦有一个成员不等，就
表示整体不相等。估计一下，可能是C创始
者觉得这种比较用处不大。
如果你想自己对s1、s2进行快速比较的话,
我觉得可以用memcmp()这个函数:

```
if(memcmp(&s1, &s2, sizeof(struct student)) == 0)
puts("s1 == s2");
```

问2:

对于两个指针 如果p,q是指向同一类型的指针 q=&p有意义么? 如果p,q是指向不同类型的呢?

答2:

比如:
int a=1, b=2;
int *p=&a, *q=&b;
这里，p、q是指向同一类型的指针。

如果你写成
q = &p;
必定是语法错误。因为两者类型不匹配:
q 的类型是： int *
&p的类型是： int **

如果你一定要进行上述赋值，则必须进行强制类型转化：
q = (int *)&p;
但这样写似乎没有啥意义。
*因为此时，你引用*q得到的将是
p的值即a的地址，但你不能进一
步引用**q企图得到a的值。

除非你这样写：
int *p=&a, **q=&p;**
这样的话，
引用q得到的就是a的值。

如果p,q是指向不同类型，写成q = &p一般也没有啥意义，
除非象前面提到那样p是一颗星的指针，q是两颗星的指针。
倒是写成q = p(要进行类型转化)可能有些用处：
char a[2]={0x12,0x34};
char *p=a;
short int *q, x;
q = (short int *)p;
x = *q;
此时x的值为0x3412。

069 p是结构数组名，为什么p->k=2是正确的?

问1：

13. 对于以下的变量定义，表达式_____是正确的

```
struct node {
       char s[10];
int k;
} p[4];
```

A. p->k=2 B. p[0].s=“abc” C. p[0]->k=2 D. p->s=‘a’

碰到这样的问题就是搞不懂 ^ ^ 请教达人

答1：

p是结构数组名==> p == &p[0]
p->k == (&p[0])->k
(&p[0])->k == p[0].k
所以是正确的。

B和D企图对数组名s进行赋值，所以是错的。

p[0]是结构变量，后面不能跟->，所以C错。

归纳一下：
.前面必须是结构变量
->前面必须是结构指针

070 如何在TC里画图？initgraph怎么用？

问：

就是用C语言写了个程序，调用了一下graphics.h，出现了这种情况~~

答：

```
#include "Conio.h"
#include "graphics.h"
#define closegr closegraph

void initgr(void) /* BGI初始化 */
{
  int gd = DETECT, gm = 0; /* 和gd = VGA,gm = VGAHI是同样效果 */
  registerbgidriver(EGAVGA_driver);/* 注册BGI驱动后可以不需要.BGI文件的支持运行 */
  initgraph(&gd, &gm, "");
}

int main(void)
{
  initgr(); /* BGI初始化 */

  /*****此部分添加你自己的代码，例如
  line(25, 25, 220, 220);
  circle(100, 100, 50);
  等等*****/

  getch(); /* 暂停一下，看看前面绘图代码的运行结果 */
  closegr(); /* 恢复TEXT屏幕模式 */
  return 0;
```

我的图形模板送给你用用。

071 区别:1. EOF和'\0'; 2.'\n'和数据输入

问：

区别:1. EOF和'\0'; 2.'\n'和数据输入

答：

1.'\0'和EOF有什么区别?
区别(1)
EOF的值= -1
'\0'的值= 0
区别(2)
EOF通过按键盘Ctrl+Z可以输入，但是'\0'没有办法输入。

2.'\n'和把数据输入有什么区别?
输入时敲回车用来表示scanf()输入结束，但是'\n'本身
并没有被输入的变量接收。如果下次又通过scanf()或者
getchar()输入一个字符，则上次输入时所敲的回车会被
第2次输入接收。

072 二维数组与指针的指针的区别

问：

二维数组与指针的指针的区别

答：

问2：

看完解释之后，有些明白，有点困惑。

二维数组不能化为指针的指针，是不是仅仅因为&a[0][0]不被存储而不能被指针指向这一个问题？ 还有一个没想明白的地方就是，数组名作为指针会不会被存储在内存空间中？ 期待老师回答……

答2：

对的。&a[0][0]根本没有存储空间，它不可能成为任何指针所指的对象。
当a->a[0]时，a[0]只能理解成一整行即一个一维数组(此时的a[0]是有存储空间的),
而不可以再把a[0]转成&a[0][0]来理解。
当a[0]->a[0][0]时，a[0]又摇身一变成了&a[0][0]的地址，注意此时的&a[0][0]是指针，
而不是指针指向的对象。
归纳一下，&a[0][0]由于没有存储空间，所以它只能指向别人，而不可以让别人指向它。

问3：

还有一个没想明白的地方就是，数组名作为指针会不会被存储在内存空间中？ 期待老师回答……

答3:

不可能。它跟一个变量的地址一样，是没有存储空间的。

073 如何对结构中的字符指针成员进行赋值?

问:

如何对结构中的字符指针成员进行赋值?

答:

假定定义如下结构类型:

```
struct st
{
   char *s;
   int score;
};
```

再定义结构数组:

```
struct st p[3];
```

则要对p[0]中的s赋值，可以这样写:

```
p[0].s = "abc";
```

以下两个写法均是错误的:

```
p[0].*s = "abc";
*p[0].s = "abc";
```

也可以先定义一个数组，把数组首地址赋值给p[0].s，例如:

```
char a[10];
p[0].s = a;
```

在做完以上这个赋值之后，再做以下两个之一也可:

```
*p[0].s='a';
*(p[0].s+1)='b';
*(p[0].s+2)='c';
*(p[0].s+3)='\0';
```

或

```
p[0].s[0]='a'; 即 (p[0].s)[0]='a'; 因为[]和.运算符同级且左结合，下同
p[0].s[1]='b'; 即 (p[0].s)[1]='b';
p[0].s[2]='c'; 即 (p[0].s)[2]='c';
p[0].s[3]='\0';  即 (p[0].s)[3]='\0';
```

注意：

```
*p[0].s='a';
```

不可写成：

```
p[0].(*s) = 'a';
```

所以，以下写法全部有错：

```
p[0].(*s) = 'a';
p[0].(*(s+1)) = 'b';
p[0].(*(s+2)) = 'c';
p[0].(*(s+3)) = '\0';
```

074 int a,b[3]; a=b; 这样对吗?

问1：

int a,b[3];a=b;这样对吗？ 好像地址是整数型的，而a 也是，应该不会错吧！

答1：

错的。因为a是整数类型，而b是指向整数的指针类型，完全不同的东东。

地址的值虽然是整型，但是这个并不能推出 地址=整数

如果你一定要把b赋值给a，则必须进行强制类型转化：

a = (int) b;

问2：

```
int a;a="abc";
```

把“abc"的地址给a，这样总对了吧。。。嘿嘿

答2：

错！

整数类型的变量只能被赋值为一个整数，不能被赋值为一个地址。

如果你一定要把"abc"的首地址赋值给a，则同样需要作强制类型转化：

```
int a;

a = (int) "abc";
```

075 若p是结构指针，为啥*p.pk是错的?

问：

15. 对于以下的变量定义，表达式_____不符合C语言语法。

```
struct node {    int len;     char *pk; } x = {2, "right"}, *p = &x;
```

A、p->pk B、*p.pk C、*p->pk D、*x.pk

答：

这里你一定要注意它的优先级， 句点的优先级高于星号，

所以

```
*p.pk
```

就是

```
*(p.pk)
```

根据上课讲过的知识，句点前面必须是结构变量，不能是结构指针。

076 arr是char二维数组,printf("%s\n",arr)是啥意思?

问：

5. 若以下定义：

```
1 | char s[20],*ps=s;
```

则，以下赋值语句正确的是：
A、s=ps+s; B、ps=ps+20; C、s[5]=ps[9]; D、ps=s[0];
答案是B。C为什么不对呢?

答：

ps[9]相当于s[9]，按道理把它赋值给s[5]是没有语法错误的，
但是s[9]本身的值不确定，所以编译的时候会有警告的。
这个题出得不好。

问：

5．下列程序执行后的输出结果是：_____。(4分)

```
#include<stdio.h>
#include<string.h>
void main()
{
 char arr[2][4];
 strcpy(arr[0],"you");
 strcpy(arr[1],"me");
 arr[0][3]='&';
 printf("%s\n",arr);
 }
```

arr表示&arr[0]
那为什么不是就输出you&

答：

这个题同样出得不好，因为出题的人没有弄清楚行指针与列指针的区别。
printf("%s\n", arr);中的arr是行指针，不应该代入作为参数的。如果
这个printf()换成
puts(arr);
将必然导致编译错误。由于printf()函数比较特殊，编译器对这种错误检测不出来。

不过，上课的时候也提到过，行指针arr与列指针arr[0]在数值上是相等的，所以
这个printf()也能打印出结果来的。
arr[0][3]='&'; 把a[0]行中原先存放的字符串"you"中的结束符'\0'给替换掉了，
所以相当于该字符串就没有结束，于是打印的时候就把后面的内容也一起打出来，
最终的结果就是"you&me"。

077 左结合与右结合是指运算符还是变量?

问：

*p++为什么是(*p)++
而-i++是-(i++)?

“左结合”与“右结合”是指 运算符 还是 变量?
*p++ 中*与++优先级相同，按变量右结合应该是先p++然后再*（p++）啊？？

答:

当一个变量左边及右边都有运算符时，它到底与右边还是与左边结合
决定于:

1. 哪个优先级高
2. 优先级相等时，再看是右结合还是左结合
   对的，*p++确实就是*(p++)
   而不是 (*p)++ **<== 这个是谁说的？宰了他。**

078 如何重命名.txt为.c？

问:

做上机练习题时，为什么传文本文件批改是错的，可以上传哪种格式啊，我自己电脑保存的文件形式只能是文本文件

答:

1. 去掉以下选项中前面的勾
   我的电脑->工具->文件夹选项->查看->隐藏已知文件类型的扩展名
2. 重命名.txt为.c

079 关于 编译指令 操作符'#'

问:

比如说

```
1 | #define print(d)  printf(#d " = %d\n", d)
```

展开成

```
1 | printf("5 =%d\n",5)
```

那么#到底是怎么工作的,表达式中有这个么?

答:

这个用法我还是第一次看到，查msdn得：
http://msdn2.microsoft.com/zh-cn/library/7e3a913x.aspx

> The number-sign or "stringizing" operator (#) converts macro parameters to string literals without expanding the parameter definition. It is used only with macros that take arguments. If it precedes a formal parameter in the macro definition, the actual argument passed by the macro invocation is enclosed in quotation marks and treated as a string literal. The string literal then replaces each occurrence of a combination of the stringizing operator and formal parameter within the macro definition.
> White space preceding the first token of the actual argument and following the last token of the actual argument is ignored. Any white space between the tokens in the actual argument is reduced to a single white space in the resulting string literal. Thus, if a comment occurs between two tokens in the actual argument, it is reduced to a single white space. The resulting string literal is automatically concatenated with any adjacent string literals from which it is separated only by white space.
> #用来把参数加上双引号。
> 另外还有#@以及##的用法。#@用来把参数加上单引号，##把参数与前面的东东粘在一起。
> 例如：

```
#define makechar(x)  #@x

causes the statement

a = makechar(b);

to be expanded to

结果：
a = 'b';

#include <stdio.h>
#define paster( n ) printf_s( "token" #n " = %d", token##n )
int token9 = 9;

int main()
{
   paster(9);
}
结果：
token9 = 9
```

080 函数声明能写在主函数里面吗?

问：

函数声明能写在主函数里面吗?

答:

可以的，但是最好写在程序最前面。
假定你声明的函数叫f()，
你把f()的原型写在main()里面的话，
f()只能被main()调用，如果还有一个
函数比如叫g()，它也要调用f()的话，
那g()里面也要写上f()的原型。
如果把f()的原型写在程序最前面，则
随便什么函数都可以调用f()了。

081 关于C语言指针的一些问题

问1:

1、胡乱的在程序中写字符串或者一个数什么的，是不是直接将这个字符串或数字丢进内存里面，然后它就有了一个地址？然后再作赋值或者取地址等等运算?
比如：char *p="XYZ";
这样p就代表了字符串"XYZ"的首地址了，其实一直很想不明白，连"XYZ"都是凭空捏造出来的，为什么会有地址呢？是不是这样理解的呢?

答1:

1.p就是‘x’的地址，输出时则是按地址依次输出（xyz的地址是相邻的），而char是用来确定一个变量所占用的地址（一个地址为8位一字节），如char只有一个字节，即对于每个变量只需一个地址，在tc中，对int每个变量则需两个地址（所需地址数与变量依次输出是没有关系的，但在读取地址内容时则是会根据类型判断是读取多少个地址，如tc中long是32位4字节（地址），若将指向int的指针来取long的内容则会只读取前两个地址的内容而忽略后两个地址的内容

问2:

2、关于指针的指针。
按理说，比如

```
1 int m,*p;
2 *p=&m;
```

这种事情好像是不对的吧，这样p就表示指针的指针啦，可是在tc里面却是允许的啊。
与之类似的，还有比如说:

```
int m,*p,**q;
q=&m;
p=**q;
```

这种怎么想都不对，可是tc里面都是允许的，似乎没有什么问题。
我想是不是有可能这样，既然定义里p前面就一个*，那么下面的p前面就不能有**,其它的么～～试了很久，好像都没有那么严格规定啊。

答2:

2.

```
int m, *p;
*p=&m;
```

这种写法是错误的（书上有勘误）。如前所述，int是用来表示p所指向的内容占用的两个地址，p是其首地址，应写成p=&m;
另一个也是不对的，应该是

```
int m,*p,**q;
p=&m;
q=&p;
m=**q;
```

问3:

3、int 的类型到底怎么算啊呢，按tc的话地址应该可以算是一个int么，可是按vc就不行啦，像上面2的那些例子的话改成long int 的话完全就不行啦，要是改成:

```
int m,*p;
*p=(long)&m;
```

这样倒可以了。这算什么呢？地址被从int 提升了？
最关键的问题是，碰到int 到底算什么呢？？？ short or long???

答3:

3.在tc中int就是16位的，long就是long int 32位的
vc则有自己的规定

问4:

4、关于

```
p=&*&x;
```

这个对我来说真的很难理解啊，比如说x=1;取x的地址，然后取这个地址的内容，那不就是0000 0001这样子？取出来以后不是一个常数么，为什么还有地址？而且试验了一下就是x的地址？
也就是说有时候*&*&*&这种东西多了可以约掉？？？不大理解啊。

答4：

> x 是一个某类型变量
> &x 是一个某类型变量地址
> *&x 是一个某类型变量地址处的某类型变量，也就是 x 这个变量本身
> &*&x 也就是 &x
> ps:感谢来自cc98**溪流**同学的解析，但是关于“&*和*& 总是可以拿掉”这个说法不太严谨。

082 char *a="abc",*b="edf";strcat(a,b);出问题

问：

```
#include <stdio.h>
void f(char *s,char *t)
{
  while(*s!='\0')
      s++;
  while((*s++=*t++)!='\0')
  ;
}
void main()
{
  char *a="abc",*b="edf";
  f(a,b);
  printf("%s\n%s\n",a,b);
}
```

结果b输出了df,e怎么被吃掉了呢？？

答：

> 内存情况：
> **a**, b, c, \0, **e**, d, f, \0, ... (粗体标的是 a 和 b 的开头)
> 进行 f(a, b) 时，这部分的变化：

```
abc\0edf\0
^    ^
s    t
while(*s!='\0')
    s++;
```

这一步之后

```
abc\0edf\0
    ^^
    st
```

while((*s++=*t++)!='\0') 之后每一步这段数据变化如下：
a, b, c, e, **e**, d, f, 0, ... (复制 e)
a, b, c, e, **d**, d, f, 0, ... (复制 d)
a, b, c, e, **d**, f, f, 0, ... (复制 f)
a, b, c, e, **d**, f, 0, 0, ... (复制 0)
所以这时候 b=="df"
ps:感谢来自cc98的**hmilynavi**同学和**溪流**同学的解析。可能因为网页排版的问题，链接中的t指向的是\0，这里需要注意一下，**b和初始的t指向的是e**。

083 一道笔试题:char *a[2]={"one","two"},**p=a;

问：

```
char *a[2]={"one","two"},**p=a;
printf("%s,",*(p++)+1);
printf("%c\n",**p-1);
```

这个程序段输出什么呢?

答：

p是一个指向指针的指针，它的地址等于a的地址，那么，
p指向的是"one",(p+1)指向的是"two"，即*p相当于a[0]\，*(p+1)相当于a[1]
再来看程序
*(p++)这个表达式的值等于*p （这里的p是未++前的p的值），即a[0]，执行完后，*p的值等于a[1]
再来看*(p++)+1。前面说了*(p++)相当于a[0]，注意a[0]也是一个指针，视char *temp=a[0]，则temp指向"one"
将它的值加1，即(temp+1)，指向"ne"，以字符串输出，所以输出"ne"。
上面说过，此时*p相当于a[1]，那么再设一个临时指针char *t = a[1]，则 t 指向 "two"

> **p = *(*p) = *t = 't'
> 然后将't'（字符）减1，将得到的值以字符形式输出，于是输出's'
> 所以，整个程序输出是 ne,s
> ps:感谢来自cc98的**烟柳如痕**同学的解析。

084 %d格式输入字母时，为什么循环停不下来?

问:

```
#include <stdio.h>
void main()
{
   int n;
   do{
      printf("input\n");
      scanf("%d", &n);
   }while(n<1 || n>9);
}
```

当输入字母时，为什么该循环停不下来呢?
一直输出input，好像scanf()没有被执行

答:

当你输入一个字母时,
%d是不要这个字母的,
它就是想要数，而这个字母
一直留在输入缓冲区中会阻
止你输入下一个数。
解决办法如下:

```
#include <stdio.h>
void main()
{
int n;
do{
  printf("input\n");
  fflush(stdin); /* 这句话的作用是清除输入缓冲区 */
  scanf("%d", &n);
}while(n<1 || n>9);
}
```

ps:感谢来自cc98的**如嫣**同学的解析。

085 p=&x++;为什么错?

问:

假设变量已经被正确定义.
p=&x++;
为什么不正确

答:

&与++优先级别相同，根据右结合，得
p = &x++; 相当于
p = &(x++);
而x++作为整体并不是一个变量，它不可以被取地址。

086 char s[100]，scanf("%s",&s)有没有问题?

问:

```
1 char s[100];
2 scanf("%s",s);    /*假设s为字符型数组*/
```

于是我试了下，发现scanf("%s",s);和scanf("%s",&s);的效果好像是相同的。显然&s不是一个指向字符数组的指针，那么 &s是什么呢，指向指针的指针？而为什么 &s的值和s一样呢?

答:

&s中的s应该理解为数组s整体，而不能理解为该数组的首地址即&s[0]，因为地址是不能再取地址的。
所以&s指向的对象是一个数组，该数组由100个char构成，即&s实际上是一个行指针。
VC中显示出来的&s类型是错误的，&s的类型应该为:

```
char (\*)[100]
```

而&s[0]的类型为:

```
char *
```

两者的类型是完全不同的。
如果定义两个变量如下并分别赋值为&s[0]、&s:

```
char s[100], t[100];
char *p = &s[0];
char (*ps)[100] =&s;
```

接下去把这两个指针变量分别加1后：

p++;

ps++;

则p指向s[1],而ps指向数组t。

不过，&s[0]的值与&s的值确实是相等的。因为数组s作为一个整体，它的地址=它的首元素的地址。

那么C编译器为什么对scanf("%s",&s[0])与scanf("%s", &s)不加以区别呢？或者说，C编译器为什么对scanf("%s", &s)这种明显不正确的用法不报错呢?

那是因为scanf()是一个特殊的函数，该函数的原型为：

int scanf(char *format, ...);

此函数只有第1个参数的类型是确定的，后面的参数个数以及参数类型全部不能确定，因此C编译器也就不可能对第2个参数以及后面参数的类型进行检测。换一句话说，C编译器绝对不会把第2个参数以及后面参数的类型与格式串format中指定的格式进行匹配检测。

如果你想自己检验一下上面的说法是否正确，完全可以自己编个简单的程序进行验证：

```
void  input(char *p)
{
   scanf("%s", p);
}

main()
{
    char s[100];
    input(s); /* 你可以试一下把此处改成 input(&s); 看看结果会咋样 */
    puts(s);
}
```

087 void p(int *x,int *y) {p=x;x=y;y=p;}

问：

```c
#include<stdio.h>
void p(int *x,int *y)
{  int *p;
 p=x;x=y;y=p;

}
void main()
{ int x=0,y=3;
  p(&y,&x);
  printf("%d,%d",x,y);


}
```

为什么xy的输出还是0，3
没有调换

答：

> 不妨把这里的 x、y 记为 a、b：
>
> ```c
> void p(int *a,int *b)
> {
>     int *p;
>     p=a;
>     a=b;
>     b=p;
> }
> ```
>
> main 里的 x、y 还叫它 x、y。
>
> 函数调用
> p(&y,&x);
> 的时候，
> 把 y 的地址给了 a，把 x 的地址给了 b。
>
> 接下去：
> p=a; // p = a = y 的地址
> a=b; // a = b = x 的地址
> b=p; // b = p = y 的地址
> 于是这样的结果仅仅是让 a 指向了 x，让 b 指向了 y
>
> x、y 本身何曾变过?
> ps:感谢来自cc98的**溪流**同学的解析。

打个比方。
桌上有两个盒子，一个盒子编号为1，另一个编号为2，
现把1和2分别抄到两张小纸条上，
再把两张纸条装入容器a和容器b，
最后把容器a和容器b中的两张纸条交换一下。
于是盒子1与盒子2中的内容并没有发生变化。

088 如果局部变量与全局变量同名

问：

```
类似的两个题，怎么结果不一样啊？？
第一题：#include<stdio.h>
int c;
void change(int *a,int b)
{
   --*a;
   b++;
   c=c+1;
}
main()
{
   int a=10,b=20,c=30;
   change(&a,b);
   printf("%d,%d,%d$",a,b,c);
}
```

结果c=30
第二题：

```
#include<stdio.h>
func(int a,int b);
int c,a=4;
main()
{  int b=2,p=0;
   c=1;
   p=func(a,b);
   printf("%d,%d,%d,%d#",a,b,c,p);
}
func(int a,int b)
{
   c=a*b;
   a=b-1;
   b++;
   return(a+b+1);
}
```

结果c=8

答:

如果某个函数内部定义了与全局变量同名的变量，则此内部变量优先，
外面的全局变量被屏蔽。

089 实参数组元素个数与形参数组元素个数不一致问题

问:

若有以下调用语句,则不正确的fun函数的首部是:
A) void fun(int m, int x[ ]) B) void fun(int s, int h[41])
C) void fun(int p, int *s) D) void fun(int n, int a)

```
main( )
{  int a[50], n;
     . . .
    fun( n, &a[9] );
     . . .
}
```

答案为什么不是B

答:

B) void fun(int s, int h[41])
函数头中的形参如果是数组，它实际是一个假的数组，本质是一个指针，
相当于
void fun(int s, int *h)
因此，其中的数组元素个数只是一个摆设，并没有意义的，即使写成0都没事。

090 全局变量未赋值时，它的初始值是0吗?

问:

如果定义了一个全局变量，没有赋值，用到时候是按0算么？如果是局部变量呢?

答:

全局变量、局部静态变量在定义时如果没有赋值，则初始值=0；
局部动态变量在定义时如果没有赋值，则初始值=随机值。

091 如何得到某个文件夹中的所有文件名?

问：

小白老师，最近在做实验数据处理，要从很多txt文件里提取字符数据。
自己写了个小程序实现了这个功能。
不过现在想提高一下，想问问老师有没有哪个函数可以实现搜索当前目录下的文件然后获取文件名。

答：

以下程序搜索C:\TC文件夹，在屏幕上输出该文件夹下面的所有文件名。
注意文件名是8.3格式，不支持长文件命；另外本程序不搜索子文件夹，
对子文件夹一概忽略。如果要搜索子文件夹的话，那一般需要用递归来
处理。

```
#include <stdio.h>
#include <dir.h>
#include <dos.h>
void main()
{
   struct ffblk fb;
   char search_pattern[]="c:\\tc\\*.*"; /* 要搜索的文件夹 */
   int attrib = 0xFF;   /* 文件属性为0xFF表示搜索各种属性的文件，包括卷标、子文件夹在内 */
   char filename[100];
   if(findfirst(search_pattern, &fb, attrib) != 0) /* 若搜索不成功或找不到匹配文件 */
   {
      puts("Search failed!");
      return;
   }
   do {
      if(fb.ff_attrib & FA_LABEL || fb.ff_attrib & FA_DIREC) /* 排除卷标以及子文件夹 */
           continue;
      strcpy(filename, fb.ff_name);
      puts(filename);
   } while(findnext(&fb)==0); /* 若搜索成功，则继续搜 */
}
```

以下是可以搜索子文件夹的程序，子文件夹最多16层且空的子文件夹不予理睬：

```c
#include <stdio.h>
#include <string.h>
#include <dir.h>
#include <dos.h>
char * search_file(char path[], struct ffblk fb[],
         int *first_search, int *plevel);
void main()
{
   struct ffblk fb[16];
   int first_search=1, level=0;
   char path[256]="c:\\tc";
   char *p;
   clrscr();
   while((p=search_file(path, fb, &first_search, &level)) != NULL)
      puts(p);
}
char * search_file(char path[], struct ffblk fb[],
         int *first_search, int *plevel)
{
   int status, n;
   static char filename[256], pattern[256];
   char *p;
   n=strlen(path);
   if(path[n-1]=='\\')
      path[n-1]='\0';
   strcpy(pattern, path);
   strcat(pattern, "\\*.*");
   if(*first_search)
   {
      *first_search = 0;
      if((status=findfirst(pattern, &fb[*plevel], 0xFF)) != 0)
         goto prev_level;
   }
   else
   {
      if((status=findnext(&fb[*plevel])) != 0)
         goto prev_level;
   }
   do
   {
      if(fb[*plevel].ff_attrib & FA_LABEL)
         continue;
      if(fb[*plevel].ff_attrib & FA_DIREC)
      {
         if(fb[*plevel].ff_name[0]=='.')
            continue;
      }
      break;
   }  while((status=findnext(&fb[*plevel]))== 0);
   if(status != 0)
      goto prev_level;
```

```
    if((fb[*plevel].ff_attrib & FA_DIREC) == 0)

    {
       return strcat(strcat(strcpy(filename, path),"\\"),fb[*plevel].ff_name);
    }
    strcat(strcat(path, "\\"), fb[*plevel].ff_name);
    (*plevel)++;
    *first_search = 1;
    p =  search_file(path, fb, first_search, plevel);
    if(p != NULL)
       return p;
prev_level:
    do
    {
       (*plevel)--;
       if(*plevel < 0)
          return NULL;
       p=strrchr(path, '\\');
       if(p!=NULL)
          *p='\0';
       *first_search=0;
       p=search_file(path, fb, first_search, plevel);
    } while(p==NULL);
    return p;
}
```

092 如何判断键盘是否有键按下但不停止程序继续执行?

问:

在c语言中怎么样判断在一定时间内 没有从键盘上输入东西 或者说没敲过键盘 用getch（）好像不行

答:

```
#include <stdio.h>
#include <bios.h>
main()
{
   unsigned short int key;
   while(bioskey(1)==0); /* 当没有键按下时，bioskey(1)返回0，否则返回1 */
   key=bioskey(0); /* bioskey(0)用来读取一个键，具体Ctrl+F1查帮助 */
   printf("key=%04X\n", key);
}
```

我这个是演示程序啊，如果我不用while循环的话，你就看不出bioskey(1)的效果了。
你自己在用的时候可以把bioskey(1)放到你自己的循环里面。
比如:

```
int done=0;
int key;
while(!done)
{
   if(bioskey(1) != 0)
   {
      key=bioskey(0);
      switch(key)
      {
          ...
      }
   }
   ...
}
```

093 void *类型指针变量与其它类型指针变量的相互赋值问题

问:

为什么void类型，malloc还可返回一个指针?
什么意思?

答:

void *类型指针变量与其它类型指针变量的相互赋值问题
void 与 void *
是两回事情。void 不能用来定义变量，只能用来声明某个函数无返回值或者声明函数
无参数。而void *可以用来定义变量，也可以用来声明函数的返回值类型及函数的参数类型。

void * 一般称为通用指针类型，表示该指针所指向的对象类型不确定。
在C语言中，该类型的指针变量与其它类型的指针变量可以
互相赋值而不需要进行强制类型转换。
在C++中，该类型的指针要赋值给其它类型的指针变量时，要强制类型转换;
但把其它类型的指针赋值给该类型的指针变量时，则不需要类型转换，例如:

```c
#include <stdio.h>
main()
{
  int x=123, y=456;
  int *p;
  void *v;
  v = &x; /* 这个语句在C及C++中均正确 */
  p = &y;
  v = p; /* 这个语句在C及C++中均正确 */
  p = v; /* 这个语句在C中正确，但在C++中不正确 */
  p = (int *) v; /* 在C++中应该这样写才对 */
  printf("%d\n", *p);
  return 0;
}
```

094 如何动态分配数组?

问:

数组可以定义多维的,
比如arry［3］ ［5］ ［3］
这样定义在有时候很好用;
但是这是静态空间,
如果用malloc申请了一段空间,
是否也可以这样用?？

显然p＝malloc（）；
之后p［i］ ［j］ ［k］根本不知道是哪个

或者说是不是可以
int * p=int［3］ ［5］ ［3］；
p＝(int*)malloc（）；
（貌似没道理）

所以我动态空间只能痛苦的
p［（i＊L_MAX+j）＊M_MAX＋k］
用这样的方式调用

如果定义结构还好一点
struct my ｛｝；
然后可以my＊ p＝（my＊）malloc（）；
方便很多,

但是很多时候为了方便调用定义一个结构不划算

小白有没有其他办法？？？

答：

简单一点，用二维数组为例。
例如想要动态分配一个3*4的二维整型数组，则可以这样：

```
int  (*p)[4];
p = ( int (*)[4] ) malloc(sizeof(int)*3*4);
```

以后可以用p[i][j]引用第i行第j列。
这里的强制类型转换不容易看明白，也可以写成如下稍微简单的：

```
typedef   int  (*AP)[4];
AP p;
p = (AP) malloc(sizeof(int)*3*4);
```

以二维数组为例，如果不知道每行的列数，那就没有办法计算a[i][j]在哪里。
你可以随便分配一块空间，可以把该空间当成二维、三维来用，但是引用
比如a[i][j][k]的时候，只有i可以为任意值，j、k所对应的总行数及总列数必须是固定的。

```
typedef int (*T2)[4];
typedef int (*T3)[3][4];
char *a;
T2 p;
T3 q;
a = (char *)malloc(1000);
p = (T2)a;
q = (T3)a;
```

以后可以引用p[i][j]、q[i][j][k]。

095 VC下strlwr函数参数的指针问题

问：

刚才室友问我一个简单的程序的错误，我调试看了下，发现在调用strlwr函数时发生访问冲突。开始也没弄明白怎么回事，然后写个程序OD了一下，发现原因了，下面逐步道来。
先看这个简单的程序：

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char *s = "Hello,World";
    printf("%s\n",strlwr(s));  /* strlwr函数用于将字符串中的大写字母转换为小写字母 */
}
```

怎么样，简单吧，可是有错误 ==b 在printf处提示访问冲突。
显然，printf写得是没有问题的，那么问题发生在strlwr函数调用上。
于是进入strlwr函数内部（VS08的版本），略去不相关的代码，其中string是它的参数：

```c
    char * cp;

    /* validation section */
    _VALIDATE_RETURN(string != NULL, EINVAL, NULL);

    for (cp=string; *cp; ++cp)
    {
        if ('A' <= *cp && *cp <= 'Z')
            *cp += 'a' - 'A';
    }

    return(string);
```

可以看到，这个函数实际是对参数传入的字符串进行直接操作的，它把原字符串中的大写字母转换成小写字母，再把这个字符串的指针返回。那么发生访问冲突，就是因为改写了原字符串的内容了。
再看主函数中这个字符串的定义：char *s = "Hello,World";
一般想变量变量它的值总是能改变的，那么这里为什么不能变了呢？然后打开OD调试刚才的程序，执行到如下位置：

```
000B1000  /$  68 F4200B00   push    000B20F4                        ; /s = "Hello,w
000B1005  |.  FF15 A4200B00 call    dword ptr [<&MSVCR90._strlwr>]  ; \_strlwr
000B100B  |.  50            push    eax                             ; /<%s>
000B100C  |.  68 04210B00   push    000B2104                        ; |format = "%s'
000B1011  |.  FF15 A0200B00 call    dword ptr [<&MSVCR90.printf>]   ; \printf
```

发现"Hello,world!"这个字符串并不是在堆栈中的，它的地址是000B20F4。然后用peid查看刚才的生成的exe文件，发现"Hello,world!"它的地址是在.rdata段中，段的Characteristics是40000040，意义为IMAGE_SCN_CNT_INITIALIZED_DATA || IMAGE_SCN_MEM_READ。于是它是只读的，所以在strlwr函数执行中发生访问冲突。

于是得到上面错误的原因。
[color=white]

答：

找到错误原因，于是考虑更正。
方法一是用字符数组替换上面的字符指针，程序如下：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
        char t[] = "Hello,world!";
        printf("%s\n",strlwr(t));
}
```

运行OK，OD查看有这样一段代码：

```
0006102B  |.  8D0C24        lea     ecx, dword ptr [esp]
0006102E  |.  51            push    ecx                              ; /s
00061037  |.  FF15 A4200600 call    dword ptr [<&MSVCR90._strlwr>]   ; \_strlwr
```

在地址区查看ecx指向内存的值，正是堆栈中的Hello,world!，所以这样程序OK。在这之前有一段代码，对堆栈中的单元赋值，如下：

```
0006100E  |.  8B0D F8200600 mov     ecx, dword ptr [620F8]
00061014  |.  A1 F4200600   mov     eax, dword ptr [620F4]
00061019  |.  8B15 FC200600 mov     edx, dword ptr [620FC]
0006101F  |.  894C24 04     mov     dword ptr [esp+4], ecx
00061023  |.  890424        mov     dword ptr [esp], eax
00061026  |.  A0 00210600   mov     al, byte ptr [62100]
0006102F  |.  895424 0C     mov     dword ptr [esp+C], edx           ; |
00061033  |.  884424 10     mov     byte ptr [esp+10], al            ; |
```

查看620F4单元的值，于是Hello,world又出现了。这个Hello,world也是在.rdata段中，同样是只读的。

方法二：

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
char *s;
s = (char *)calloc(12,sizeof(char));
strcpy(s,"Hello,World");
printf("%s\n",strlwr(s));
}
```

程序同样OK。这个程序OK的原因很简单，因为s指向的空间是用calloc申请的，显然是在堆栈中的空间，于是是可读可写的。

综上，得出结论，程序中所有用引号引起来的字符串，其原始地址是在程序的.rdata段，这个段是只读的，如果将指针直接指向这个段的地址，并且期待使用写操作的话，那么就会是一个访问冲突。因此，下面代码都是错误的（VC下）：

```c
char *s = "Hello,world";
*(s+1) = 'E';  /* 访问违规 */
```

再如：

```c
char *s = "Hello,world";
strcat(s,"Hello");  /* 访问违规 */
```

只有在函数原型声明参数为const char *时，使用这些函数是没有问题的，如：

```c
char *s = "Hello,world";
int l;
l = strlen(s);  /* OK，因为strlen函数的原型是size_t strlen([color=red]const[/color]
```

引用自cc98的**如嫣**同学

不同的编译器对这种字符串常量应该有各自的方式，
不能说VC++就是标准的或者是好的，
我就是不喜欢它的一些武断的处理方式(如强制对结构进行4字节对齐)。

C++Builder对此程序进行编译并运行后，完全没有问题。
显然，Borland公司贯彻了自Turbo C以来的一贯风格。

如果要让VC++乖乖地干活，就得给它来点硬的：

```
#pragma comment(linker,"/SECTION:.rdata,RW")
```

把它的.rdata节属性弄成可以读写，它就没有办法了。

096 40012题{double y=1;if(y/10==0.1)}已修复

经过两天不懈努力，我找到了系统误报答案错误的原因。
简单归纳如下：
练习服务器内置的编译器是GCC，它本来既支持C89标准也支持C99标准，
但是练习系统给它指定了-ansi编译开关，意思就是要它按C89标准编译。
按照此标准，它会支持long double这种类型，此类型比double还要厉害，
占用10字节(80位)内存，当然用这种类型表达的小数范围也大大增加，但是同时
也带来一个副作用：
例如，当我们把一个表达式与某个double类型的变量或常数比较时：

```
#include <stdio.h>
main()
{
   double x=0.1;
   double y=1;
   if(y/10 == x)
      puts("Equal!");
   else
      puts("Not equal!");
}
```

y/10这个表达式的值会临时保存到一个80位的浮点寄存器中，
而x的类型是double类型即64位，因此==两边属于混合类型运算，
必须要对其中一方进行转化，统一双方的类型才能进行比较。那么
究竟是把双方都统一成64位浮点数还是80位浮点数？答案是意想
不到的80位，结果原本只有64位的x值被放大到80位后出现误差，
变成以下这个值：
+0.1000000000000000056
该值在内存中对应的16进制机器码是：
0x3ffbcccccccccccd000
而y/10的计算结果即0.1的机器码是：
0x3ffbcccccccccccccd
显然两者是不相等的。

修正这个毛病一个办法可能是改编译开关，但是我没有这样做，
而是直接改库函数，把libc.a这个库中的一个函数代码改了，

使得浮点数处理器的精度从80位减少到64位，应该可以彻底杜绝此类问题的两次发生。

同理，TC也有这个毛病，我把它修理好之后会放到主页上，请大家在以后上机的时候重新下载。

但是0.1 != 0.1 这个问题很严重。
VC不会出现这样的问题，查了一下资料，发现VC把long double这个类型取消了，即使你定义了一个某个变量为long double类型，VC也会强制把它转化成double类型。因此当double x=1;再比较x/10 == 0.1时，VC是把两者都转成double类型来比，这样做也符合C语言中关于混合类型运算的规范。
反之，TC与GCC的做法非常奇怪：

```
double x=1, y;
if(x/10 == 0.1)
   puts("Equal");
else
   puts("Not equal"); /* 结果输出Not equal */

y = x/10;
if(y == 0.1)
   puts("Equal"); /* 结果输出Equal */
else
   puts("Not equal");

if(fabs(x/10) == 0.1)
   puts("Equal"); /* 结果输出Equal */
else
   puts("Not equal");
```

后面两段程序都会输出"Equal"是因为把x/10赋值给y之后，y本身的类型为double，所以最后是double与double比，因此结果没问题；而fabs()这个函数的返回值类型也为double，所以结果也为double与double比，因此结果也正确。

再来看上次一个同学给我程序：

```
#include <stdio.h>
#include <math.h>
int main(void)
{
    int i, flag;
    int repeat, ri;
    double eps, sum;
    scanf("%d", &repeat);
    for(ri = 1; ri <= repeat; ri++)
    {
        scanf("%le", &eps);
        i=-1;
        sum=0;
        flag=1;
        do
        {
            i++;
            sum=sum+(double)flag/(3*i+1);
            flag=-flag;
        } while(1.0/(1+3*i)>=eps);
        printf("sum = %.6f\n", sum);
   }
}
```

此程序提交后死活通不过，但是如果把第21行改成：

while(fabs(1.0/(1+3*i))>=eps);

就可以通过。

*或者，把1.0/(1+3*i)先赋值给一个double变量，*

再与eps比较，则提交也可以通过。

输入测试数据如下：

3

1e-2 1e-4 1e-3

TC与GCC的输出为：

```
sum = 0.830724
sum = 0.835599
sum = 0.835150
```

而VC的输出为：

```
sum = 0.840433
sum = 0.835699
sum = 0.836147
```

通过跟踪，很容易发现当比较到0.1 >= 0.1时发生问题。
现在练习服务器中经过我修理的GCC以及我主页上的
修正版TC都可以得到与VC一样的结果。

平时用TC做题的同学请尽快从我主页下载修正版。

097 上机题目汇总

- 1. http://file.cc98.org/uploadfile/2009/6/7/0242237649.doc
- 2. http://file.cc98.org/uploadfile/2009/6/6/1432779350.doc
- 3. http://file.cc98.org/uploadfile/2009/6/6/14321750802.doc
- 4. http://file.cc98.org/uploadfile/2009/6/6/14314960992.doc
- 5. http://file.cc98.org/uploadfile/2009/6/6/1431268613.doc
- 6. http://file.cc98.org/uploadfile/2009/6/6/14313280426.doc
- 7. http://file.cc98.org/uploadfile/2009/6/6/16482253766.doc
  以上是上课布置的作业。
  全部练习题在此下载：
- 1. http://file.cc98.org/uploadfile/2009/6/6/15261042537.zip

098 函数声明在main()外面与里面有啥区别?

问：

- 1. 函数声明放在main里面和外面有什么区别?
- 2. 还有负数的左移， 右移是怎么计算?

答：

- 1. 放在main外面表示此声明是全局的，否则就是局部的。
- 2. 左移1位 == * 2，右移1位 == / 2。如果转成二进制计算，则负数右移要在左边补1，正数右移左边补0。**负数左移与正数左移没有区别。**

099 数组名为什么不能被赋值?

5. 不正确的赋值或赋初值的方式是_____。
   A．char str[]="string"； B．char str[7]={'s', 't', 'r', 'i', 'n', 'g'};
   C．char str[10]；str="string"； D．char str[7]={'s', 't', 'r', 'i', 'n', 'g', '\0'};
   为什么C是错的?

因为C中的str是数组名，相当于它的首元素地址，即一个常数，而常数是不能被赋值，
也不能被改变的。

100 结构指针*p->y=3.0为啥错(其中y是float)?

9. 对于以下韵变量定义，表达式_____是不正确的。
   Struct node{float x,y;}point,*p=&point;
   A．p->x=2.0 B．(*p).y=3.0 C．point.x=2.0 D．*p->y=3.0
   为什么选D啊

因为->比* 优先， 因此*p->y相当于 *(p->y)。而y是一个小数，不是指针，所以，它前面再加个*就是错的。

101 为什么不能用%f格式输出一个字符串?

8. 执行语句printf(“_____”,”hello”); 将得到出错信息。
   A．%u B．%s C．%x D．%f
   为什么选D啊

因为%f是小数。"hello"在这里其实是一个地址，
既然是一个地址，那它其实就是一个整数。地址不可能有小数的。
此题出题者肯定是想当然了。
因为"hello"是一个地址，在32位编译器中，长度与float相等，都是32位，
输出结果最多是把整数解释成小数格式来输出；
在16位编译器中，长度与float有出入，例如在TC中"hello"的地址是16位，
而float是32位，实际输出结果时可能会造成内存越界。

102 FROG、SEAL、LION、LAMB构成defy()-08程序阅读1

```
#include <stdio.h>
#include <string.h>
char (* defy(char *p) ) [5]
{
int i;
for(i=0; i<3; i++)
    p[strlen(p)] = 'A';
return (char (*)[5])p + 1;
}
void main()
{
char a[]="FROG\ØSEAL\ØLION\ØLAMB";
puts( defy(a)[1]+2 );
}
```

1. 先看函数头。按“先右边再看左边”法，找到defy，
   它的右边是()，所以defy是一个函数，该函数有一个参数 char *p；*再看左边是一颗*，所以defy()返回一个指针；

再看右边是[5]，所以该指针指向一个数组，该数组一共5个元素；
再看左边是char，所以每个元素的类型是char。
概括一下，defy()的返回值是一个数组指针，该指针所指向的对象是由5个char所构成的一维数组。

2. 再看实参。实参a很简单，就是数组a的首地址，即&a[0]。
3. 函数defy()里的循环。
   实参a代入形参p后，p指向a[0]。
   第1次循环，strlen(p)的值=4，因为FROG后面有一个'\0'，
   而p[strlen(p)]='A'的作用是把这个'\0'填成'A'；
   第2次循环，SEAL后面的'\0'被填成'A'；
   第3次循环，LION后面的'\0'被填成'A'。
4. 函数defy()的返回值。
   (char ()*[5])*是一个强制类型转换，表示目标类型为数组指针，该指针指向由5个char构成的一维数组。注意这里的必须用()括起来，强调指针优先，理解的时候可以设想*后面有一个变量p；*
   (char ()[5])p 就是把p的类型转换成数组指针，于是p就变成行指针了，再加1，则p指向下一行，即"SEAL"。
   "FROG"
   p-> "SEAL"
   "LION"
   "LAMB"
5. puts()里面的值。
   defy(a)就是上述那个p；
   defy(a)[1]就是*(defy(a)+1)，即"LION"那一行本身；
   defy(a)[1]+2就是"LION"的首地址+2，即'O'的地址；
   注意LION后面的'\0'已经被defy()里面的循环填成'A'；
   因此最后的输出结果为"ONALAMB"及回车。

P.S. 此题那一年的杀伤力据我初步估计达95%以上，我自己教的班也未能幸免，令我非常失望。我希望今年我的班上的同学在不看上述解说的情况下，至少有50%可以做出来，只是希望而已。指针其实不难，只不过不用心的人是学不成的，C语言也一样。

103 求问08卷选择题第10题（有关strcmp）

问：

执行以下语句后，
int n = strcmp("ABC", "AB");
则以下关于n值的描述哪一个是正确的?

A. n=0
B. n=1
C. n>1
D. n<0

答案是c，我不明白。。。
且用vc编译后 n输出等于1啊。。

答：

当时出题的时候没有用VC，只测试了TC、C++Builder，结果n的值
都是67即'C'。
现在用VC试了一下，结果确实是1，再用GCC试了一下，结果也是1。
嗯，承认此题出得不好。

104 关于数组指针是否就是指针的指针的探讨

假定已经定义了以下数组、变量：

```
int a[2][3]={{1,2,3}, {4,5,6}};
int (*p)[3],  *q[2];
int **pp;
p = a; /* 即p=&a[0]; */
q[0]=&a[0][1];
q[1]=&a[1][1];
pp = q; /* 即pp = &q[0]; */
```

有些同学觉得 数组指针==指针的指针，他们的理由如下：
因为p -> a[0]
且a[0]可以理解为&a[0][0]
所以p -> &a[0][0]
而&a[0][0]是一个指针，所以p当然是指针的指针

再还有一个理由：可以引用 **p 来得到a[0][0]的值
既然变量p前面可以放2颗*，则p当然是指针的指针。

我的反驳：
任何一个指针所指向的对象必须占用内存空间，并且该对象必然有它自己的地址，
否则怎么让指针指向它呢?
如果p -> &a[0][0]成立，则&a[0][0]必须是存在于内存中的一个实体，
但事实上&a[0][0]只是a[0][0]的地址，根本不占用内存空间，
而且&a[0][0]也没有自己的地址。
所以，p只能指向a[0]这一行，而不能让p指向&a[0][0]。

第2个理由：
如果p是指针的指针，则p++以后，p应该指向下一个指针，
但事实上p++以后将使p指向下一行即a[1]。

最后的结论：
数组指针 不是 指针的指针
指针数组名 是 指针的指针 (如上面的q)

P.S. 有同学问&a[0]的值是什么？它与&a[0][0]有没有关系？
答：&a[0]的值在数值上等于&a[0][0]，但是它们两个类型不同。

105 表达式'A'+'B'的类型为什么是int?

请看书上关于混合类型运算中类型的自动提升一节或看我主页“知识要点”：
凡是char一律提升为int
凡是float一律提升为double

106 复杂字符串求strlen()、sizeof()的自助解决方法

这种题求人不如求已，完全可以通过自己编个程序输出各个字符的ASCII码
来解决：

```
#include <stdio.h>
#include <string.h>
main()
{
  char s[]="\\\25ab,0\n";
  int i, n;
  n = strlen(s);
  printf("strlen(s)=%d, sizeof(s)=%d\n", n, sizeof(s));
  for(i=0; i<n; i++)
     printf("%d ", s[i]);
}
```

或者在TC中Ctrl+F7观察数组s时输入以下内容：
s, d
或
s, x
分别表示以10进制和16进制格式观察s的内容。

107 表达式优先级、结合性疑问：优先级高就先计算吗?

问：

比如说0&&++y,那么++y是不进行运算的,可是++的优先顺序不是比&&高么,为什么要先考虑&&呢?于是如果左边有一堆很长的表达式该怎么考虑呢?
还有就是先计算左边的比如 x*y+z/t&&y+z
那么左边的表达式是x*y+z/ (t)&&y+z 还是x*y+ (z/t)&&y+z 还是(x*y+z/t)&&y+z 呢?
哪位达人系统讲解下?

答:

优先级及结合性只是表示谁与谁先括号，即谁与谁先结合起来看作一个整体的意思，
计算顺序还是按从左到右来。
比如： x*y+z/t&&y+z 根据优先级及结合性添上括号后，就变成：
((x*y) + (z/t)) && (y + z)
最后就变成了 A && B 这么一种形式，于是再按从左到右顺序进行计算
(1) x*y
(2) z/t
(3) (x*y) + (z/t) 如果这步算出0，则后面不再计算，这个属于&&的特殊情况*
(4) (y + z)
*(5) ((x*y) + (z/t)) && (y + z)*

108 二维数组如何转成指针的指针?

问:

如果我有一个函数int apc（int **p)
还有一个二维数组a[i][j];
我希望将a的地址传给p怎么办呢？就是说我想在apc函数中通过p来改变数组a[i][j]

我试过了，直接用apc（a）不行，但是有个要求apc必须那么写成int apc（int **p)

编译环境是VC6.0
（唉，apc这个函数是我从网上down下来的一个匈牙利算法，现在写个程序必须要用）

答:

1. 二维数组 不等于 指针的指针
2. 指针数组 可以看作 指针的指针
3. 假定你的二维数组a由2行x3列构成，则可以用以下方式把a传给p：

```c
void f(int **p)
{
   int i, j;
   for(i=0;i<2;i++)
   {
      for(j=0;j<3;j++)
         printf("%d ", p[i][j]);
      printf("\n");
   }
}
main()
{
       int a[2][3]={{1,2,3},{4,5,6}};
   int *b[2];
   int i;
   for(i=0; i<2; i++)
      b[i] = a[i];
   f(b);
}
```

109 如何防止scanf("%d",&x)时因输入字符而陷入死循环?

问:

```c
#include <stdlib.h>
#include <stdio.h>
main()
{
char exit='y';
int computer, user;
randomize();
while(exit!='n' && exit!='N')
  {
  computer=random(100)+1;
  for(user=0; user!=current; )
    {
    scanf("%d", &user);
    if(user>computer) printf("too big\n");
    else if(user<computer) printf("too small\n");
    else printf("You got it!\n");
    }
  printf("Wanna play again?(Y/N)");
  scanf(" %c", &exit);
  }
}
```

这是我编的猜数游戏的扩展，根据另一个问题贴中白老师的解答，已经用scanf(" %c", &exit)中的空格解决了死循环的问题

在没有空格时，会因scanf("%c", &exit)跳过而直接回到scanf("%d", &user)，此时输入"y"会进入死循环
但是为什么当整数型字符被赋值为非数字的字符时会出现这样的混乱呢?
我在单步跟踪时发现，当user被赋为"y"而出现混乱时，程序再次运行到scanf("%d", &user)时，它也不会让你输入。这其中的原理是什么?

那如果在玩这游戏时，因为操作错误而在输入数字时输入了字母，应该也会造成同样的混乱。
能否在编程时就避免这种错误、混乱的发生?

答:

因为你输入的字符本来就不可能被%d接收的，只能
仍旧留在输入缓冲区中。
一个有效的解决方法是在scanf("%d", &c);前面加以下语句:
fflush(stdin);
表示把输入缓冲区清空。
这个函数调用在TC及VC中均有效。
如果在linux下面，则应该调用以下语句:
__fpurge(stdin);

110 BMP Quick Show in 12h VGA mode

程序中用到的16色BMP样本文件:
http://www.cc98.org/bhh/face.bmp
在写本程序过程中，参考了以下文献:
Programmer's Guide to PC & PS/2 Video Systems(Page 117-118), Richard Wilton,
Microsoft Press, 1987. ISBN 1-55615-103-9

```
/*******************************************************/
/* BMP Quick Show in 12h VGA Mode(640*480*16)       */
/* copyright (c) Black White                        */
/* email: iceman[url="/user/name/zju.edu.cn"]@zju.edu.cn [/url]
/* This program is for teaching purpose only,       */
/* if you want to reference it, please keep         */
/* the above copyright information.                 */
/* How to compile:                                  */
/* Turbo C -> Options -> Compiler -> Model -> Large */
/* If you want to show bmp pixel by pixel, please   */
/* change SLOW to 1.                                */
/*******************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <mem.h>
#include <dos.h>
#include <bios.h>
#define SLOW 0 /* change it to 1 if you want the slower effect */

void text(void)
{
   _AX=0x0003;
   geninterrupt(0x10);
}

void vga(void)
{
   _AX=0x0012;
   geninterrupt(0x10);
}

char  * video_addr(int x, int y, char *mask)
{
   char  far *v = (char far *)0xA0000000;
   v += y*(640/8) + x/8;
   *mask = 1 << 7-(x%8);
   return v;
}

void put_pixel(int x, int y, int color)
{
   char far *v, mask, c;
   v = video_addr(x, y, &mask);
   outportb(0x3CE, 8);    /* 8=bit mask register */
   outportb(0x3CF, mask); /* enable 1 bit to update */
   outportb(0x3CE, 5);    /* 5=mode register */
   outportb(0x3CF, 2);    /* 2=0000 0010 */
                          /* Bit1Bit0=write mode 2, Bit3=read mode 0 */
   outportb(0x3CE, 3);    /* 3=data rotate/function select register */
   outportb(0x3CF, 0);    /* Bit4Bit3=00 replace, Bit4Bit3=01 AND */
```

```
52                              /* Bit4Bit3=10 OR, Bit4Bit3=11 XOR */

53      c = *v;                 /* latch one byte from each bit plane */
54      *v = color;
55      outport(0x3CE, 0xFF08);/* default bit mask */
56      outport(0x3CE, 0x0005);/* default mode register */
57      outport(0x3CE, 0x0003);/* default function select */
58  }
59
60  unsigned char  ** read_dissolve_file(char *filename)
61  {
62      FILE *fp;
63      static unsigned char  *p[5];
64      unsigned char  *b, c;
65      unsigned long u;
66      long size, wbit, hbit, wbyte, plane_bytes, plane_wbyte;
67      long i, j;
68      int k, t;
69      fp = fopen(filename, "rb");
70      if(fp == NULL)
71          return NULL;
72      fseek(fp, 0, SEEK_END);
73      size = ftell(fp);
74      fseek(fp, 0, SEEK_SET);
75      p[0] = malloc(size);
76      if(p[0] == NULL)
77      {
78          fclose(fp);
79          return NULL;
80      }
81      fread(p[0], 1, size, fp);
82      fclose(fp);
83      if(*(short int  *)(p[0]+0x1C) != 4)
84      {
85          puts("Not a 4-bit color bmp file!");
86          free(p[0]);
87          return NULL;
88      }
89      wbit = *(long  *)(p[0]+0x12);
90      hbit = *(long  *)(p[0]+0x16);
91      wbyte = (wbit*4+7) / 8;
92      if(wbyte * hbit > size - *(long  *)(p[0]+0x0A))
93      {
94          puts("Incorrect bmp size!");
95          free(p[0]);
96          return NULL;
97      }
98      b = p[0] + *(long  *)(p[0]+0x0A);
99      plane_wbyte = (wbit+7) / 8;
100     plane_bytes = plane_wbyte * hbit;
101     for(i=1; i<=4; i++)
102     {
```

```
103         p[i] = malloc(plane_bytes);
104         if(p[i] == NULL)
105         {
106            for(j=i-1; j>=0; j--)
107               free(p[j]);
108            puts("malloc() failed!");
109            return NULL;
110         }
111         memset(p[i], 0, plane_bytes);
112      }
113      for(i=0; i<hbit; i++)
114      {
115         for(j=0; j<wbyte; j+=4)
116         {
117            u = 0;
118            for(t=0; t<4; t++) /* combine 4 continuous bytes to 1 double word */
119            {                  /* in Big-Endian format */
120               u <<= 8;
121               u |= *(b + i*wbyte + j + t);
122            }
123            for(k=0; k<8; k++) /* dissolve 32 bits to 8 bits per bitmap plane */
124            {                  /* 31 30 29 28 27 26 25 24 23 22 21 20 19
125                                   |           |           |           |
126                                   \___________|___________|___________/
127                                               |
128                                         bit 7 6 5 4 for plane 1
129
130                               31 30 29 28 27 26 25 24 23 22 21 20 19 18
131                                   |           |           |           |
132                                   \___________|___________|___________/
133                                               |
134                                         bit 7 6 5 4 for plane 2
135
136                               31 30 29 28 27 26 25 24 23 22 21 20 19 18 17
137                                      |           |           |           |
138                                      \___________|___________|___________/
139                                                  |
140                                         bit 7 6 5 4 for plane 3
141
142                               31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16
143                                         |           |           |           |
144                                         \___________|___________|___________/
145                                                     |
146                                         bit 7 6 5 4 for plane 4
147                            */
148               for(t=1; t<=4; t++)  /* total 4 bitmap planes */
149               {
150                  *(p[t] + i*plane_wbyte + j/4) <<= 1;
151                  u = _lrotl(u, 1);
152                  *(p[t] + i*plane_wbyte + j/4) |= u & 1;
153               }
```

```
154             }
155         }
156     }
157     return p;
158 }
159
160 void show_plane(unsigned char *p, unsigned char far *v, long w, long h)
161 {
162     long i, j;
163     unsigned char c;
164     unsigned char *rp;
165     unsigned char far *vp;
166     for(i=0; i<h; i++)
167     {
168         vp = v + i*640/8;
169         rp = p + (h-1-i) * w;
170         for(j=0; j<w; j++)
171         {
172             c = *vp;
173             *vp++ = *rp++;
174         }
175     }
176 }
177
178 void set_palette(unsigned char *p)
179 {
180     int i;
181     unsigned char index[]={0,1,2,3,4,5,20,7,56,57,58,59,60,61,62,63};
182
183     for(i=0; i<16; i++)
184     {
185         outportb(0x3C8, index[i]);
186         outportb(0x3C9, p[2]/4);
187         outportb(0x3C9, p[1]/4);
188         outportb(0x3C9, p[0]/4);
189         p+=4;
190     }
191 }
192
193 void reset_palette(void)
194 {
195     int i;
196     unsigned char index[]={0,1,2,3,4,5,20,7,56,57,58,59,60,61,62,63};
197     unsigned char r, g, b;
198     for(i=0; i<8; i++)
199     {
200         r = 42 * (i & 0x04 ? 1 : 0);
201         g = 42 * (i & 0x02 ? 1 : 0);
202         b = 42 * (i & 0x01 ? 1 : 0);
203         outportb(0x3C8, index[i]);
204         outportb(0x3C9, r);
```

```
        outportb(0x3C9, g);
        outportb(0x3C9, b);
        outportb(0x3C8, index[15-i]);
        outportb(0x3C9, 63-r);
        outportb(0x3C9, 63-g);
        outportb(0x3C9, 63-b);
    }
}

unsigned char rotr_byte(unsigned char c, int n)
{
    return (c >> n) | (c << 8-n);
}

main()
{
    int t, k;
    unsigned char  far *v = (unsigned char  *)0xA0000000;
    unsigned char  **p;
    unsigned char  *palette, *b;
    unsigned char  c, m, latch;
    long int wbit, hbit, wbyte, plane_wbyte, i, j;
    vga();
    outportb(0x3CE, 5);
    outportb(0x3CF, 0);    /* write mode=0 */

    outportb(0x3CE, 3);
    outportb(0x3CF, 0x10); /* OR */

    outportb(0x3CE, 0);
    outportb(0x3CF, 0);    /* set/reset = 0 */

    outportb(0x3CE, 8);
    outportb(0x3CF, 0xFF); /* mask */


    p = read_dissolve_file("face.bmp");
    if(p == NULL)
        return;
    wbit = *(long  *)(p[0]+0x12); /* BMP's width in pixel */
    hbit = *(long  *)(p[0]+0x16); /* BMP's height in pixel */
    wbyte = (wbit*4+7) / 8;       /* bytes per line for BMP */
    plane_wbyte = (wbit+7) / 8;   /* bytes per line for each bitmap plane */
    palette = p[0] + 0x0E + *(long *)(p[0]+0x0E);
    set_palette(palette);

#if SLOW
    /* show bmp pixel by pixel */
    outportb(0x3CE, 5);
    outportb(0x3CF, 2);    /* write mode=2 */
    outportb(0x3CE, 3);
```

```
256     outportb(0x3CF, 0x00); /* REPLACE */
257
258     b = p[0] + *(long  *)(p[0]+0x0A);
259     for(i=0; i<hbit; i++)
260     {
261        m = 0x80;
262        for(j=0; j<wbyte; j++)
263        {
264           c = *(b + (hbit-1-i)*wbyte + j);
265           for(k=0; k<2; k++)
266           {
267              outportb(0x3CE, 8);
268              outportb(0x3CF, m); /* mask */
269              m = rotr_byte(m, 1);
270              latch = *(v + i*640/8 + j/4);
271              c = rotr_byte(c, 4);
272              *(v + i*640/8 + j/4) = c & 0xF;
273           }
274        }
275     }
276  #else
277     /* show bmp plane by plane */
278     for(t=1; t<=4; t++)
279     {
280        outportb(0x3CE, 1);
281        outportb(0x3CF, ~(8 >> t-1) & 0xF);  /* enable set/reset = 0111 */
282                                             /* enable set/reset = 1011 */
283                                             /* enable set/reset = 1101 */
284                                             /* enable set/reset = 1110 */
285        show_plane(p[t], v, plane_wbyte, hbit);
286     }
287  #endif
288
289     for(i=0; i<=4; i++)
290        free(p[i]);
291     bioskey(0);
292     text();
293  }
```

111 BMPShow in VESA Mode(102476824bit)

程序用到的演示bmp文件下载：

http://www.cc98.org/bhh/pic.bmp

在写本程序过程中，参考了以下文献：

http://www.cc98.org/bhh/int.rar 中断大全，查10 4F00、10 4F01、10 4F02、10 4F05

http://www.monstersoft.com/tutorial1/

http://www.drdobbs.com/184409592?_requestid=370904

http://en.wikipedia.org/wiki/VESA_BIOS_Extensions

```
/*****************************************************/
/* BMP Show in VESA Mode(1024*768*24bit)             */
/* copyright (c) Black White                         */
/* email: iceman[url="/user/name/zju.edu.cn"]@zju.edu.cn [/url]
/* This program is for teaching purpose only,        */
/* if you want to reference it, please keep          */
/* the above copyright information.                  */
/*****************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <mem.h>
#include <dos.h>
#include <bios.h>
#include <alloc.h>
typedef unsigned char byte;
typedef unsigned short int word;
typedef unsigned long int dword;
typedef struct
{
/* +000 */ char signature[4];
/* +004 */ word version;
/* +006 */ char far *oem_name_ptr;
/* +00A */ dword capability;
/* +00E */ word far *video_mode_ptr;
/* +012 */ word video_memory_blocks;
/* +014 */ word software_version;
/* +016 */ char far *vendor_name_ptr;
/* +01A */ char far *product_name;
/* +01E */ char far *product_revision;
/* +022 */ word vbe_version;
/* +024 */ word far *accelerated_video_mode_ptr;
/* +028 */ char reversed_for_vbe_implementation[0xD8];
/* +100 */ char oem_scratch_pad[0x100];
} VESA_INFO; /* total 0x200 bytes */

typedef struct
{
/* +00 */ word mode_attr;
/* +02 */ byte win_a_attr;
/* +03 */ byte win_b_attr;
/* +04 */ word win_granularity;
/* +06 */ word win_size;
/* +08 */ word win_a_seg;
/* +0A */ word win_b_seg;
/* +0C */ void (far *win_fun_ptr)(void);
/* +10 */ word bytes_per_line;
/* +12 */ word width_pixels;
/* +14 */ word height_pixels;
/* +16 */ byte char_width;
/* +17 */ byte char_height;
```

```
/* +18 */ byte planes;

/* +19 */ byte bits_per_pixel;
/* +1A */ byte banks;
/* +1B */ byte mem_model_type;
/* +1C */ byte bank_size;
/* +1D */ byte image_pages;
/* +1E */ byte zero_or_one;
/* +1F */ byte red_bits;
/* +20 */ byte red_bit_off;
/* +21 */ byte green_bits;
/* +22 */ byte green_bit_off;
/* +23 */ byte blue_bits;
/* +24 */ byte blue_bit_off;
/* +25 */ byte reserved_bits;
/* +26 */ byte reserved_bit_off;
/* +27 */ byte direct_color_mode_info;
/* +28 */ dword vbuf_phys_addr;
/* +2C */ dword off_screen_addr;
/* +30 */ word off_screen_mem_blocks;
/* +32 */ byte reserved[0xCE];
} MODE_INFO;

void (far * map_win_fun)(void);

void text(void)
{
   _AX=0x0003;
   geninterrupt(0x10);
}

VESA_INFO * get_vesa_info(void)
{
   VESA_INFO *p;
   struct REGPACK r;
   p = calloc(sizeof(VESA_INFO),1);
   if(p == NULL)
      return NULL;
   memset(&r, 0, sizeof(r));
   r.r_ax = 0x4F00;
   r.r_di = (dword)(void far *)p & 0xFFFF;
   r.r_es = (dword)(void far *)p >> 16;
   intr(0x10, &r);
   if((r.r_ax & 0xFF) != 0x4F)
   {
      free(p);
      p = NULL;
   }
   return p;
}

int get_mode_info(word mode, MODE_INFO far *mi)
```

```
103 {
104    struct REGPACK r;
105    r.r_ax = 0x4F01;
106    r.r_cx = mode;
107    r.r_di = (dword)mi & 0xFFFF;
108    r.r_es = (dword)mi >> 16;
109    intr(0x10, &r);
110    return (r.r_ax & 0xFF) == 0x4F;
111 }
112
113 int set_vesa_mode(word mode)
114 {
115    struct REGPACK r;
116    r.r_ax = 0x4F02;
117    r.r_bx = mode;
118    intr(0x10, &r);
119    return (r.r_ax & 0xFF) == 0x4F;
120 }
121
122 int map_window_by_interrupt(word win)
123 { /* this function is slower than map_window() */
124    struct REGPACK r;
125    r.r_ax = 0x4F05;
126    r.r_bx = 0;    /* set window=0; window A=0; */
127    r.r_dx = win;
128    intr(0x10, &r);
129    return (r.r_ax & 0xFF) == 0x4F;
130 }
131
132 void map_window(word win)
133 {
134    word old_bx;
135    old_bx = _BX;
136    _BX = 0;
137    _DX = win;
138    (*map_win_fun)();
139    _BX = old_bx;
140 }
141
142
143 main()
144 {
145    VESA_INFO *vi = NULL;
146    MODE_INFO *mi = NULL;
147    FILE *fp = NULL;
148    word mode, far *pmode;
149    word win, offset;
150    dword width, height, bmp_data_offset, bytes_per_line, win_bytes;
151    int i, j;
152    byte far *vp;   /* pointer to video buffer */
153    byte *p = NULL; /* pointer to a line of bmp data */
```

```
154
155     p = malloc(1024L * 3);  /* memory for holding a line of bmp data */
156     if(p == NULL)
157     {
158        puts("Cannot allocate enough memory for drawing 1 line.");
159        goto done;
160     }
161
162     fp = fopen("pic.bmp", "rb");
163     if(fp == NULL)
164     {
165        puts("Cannot open bmp file!");
166        goto done;
167     }
168     fread(p, 1, 0x36, fp);      /* read BMP head */
169     if(*(word *)p != 0x4D42)    /* check BMP signature */
170     {
171        puts("Not a BMP file!");
172        goto done;
173     }
174     if(*(word *)(p+0x1C) != 24)
175     {
176        puts("Not a 24-bit-color BMP file!");
177        goto done;
178     }
179     width = *(dword *)(p+0x12);
180     height = *(dword *)(p+0x16);
181     bmp_data_offset = *(dword *)(p+0x0A);
182     fseek(fp, bmp_data_offset, SEEK_SET); /* skip BMP head */
183
184     vi = get_vesa_info();
185     if(vi == NULL)
186     {
187        puts("VESA is not supported by BIOS.");
188        goto done;
189     }
190     printf("VESA version is %d.%d\n", vi->version >> 8, vi->version & 0xFF);
191
192     mi = calloc(sizeof(MODE_INFO),1);
193     if(mi == NULL)
194     {
195        puts("Cannot allocate memory for mode info.");
196        goto done;
197     }
198     pmode = vi->video_mode_ptr;
199     while(*pmode != 0xFFFF)     /* goto the end of mode list */
200        pmode++;
201     while(--pmode >= vi->video_mode_ptr) /* search backward */
202     {                                    /* for mode 1024x768x24bit */
203        if(!get_mode_info(*pmode, mi))
204           continue;
```

```
205         if((mi->mode_attr & 0x99) != 0x99)
206            continue;
207         if(mi->width_pixels != 1024 || mi->height_pixels != 768 ||
208            mi->bits_per_pixel != 24)
209            continue;
210         if(mi->bytes_per_line / mi->width_pixels != 24/8)
211            continue;
212         break;
213      }
214      if(pmode < vi->video_mode_ptr) /* search failed */
215      {
216         puts("1024 x 768 x 24bit mode is not supported!");
217         goto done;
218      }
219      mode = *pmode;
220
221      if(!set_vesa_mode(mode))
222      {
223         puts("Cannot switch to 1024 x 768 x 24bit mode!");
224         goto done;
225      }
226      map_win_fun = mi->win_fun_ptr;
227      vp = (byte far *)((dword)mi->win_a_seg << 16);
228      win_bytes = mi->win_size * 0x400L;  /* win_size's unit is Kilobyte */
229      bytes_per_line = (width * 3 + 3) / 4 * 4; /* must be multiple of 4 */
230      for(i=height-1; i>=0; i--)          /* draw from bottom to top */
231      {
232         fread(p, 1, bytes_per_line, fp); /* read a line of bmp data */
233         /* compute which window this line resides in */
234         win = (i*1024L*3) / win_bytes;
235         offset = (i*1024L*3) % win_bytes;
236         if(offset + width*3 <= win_bytes)
237         {    /* if this line is within one window */
238            map_window(win);
239            movedata((dword)(void far *)p >> 16,
240                     (dword)(void far *)p & 0xFFFF,
241                     (dword)(vp+offset) >> 16,
242                     (dword)(vp+offset) & 0xFFFF,
243                     width*3);
244         }
245         else /* if this line overrides two windows */
246         {
247            dword bytes_in_win, bytes_left;
248            bytes_in_win = win_bytes - offset;
249            map_window(win);
250            movedata((dword)(byte far *)p >> 16,
251                     (dword)(byte far *)p & 0xFFFF,
252                     (dword)(vp+offset) >> 16,
253                     (dword)(vp+offset) & 0xFFFF,
254                     bytes_in_win);
255            bytes_left = width*3 - bytes_in_win;
```

```
256             map_window(win+1);

257             movedata((dword)((byte far *)p+bytes_in_win) >> 16,
258                      (dword)((byte far *)p+bytes_in_win) & 0xFFFF,
259                      (dword)vp >> 16,
260                      (dword)vp & 0xFFFF,
261                      bytes_left);
262          }
263       }
264       bioskey(0);
265       text();
266    done:
267       if(vi != NULL) free(vi);
268       if(mi != NULL) free(mi);
269       if(fp != NULL) fclose(fp);
270       if(p  != NULL) free(p);
271    }
```

112 FractalTree in VESA Mode

http://www.cc98.org/bhh/tree.bmp

```
/*****************************************************/
/* FractalTree in Vesa Mode(1024*768*24bit)          */
/* copyright (c) Black White                         */
/* email: iceman[url="/user/name/zju.edu.cn"]@zju.edu.cn [/url]
/* This program is for teaching purpose only,        */
/* if you want to reference it, please keep          */
/* the above copyright information.                  */
/*****************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <mem.h>
#include <dos.h>
#include <bios.h>
#include <alloc.h>
#include <math.h>
#define PI 3.14159
#define R 0.618
typedef unsigned char byte;
typedef unsigned short int word;
typedef unsigned long int dword;
typedef struct
{
/* +000 */ char signature[4];
/* +004 */ word version;
/* +006 */ char far *oem_name_ptr;
/* +00A */ dword capability;
/* +00E */ word far *video_mode_ptr;
/* +012 */ word video_memory_blocks;
/* +014 */ word software_version;
/* +016 */ char far *vendor_name_ptr;
/* +01A */ char far *product_name;
/* +01E */ char far *product_revision;
/* +022 */ word vbe_version;
/* +024 */ word far *accelerated_video_mode_ptr;
/* +028 */ char reversed_for_vbe_implementation[0xD8];
/* +100 */ char oem_scratch_pad[0x100];
} VESA_INFO; /* total 0x200 bytes */

typedef struct
{
/* +00 */ word mode_attr;
/* +02 */ byte win_a_attr;
/* +03 */ byte win_b_attr;
/* +04 */ word win_granularity;
/* +06 */ word win_size;
/* +08 */ word win_a_seg;
/* +0A */ word win_b_seg;
/* +0C */ void (far *win_fun_ptr)(void);
/* +10 */ word bytes_per_line;
/* +12 */ word width_pixels;
```

```
/* +14 */ word height_pixels;

/* +16 */ byte char_width;
/* +17 */ byte char_height;
/* +18 */ byte planes;
/* +19 */ byte bits_per_pixel;
/* +1A */ byte banks;
/* +1B */ byte mem_model_type;
/* +1C */ byte bank_size;
/* +1D */ byte image_pages;
/* +1E */ byte zero_or_one;
/* +1F */ byte red_bits;
/* +20 */ byte red_bit_off;
/* +21 */ byte green_bits;
/* +22 */ byte green_bit_off;
/* +23 */ byte blue_bits;
/* +24 */ byte blue_bit_off;
/* +25 */ byte reserved_bits;
/* +26 */ byte reserved_bit_off;
/* +27 */ byte direct_color_mode_info;
/* +28 */ dword vbuf_phys_addr;
/* +2C */ dword off_screen_addr;
/* +30 */ word off_screen_mem_blocks;
/* +32 */ byte reserved[0xCE];
} MODE_INFO;

void (far * map_win_fun)(void);
dword win_bytes; /* total bytes contained by 1 window */
byte far *vp;    /* pointer to video buffer */
word old_win = -1;

void text(void)
{
   _AX=0x0003;
   geninterrupt(0x10);
}

VESA_INFO * get_vesa_info(void)
{
   VESA_INFO *p;
   struct REGPACK r;
   p = calloc(sizeof(VESA_INFO),1);
   if(p == NULL)
      return NULL;
   memset(&r, 0, sizeof(r));
   r.r_ax = 0x4F00;
   r.r_di = (dword)(void far *)p & 0xFFFF;
   r.r_es = (dword)(void far *)p >> 16;
   intr(0x10, &r);
   if((r.r_ax & 0xFF) != 0x4F)
   {
      free(p);
```

```
103         p = NULL;
104     }
105     return p;
106 }
107
108 int get_mode_info(word mode, MODE_INFO far *mi)
109 {
110     struct REGPACK r;
111     r.r_ax = 0x4F01;
112     r.r_cx = mode;
113     r.r_di = (dword)mi & 0xFFFF;
114     r.r_es = (dword)mi >> 16;
115     intr(0x10, &r);
116     return (r.r_ax & 0xFF) == 0x4F;
117 }
118
119 int set_vesa_mode(word mode)
120 {
121     struct REGPACK r;
122     r.r_ax = 0x4F02;
123     r.r_bx = mode;
124     intr(0x10, &r);
125     return (r.r_ax & 0xFF) == 0x4F;
126 }
127
128 int map_window_by_interrupt(word win)
129 { /* this function is slower than map_window() */
130     struct REGPACK r;
131     r.r_ax = 0x4F05;
132     r.r_bx = 0;     /* set window=0; window A=0; */
133     r.r_dx = win;
134     intr(0x10, &r);
135     return (r.r_ax & 0xFF) == 0x4F;
136 }
137
138 void map_window(word win)
139 {
140     word old_bx;
141     old_bx = _BX;
142     _BX = 0;
143     _DX = win;
144     (*map_win_fun)();
145     _BX = old_bx;
146 }
147
148 void draw_pixel(int x, int y, byte color[3])
149 {
150     word win, offset;
151     int j;
152     win = ((y*1024L+x)*3) / win_bytes;
153     offset = ((y*1024L+x)*3) % win_bytes;
```

```
154     if(offset + 3L <= win_bytes)
155     {    /* if this dot is within one window */
156        if(win != old_win)
157        {
158           map_window(win);
159           old_win = win;
160        }
161        for(j=0; j<3; j++)
162        {
163           vp[offset+j] = color[j];
164        }
165     }
166     else /* if this dot overrides two windows */
167     {
168        dword bytes_in_win, bytes_left;
169        bytes_in_win = win_bytes - offset;
170        if(win != old_win)
171        {
172           map_window(win);
173           old_win = win;
174        }
175        for(j=0; j<bytes_in_win; j++)
176        {
177           vp[offset+j] = color[j];
178        }
179        win++;
180        map_window(win);
181        old_win = win;
182        offset = 0;
183        while(j < 3)
184        {
185           vp[offset++] = color[j++];
186        }
187     }
188  }
189
190  void draw_line(int x1, int y1, int x2, int y2, byte color[3])
191  {  /* Bresenham Algorithm */
192     int x, y, dx, dy;
193     int cnt, err, yoff;
194     dx = x2 - x1;
195     dy = y2 - y1;
196     x = x1;
197     y = y1;
198     yoff = 1;
199     if(dx < 0)
200     {
201        x += dx; dx = -dx;
202        y += dy; dy = -dy;
203     }
204     if(dy < 0)
```

```
205     {
206         yoff = -1;
207         dy   = -dy;
208     }
209     if(dx == 0) /* vertical line */
210     {
211        cnt = dy;
212        do
213        {
214           draw_pixel(x, y, color);
215           y += yoff;
216        } while(--cnt >= 0);
217        return;
218     }
219     if(dy == 0) /* horizontal line */
220     {
221        cnt = dx;
222        do
223        {
224           draw_pixel(x, y, color);
225           x++;
226        } while(--cnt >= 0);
227        return;
228     }
229     if(dx > dy)
230     {
231        err = (cnt = dx) >> 1;
232        do
233        {
234           draw_pixel(x, y, color);
235           if((err -= dy) < 0)
236           {
237              err += dx;
238              y += yoff;
239           }
240           x++;
241        } while(--cnt >= 0);
242     }
243     else /* dy > dx */
244     {
245        err = (cnt = dy) >> 1;
246        do
247        {
248           draw_pixel(x, y, color);
249           if((err -= dx) < 0)
250           {
251              err += dy;
252              x++;
253           }
254           y += yoff;
255        } while(--cnt >= 0);
```

```
256     }
257  }
258
259  void draw_a_tree(int x, int y, int len, int angle, byte color[])
260  {
261     int x2, y2;
262     int i;
263     x2 = x + len * cos(angle * PI / 180);
264     y2 = y - len * sin(angle * PI / 180);
265     draw_line(x, y, x2, y2, color);
266     if(len < 1)
267        return;
268     color[color[3]]++;  /* color[3] is the index */
269                         /* when color[3]=0, it means to change color[0];
270                            when color[3]=1, it means to change color[1];
271                            when color[3]=2, it means to change color[2];
272                          */
273     if(color[color[3]] == 0)
274     {
275        color[color[3]] = 0x40;
276        color[3]++;
277        if(color[3] == 3)
278           color[3] = 0;
279        color[color[3]] = 0x40;
280     }
281          for(i=0; i<16; i++)
282     {
283        x += len * (1-R) * cos(angle * PI / 180);
284        y -= len * (1-R) * sin(angle * PI / 180);
285        draw_a_tree(x , y , len * (1-R), angle + 60, color); /* left leaf */
286        draw_a_tree(x , y , len * (1-R), angle - 60, color); /* right leaf */
287        len *= R;
288     }
289  }
290
291  main()
292  {
293     VESA_INFO *vi = NULL;
294     MODE_INFO *mi = NULL;
295     word mode, far *pmode;
296     byte color[4] = {0x40, 0x00, 0x00, 0x00};
297     vi = get_vesa_info();
298     if(vi == NULL)
299     {
300        puts("VESA is not supported by BIOS.");
301        goto done;
302     }
303     printf("VESA version is %d.%d\n", vi->version >> 8, vi->version & 0xFF);
304
305     mi = calloc(sizeof(MODE_INFO),1);
306     if(mi == NULL)
```

```
307     {
308        puts("Cannot allocate memory for mode info.");
309        goto done;
310     }
311     pmode = vi->video_mode_ptr;
312     while(*pmode != 0xFFFF)    /* goto the end of mode list */
313        pmode++;
314     while(--pmode >= vi->video_mode_ptr) /* search backward */
315     {                                    /* for mode 1024x768x24bit */
316        if(!get_mode_info(*pmode, mi))
317           continue;
318        if((mi->mode_attr & 0x99) != 0x99)
319           continue;
320        if(mi->width_pixels != 1024 || mi->height_pixels != 768 ||
321           mi->bits_per_pixel != 24)
322           continue;
323        if(mi->bytes_per_line / mi->width_pixels != 24/8)
324           continue;
325        break;
326     }
327     if(pmode < vi->video_mode_ptr) /* search failed */
328     {
329        puts("1024 x 768 x 24bit mode is not supported!");
330        goto done;
331     }
332     mode = *pmode;
333
334     if(!set_vesa_mode(mode))
335     {
336        puts("Cannot switch to 1024 x 768 x 24bit mode!");
337        goto done;
338     }
339     map_win_fun = mi->win_fun_ptr;
340     vp = (byte far *)((dword)mi->win_a_seg << 16);
341     win_bytes = mi->win_size * 0x400L;  /* win_size's unit is Kilobyte */
342     draw_a_tree(1024/2, 768-1, 768, 90, color);
343     bioskey(0);
344     text();
345  done:
346     if(vi != NULL) free(vi);
347     if(mi != NULL) free(mi);
348  }
```

113 MandelbrotFractal in Vesa Mode

http://www.cc98.org/bhh/fractal1.bmp

http://www.cc98.org/bhh/fractal2.bmp

```c
/*******************************************************/
/* MandelbrotFractal in Vesa Mode(1024*768*24bit)   */
/* copyright (c) Black White                        */
/* email: iceman[url="/user/name/zju.edu.cn"]@zju.edu.cn [/url]
/* This program is for teaching purpose only,       */
/* if you want to reference it, please keep         */
/* the above copyright information.                 */
/*******************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <mem.h>
#include <dos.h>
#include <bios.h>
#include <alloc.h>
#include <math.h>
typedef unsigned char byte;
typedef unsigned short int word;
typedef unsigned long int dword;
typedef struct
{
   double re, im;
} COMPLEX;

typedef struct
{
/* +000 */ char signature[4];
/* +004 */ word version;
/* +006 */ char far *oem_name_ptr;
/* +00A */ dword capability;
/* +00E */ word far *video_mode_ptr;
/* +012 */ word video_memory_blocks;
/* +014 */ word software_version;
/* +016 */ char far *vendor_name_ptr;
/* +01A */ char far *product_name;
/* +01E */ char far *product_revision;
/* +022 */ word vbe_version;
/* +024 */ word far *accelerated_video_mode_ptr;
/* +028 */ char reversed_for_vbe_implementation[0xD8];
/* +100 */ char oem_scratch_pad[0x100];
} VESA_INFO; /* total 0x200 bytes */

typedef struct
{
/* +00 */ word mode_attr;
/* +02 */ byte win_a_attr;
/* +03 */ byte win_b_attr;
/* +04 */ word win_granularity;
/* +06 */ word win_size;
/* +08 */ word win_a_seg;
```

```
/* +0A */ word win_b_seg;
/* +0C */ void (far *win_fun_ptr)(void);
/* +10 */ word bytes_per_line;
/* +12 */ word width_pixels;
/* +14 */ word height_pixels;
/* +16 */ byte char_width;
/* +17 */ byte char_height;
/* +18 */ byte planes;
/* +19 */ byte bits_per_pixel;
/* +1A */ byte banks;
/* +1B */ byte mem_model_type;
/* +1C */ byte bank_size;
/* +1D */ byte image_pages;
/* +1E */ byte zero_or_one;
/* +1F */ byte red_bits;
/* +20 */ byte red_bit_off;
/* +21 */ byte green_bits;
/* +22 */ byte green_bit_off;
/* +23 */ byte blue_bits;
/* +24 */ byte blue_bit_off;
/* +25 */ byte reserved_bits;
/* +26 */ byte reserved_bit_off;
/* +27 */ byte direct_color_mode_info;
/* +28 */ dword vbuf_phys_addr;
/* +2C */ dword off_screen_addr;
/* +30 */ word off_screen_mem_blocks;
/* +32 */ byte reserved[0xCE];
} MODE_INFO;

void (far * map_win_fun)(void);
dword win_bytes; /* total bytes contained by 1 window */
byte far *vp;    /* pointer to video buffer */
word old_win = -1;
byte Ar=0x12, Ag=0x34, Ab=0x56, Br=0x78, Bg=0x90, Bb=0xAB;

void text(void)
{
   _AX=0x0003;
   geninterrupt(0x10);
}

VESA_INFO * get_vesa_info(void)
{
   VESA_INFO *p;
   struct REGPACK r;
   p = calloc(sizeof(VESA_INFO),1);
   if(p == NULL)
      return NULL;
   memset(&r, 0, sizeof(r));
   r.r_ax = 0x4F00;
   r.r_di = (dword)(void far *)p & 0xFFFF;
```

```
102     r.r_es = (dword)(void far *)p >> 16;
103     intr(0x10, &r);
104     if((r.r_ax & 0xFF) != 0x4F)
105     {
106        free(p);
107        p = NULL;
108     }
109     return p;
110  }
111
112  int get_mode_info(word mode, MODE_INFO far *mi)
113  {
114     struct REGPACK r;
115     r.r_ax = 0x4F01;
116     r.r_cx = mode;
117     r.r_di = (dword)mi & 0xFFFF;
118     r.r_es = (dword)mi >> 16;
119     intr(0x10, &r);
120     return (r.r_ax & 0xFF) == 0x4F;
121  }
122
123  int set_vesa_mode(word mode)
124  {
125     struct REGPACK r;
126     r.r_ax = 0x4F02;
127     r.r_bx = mode;
128     intr(0x10, &r);
129     return (r.r_ax & 0xFF) == 0x4F;
130  }
131
132  int map_window_by_interrupt(word win)
133  { /* this function is slower than map_window() */
134     struct REGPACK r;
135     r.r_ax = 0x4F05;
136     r.r_bx = 0;    /* set window=0; window A=0; */
137     r.r_dx = win;
138     intr(0x10, &r);
139     return (r.r_ax & 0xFF) == 0x4F;
140  }
141
142  void map_window(word win)
143  {
144     word old_bx;
145     old_bx = _BX;
146     _BX = 0;
147     _DX = win;
148     (*map_win_fun)();
149     _BX = old_bx;
150  }
151
152  void draw_pixel(int x, int y, byte color[3])
```

```
153 {
154    word win, offset;
155    int j;
156    win = ((y*1024L+x)*3) / win_bytes;
157    offset = ((y*1024L+x)*3) % win_bytes;
158    if(offset + 3L <= win_bytes)
159    {    /* if this dot is within one window */
160       if(win != old_win)
161       {
162          map_window(win);
163          old_win = win;
164       }
165       for(j=0; j<3; j++)
166       {
167          vp[offset+j] = color[j];
168       }
169    }
170    else /* if this dot overrides two windows */
171    {
172       dword bytes_in_win, bytes_left;
173       bytes_in_win = win_bytes - offset;
174       if(win != old_win)
175       {
176          map_window(win);
177          old_win = win;
178       }
179       for(j=0; j<bytes_in_win; j++)
180       {
181          vp[offset+j] = color[j];
182       }
183       win++;
184       map_window(win);
185       old_win = win;
186       offset = 0;
187       while(j < 3)
188       {
189          vp[offset++] = color[j++];
190       }
191    }
192 }
193
194 void draw_line(int x1, int y1, int x2, int y2, byte color[3])
195 {  /* Bresenham Algorithm */
196    int x, y, dx, dy;
197    int cnt, err, yoff;
198    dx = x2 - x1;
199    dy = y2 - y1;
200    x = x1;
201    y = y1;
202    yoff = 1;
203    if(dx < 0)
```

```
204     {
205        x += dx; dx = -dx;
206        y += dy; dy = -dy;
207     }
208     if(dy < 0)
209     {
210         yoff = -1;
211         dy   = -dy;
212     }
213     if(dx == 0) /* vertical line */
214     {
215        cnt = dy;
216        do
217        {
218           draw_pixel(x, y, color);
219           y += yoff;
220        } while(--cnt >= 0);
221        return;
222     }
223     if(dy == 0) /* horizontal line */
224     {
225        cnt = dx;
226        do
227        {
228           draw_pixel(x, y, color);
229           x++;
230        } while(--cnt >= 0);
231        return;
232     }
233     if(dx > dy)
234     {
235        err = (cnt = dx) >> 1;
236        do
237        {
238           draw_pixel(x, y, color);
239           if((err -= dy) < 0)
240           {
241              err += dx;
242              y += yoff;
243           }
244           x++;
245        } while(--cnt >= 0);
246     }
247     else /* dy > dx */
248     {
249        err = (cnt = dy) >> 1;
250        do
251        {
252           draw_pixel(x, y, color);
253           if((err -= dx) < 0)
254           {
```

```
            err += dy;
            x++;
         }
         y += yoff;
      } while(--cnt >= 0);
   }
}

COMPLEX cadd(COMPLEX a, COMPLEX b)
{
   COMPLEX c;
   c.re = a.re + b.re;
   c.im = a.im + b.im;
   return c;
}

COMPLEX cmul(COMPLEX a, COMPLEX b)
{
   COMPLEX c;
   c.re = a.re*b.re - a.im*b.im;
   c.im = a.im*b.re + a.re*b.im;
   return c;
}

void draw_a_frac(COMPLEX c, byte color[])
{
   COMPLEX z;
   int i, j, k;
   word R,G,B;
   for (i=-512; i<512; i++)
   {
      for(j=-384; j<384; j++)
      {
         z.re = i / 384.0;
         z.im = j / 384.0;
         for(k=0; k<384; k++)
         {
            if(sqrt(z.re*z.re + z.im*z.im) > 2)
               break;
            else
               z = cadd(cmul(z,z), c);
         }
         R = k*Ar+Br;
         G = k*Ag+Bg;
         B = k*Ab+Bb;
         if ((R & 0x1FF) > 0xFF) R ^= 0xFF;
         if ((G & 0x1FF) > 0xFF) G ^= 0xFF;
         if ((B & 0x1FF) > 0xFF) B ^= 0xFF;
         color[0] = B;
         color[1] = G;
         color[2] = R;
```

```
306             draw_pixel(i+512, j+384, color);
307          }
308       }
309    }
310
311    main()
312    {
313       VESA_INFO *vi = NULL;
314       MODE_INFO *mi = NULL;
315       word mode, far *pmode;
316       COMPLEX c[]=
317       {
318          {0.45, -0.1428},
319          {0.285, 0.01},
320          {0.285, 0},
321          {-0.8, 0.156},
322          {-0.835, -0.2321},
323          {-0.70176, -0.3842}
324       };
325       int i;
326       byte color[4] = {0x00, 0x00, 0x00, 0x00};
327       vi = get_vesa_info();
328       if(vi == NULL)
329       {
330          puts("VESA is not supported by BIOS.");
331          goto done;
332       }
333       printf("VESA version is %d.%d\n", vi->version >> 8, vi->version & 0xFF);
334
335       mi = calloc(sizeof(MODE_INFO),1);
336       if(mi == NULL)
337       {
338          puts("Cannot allocate memory for mode info.");
339          goto done;
340       }
341       pmode = vi->video_mode_ptr;
342       while(*pmode != 0xFFFF)    /* goto the end of mode list */
343          pmode++;
344       while(--pmode >= vi->video_mode_ptr) /* search backward */
345       {                                    /* for mode 1024x768x24bit */
346          if(!get_mode_info(*pmode, mi))
347             continue;
348          if((mi->mode_attr & 0x99) != 0x99)
349             continue;
350          if(mi->width_pixels != 1024 || mi->height_pixels != 768 ||
351             mi->bits_per_pixel != 24)
352             continue;
353          if(mi->bytes_per_line / mi->width_pixels != 24/8)
354             continue;
355          break;
356       }
```

```
357     if(pmode < vi->video_mode_ptr) /* search failed */
358     {
359        puts("1024 x 768 x 24bit mode is not supported!");
360        goto done;
361     }
362     mode = *pmode;
363
364     if(!set_vesa_mode(mode))
365     {
366        puts("Cannot switch to 1024 x 768 x 24bit mode!");
367        goto done;
368     }
369     map_win_fun = mi->win_fun_ptr;
370     vp = (byte far *)((dword)mi->win_a_seg << 16);
371     win_bytes = mi->win_size * 0x400L;  /* win_size's unit is Kilobyte */
372     randomize();
373     Ar = random(0xFF)+1;
374     Ag = random(0xFF)+1;
375     Ab = random(0xFF)+1;
376     Br = random(0xFF)+1;
377     Bg = random(0xFF)+1;
378     Bb = random(0xFF)+1;
379     for(i=0; i<sizeof(c)/sizeof(c[0]); i++)
380     {
381        draw_a_frac(c[i], color);
382          }
383          bioskey(0);
384     text();
385  done:
386     if(vi != NULL) free(vi);
387     if(mi != NULL) free(mi);
388  }
```

114 (*p) ->str++中的++加在哪里？ (问题有更新)

问：

对于以下结构定义，

```
1  struct { int len;
2           char *str;
3  } *p;
```

(*p) ->str++中的++加在 ------- 。
A.指针str上 B.指针p上 C. str所指的内容上 D. 表达式语法有错误

->优先级大于++
原式应该等同于（（*p）->str)++；

然后我就不明白了，还是看不出来加在哪个上面了，我猜是str

答：

(*p)->str 表示(*p)指向的结构的str，译成英文：
member str of some structure which is pointed to by (*p)
很明显，str是主语。
但是这题本身有错，因为*p不是指针，*p已经是一个结构变量了，
所以正确写法应该是：
(*p).str++
或
p->str++

如果一定要用(*p)->str这种形式，则p必须是结构指针的指针，例如：

```
struct
{
   int len;
   char *str;
} x={3, "ABC"}, *a, **p;
a = &x;
p =  &a;
(*p)->str++;
puts(x.str); /* 输出BC并回车 */
puts(a->str); /* 输出BC并回车 */
puts((*p)->str); /* 输出BC并回车 */
puts((**p).str); /* 输出BC并回车 */
putchar(*(**p).str); /* 输出B */
```

115 关于strlen和sizeof

问：

lz编了一个程序看两者的差别，其余的都好理解，只是第二个输出结果“2”不太明白。求解释。

```c
#include<stdio.h>
#include<string.h>
int main(void)
{
   char *str,a[10]="ABC";
   str="Hello";
   printf("%d\n",strlen(str));
   printf("%d\n",sizeof(str));
   printf("%d\n",sizeof(a));
   printf("%d\n",sizeof("ABC"));

}
```

输出结果:
5
2
10
4

答:

char 与 char *是不一样的,
sizeof(char)当然是1,
sizeof(char *)相当于sizeof(int)
因为指针变量的内存中占用的字节数与int变量是相当的,
地址其实就是一个整数。

116 关于二阶指针，二维数组，数组指针

问:

指针的指针可以这么定义

```c
char * color[5];
char ** pc=color;
```

然后，lz在题目10024“计算最长的字符串长度"中见到类似定义

```c
char s[10][80],*p[10];
```

和上面一个统一起来就是:

```
char color[5][10];
char * p[5];
```

然后lz就想是不是 * color[5] 等同于 color[5][10] ， ** pc=color 等同于 *p[5] ？
就是说多一个[ ]就能少一个*?

另外，在题目10024中给指针数组付初值很麻烦

```
for(i=0;i<5;i++){
  p[i]=color[1];
}
```

要逐个赋值，
而用二阶指针的话就可以直接赋值 char ** pc=color ， 这难道就是指针的优越所在?

答:

这个其实是不统一的。
(1)表达有问题。你本来可能想说char *color[5]中的color等同于
char color[5][10]中的color；char **pc=color中的pc等同于char *p[5]中的p。
(2)char *color[5]中的color是指针数组名，char color[5][10]中的color是二维数组名；
前者的类型是char **即二阶指针，后者的类型是char (*)[10]即数组指针，根本就是两种类型；
前者只能储存5个字符指针，后者则可以储存5个字符串。
(3)char **pc中的pc的类型为char **，char *p[5]中p的类型为char **，两者类型确实相同；
但是pc是变量名，它的值可以改变，而p是数组名，它的值不可改。

举一个排序的例子如下:

```c
#include <stdio.h>
#include <string.h>
void sort(char **pp, int n)
{
   int i, j, m;
   char *t;
   for(i=0; i<n-1; i++)
   {
      m = i;
      for(j=i+1; j<=n-1; j++)
      {
         if(strcmp(pp[j], pp[m])<0)
            m = j;
      }
      t = pp[i]; pp[i] = pp[m]; pp[m] = t;
   }
}

main()
{
   char color[5][10]={"red", "green", "blue", "purple", "yellow"};
   char *cp[5];
   char **pp;
   int i;
   for(i=0; i<5; i++)
      cp[i] = color[i];
   pp = cp;
   sort(pp, 5);
   for(i=0; i<5; i++)
      puts(*pp++);
}
```

117 输入数字后要用getchar()?

问:

这是老师解答的一道题:

{80022}程序填空，不要改变与输入输出有关的语句。
输入一个正整数 repeat (0 < repeat < 10)，做 repeat 次下列运算:
输入一个正整数 n (1 < n <=10)，然后输入 n 个字符串，输出其中最长的字符串，如果字符串的长度相同，则输出先输入的字符串。
输入:

```
2                (repeat=2)
5                (n=5)
melon peach pear strawberry orange
3                (n=3)
pear berry orange
```

输出:

```
The longest is: strawberry
The longest is: orange
```

```c
#include <stdio.h>
#include <string.h>
int main(void)
{
   char sx[80], longest[80];
   int i, n;
   int repeat, ri;

   scanf("%d", &repeat);
   for(ri = 1; ri <= repeat; ri++)
   {
      scanf("%d", &n);           //这里输入n, 而后面要输入字符串，为什么不用putchar()来消除回
      longest[0] = '\0';
      for(i=0; i < n; i++)
      {
         scanf("%s", sx);
         if(strlen(sx) > strlen(longest))
            strcpy(longest, sx);
      }
      printf("The longest is: %s\n", longest);
   }
}
```

答:

因为用scanf("%s", sx)方式读入字符串时会自动过滤空串
虽然它看到了\n，但是因为没有其它字符，所以它直接把\n忽略了
只有看到非空字符时scanf才会把读到的东西当作字符串储存到sx中

并不是所有的scanf都需要把后面的\n处理掉，只要对下面的输入不会造成影响就没关系

如果这里输入字符串用的不是scanf("%s", sx)而是gets()，那么就必须把\n处理掉，
因为gets()用回车表示结束，不处理掉就会读到一个空串。

ps:读入字符是getchar()不是putchar()

118 int a=10,x=0;x=~a 问结果x等于多少

问：

int a=10,x=0;x=~a 问结果x等于多少

答：

首先是公式： -a=~a+1 （现在得知道了）
然后看一下： a=10，所以-a=-10
然后： x=~a=-a-1=-11

要不然就老老实实按定义：
a=10 二进制即 a=00000000 00001010
x=~a 即对a进行按位求反 x=11111111 11110101 十进制即x=-11

119 对于int *p；p可以赋值为二维数组名吗?

问：

有以下说明：
int a[4][10] , * p , * q ;
为什么 p=a； 是错误的赋值

答：

因为两者类型不同。
a是数组指针，它指向10个int；
p是int指针，它指向1个int。

120 VC++ 6.0 精简版 & MSDN精简版

(1) VC++ 6.0 精简版:
http://www.cc98.org/bhh/VcSetup.exe

(2) MSDN精简版(VC的在线帮助文档)
http://www.cc98.org/bhh/msdn.exe

121 C语言：不用！求!x

问：

PS：不能递归，不能循环，不能分支，不能强制类型转换……只能用列出的运算符
这里的INT为32位的。

```
/*
 * bang - Compute !x without using !
 *   Examples: bang(3) = 0, bang(0) = 1
 *   Legal ops: ~ & ^ | + << >>
 *   Max ops: 12
 *   Rating: 4
 */
int bang(int x) {


}
```

答：

解法1：
假定int是32位的话，可以这样写：
return ((((~x+1) | x) >> 31) & 1) ^ 1;
如果int位数不确定，暂时想不出12个运算内的解法。

解法2：
本来如果没有限定运算符个数的话，还有一个罗嗦的想法：

```
y = (x>>16 & 0xFFFF) | (x & 0xFFFF);
y = (y>>8 & 0xFF) | (y & 0xFF);
y = (y>>4 & 0xF) | (y & 0xF);
y = (y>>2 & 3) | (y & 3);
y = (y>>1 & 1) | (y & 1);
return y^1;
```

上述等号运算符不在题目限定的范围内，但可以把这几步合成一步来写从而省去等号。
不过，这样写的话，运算符个数明显超过12个了。

解法3：

```c
#include <stdio.h>

int main()
{
int i, x;
for (i = 1 << 31; i < (1 << 31) - 1; ++i)
{
x = ( ( (~i + 1) | i) >> 31) + 1;
if (x != 0)
printf("%d: %d\n", i, x);
}
return 0;
}
```

解法4：

return (((x ^ x + 0xffffffff) & ~x) >> 31) & 1;

看看这个可以吗?

(x ^ x + 0xffffffff)的计算结果：

若x!=0且x!=0x80000000，最高位为0

若x=0x80000000，最高位为1

若x=0，最高位为1

讨论(x ^ x + 0xffffffff) & ~x 的计算结果

若x!=0且x!=0x80000000，最高位为0

若x=0x80000000，最高位为0

若x=0，最高位为1

之后步骤仍是取出最高位

122 分解质因数无需验证因数是否质数

分解质因数一题，想到一种无需判断因数是否为质数的方法：

```c
#include "stdio.h"
#include "math.h"
int prime(int n);
int main(void)
{
    int i, k, m;
    int repeat, ri;
    scanf("%d", &repeat);
    for(ri = 1; ri <= repeat; ri++){
        scanf("%d", &m);
        printf("%d = ", m);
        i=2;
        while(i<m)
       {
           if(m%i==0)
          {
               printf("%d*", i);
               m=m/i;
           }
           else
               i=i+1;
       }
       printf("%d\n",m);
    }
}
```

显然，如果遇到i是非质数，则i必可以写成比i更小的若干质数之积，这些质数在i之前已经被除过了，所以此时不可能出现m%i==0，这样也就无需判断i是否质数。

ps:感谢cc98的**zhengfana**同学分享。

123 C语言模拟考试答案（供参考）

一、判断题。共10题，每题1分，共10分。正确的请选择A，错误的请选择B。

1、 在C语言中，函数不可以嵌套定义，但函数可以嵌套调用。

A、正确 B、错误

解答：A

2、 在程序中 else 子句可以单独使用。

A、正确 B、错误

解答：B

3、 C 语言源程序文件通过了编译、连接之后，生成一个后缀为 .EXE 的文件。()

A、正确 B、错误

解答：A

4、 _ya是不合法的C语言标识符。
A、正确 B、错误

解答：B

5、 在数组初始化时，如果初值个数多于数组长度，将出现错误。
A、正确 B、错误

解答：A

6、 C语言规定，函数返回值的类型是由return语句中的表达式类型所决定。
A、正确 B、错误

解答：B

7、 若有定义 int *p, 则表达式 p++ 的值就是 p 所指向的变量的地址值加1。
A、正确 B、错误

解答：B

8、 '\n'不是一个字符常量。
A、正确 B、错误

解答：B

9、 表达式 (z=0, (x=2)||(z=1),z) 的值是1。
A、正确 B、错误

解答：B

10、 若打开文件的方式为“w”而该文件已经存在，则文件中原有数据被清空。
A、正确 B、错误

解答：A

ps:感谢cc98的**張良**同学分享。

124 char a[]="ABC\XYZ\007\xFF";求sizeof

问：

char a[]="ABC\XYZ\007\xFF";
int n;
n = sizeof(a);
为什么答案是B，而我运行后是10.我两个都不是很明白，这串字符是怎么数的？？？

答：

转义字符 含义 ASCII码（16/10进制）
\o 空字符(NULL) 00H/0
\n 换行符(LF) 0AH/10
\r 回车符(CR) 0DH/13
\t 水平制表符(HT) 09H/9
\v 垂直制表(VT) 0B/11
\a 响铃(BEL) 07/7
\b 退格符(BS) 08H/8
\f 换页符(FF) 0CH/12
\’ 单引号 27H/39
\” 双引号 22H/34
\ 反斜杠 5CH/92
? 问号字符 3F/63
\ddd 任意字符 三位八进制
\xhh 任意字符 二位十六进制

125 C语言函数中不可以返回尸体的地址

程序填空，不要改变与输入输出有关的语句。
编写一个程序，输入一个字符串后再输入2个字符，输出此字符串中从与第一个字符匹配的位置开始到与第二个字符匹配的位置之间的所有字符。用返回字符指针的函数实现。
例：(括号内为说明)
输入：

```
program
r
g
```

输出：

rog

```c
#include <stdio.h>
char newstr[80];
void main()
{
        char str[80],c_begin,c_end;
        char *fun(char *p,char c_begin,char c_end);

        scanf("%s",str);
        getchar();
        c_begin=getchar();
        getchar();
        c_end=getchar();
        puts(fun(str,c_begin,c_end));
}
char *fun(char *p,char c_begin,char c_end)
{
        int i=0,k=0;
        char s[80];
        while(*(p+i) != c_begin)
                i++;
        do
        {
                s[k] = *(p+i);
                k++;
                i++;
        }while(*(p+i)!=c_end);
        s[k] = *(p+i);
        s[k+1]=0;
        return s;
}
```

上述代码中，由于s是局部动态数组，因此在函数fun()返回时，
它一定会死亡。当return s;时相当于返回了一个尸体的地址，肯定是错误的。
解决方法1：
把char s[80]改成static char s[80]。但此方法有一个弊端：
万一p指向的字符串长度超过80就极有可能造成数组s的溢出。
解决方法2：

```
char *fun(char *p,char c_begin,char c_end)
{
   int i=0,k=0;
   while(p[i] != c_begin)
      i++;
   k = i+1;
   while(p[k] != c_end)
      k++;
   p[k+1]=0;
   return p+i;
}
```

126 为什么%e格式不可以输出一个字符串

问：

There will be an error if executing statement printf(" ", "A");
A．%u B．%s C．%d D． %e

答案选的是d，但是我编了以后发现可以运行的，没有错

答：

能够输出结果并不表示就是正确的。
%e一定针对float或double类型，
而"A"明显是一个地址，相当于一个int，
不能用浮点数格式输出的。

D肯定错，因为%e格式要求实际代入的值必须是64位double类型的小数，
而"A"作为地址代入在TC中是16位，VC中是32位。因此这两者是明显不匹配的，
也就谈不上是加密了。要以小数的形式输出整数值的话(就是你说的加密)，至少位
数要相同。

B肯定正确，这个不用置疑。

C一般不用，用%d输出地址是一种粗鄙的用法，因为当地址比较大的时候，%d会输出一个负值。

A可以用，但是不专业。因为输出地址的值通常应该是16进制，所以
专业用法是%p格式，不太专业的用法是%X，但很少用%u这种十进制格式。

127 求助：文本文件和二进制文件的区别弄不清啊

问：

C文件那一章中的一些函数只适用于二进制文件，我老是弄不清，二进制文件和文本文件到底有什么区别，考试会考到吗?

答:

文本文件的内容是键盘上敲得进去的字符，还包括回车、Tab、空格等特殊字符；
二进制文件的内容可以是ASCII码为0~255范围内的任意字符。
用文本格式如"r"打开文件读文件时，会对文件内容做一些转换，例如把0x0D、0x0A合并成0x0A；
用二进制格式如"rb"打开文件读文件时，不会对文件内容做任何转换。
用二进制格式既可以打开文本文件也可以打开二进制文件；
文本格式一般只用来打开文本文件，否则极有可能造成读出来的内容与文件原始内容不符。

128 int8h时钟中断程序例子

下载链接:
http://blackwhite.8866.org/bhh/myint8h0.c
http://blackwhite.8866.org/bhh/myint8h1.c

myint8h0.c

```c
#include <dos.h>
#define RED 0x0C
#define GREEN 0x0A
#define BLUE 0x09
#define YELLOW 0x0E
void interrupt my_int8h(void)
{
   char far *p = (char far *) 0xB8000000 + 1;
   static int count = 0;
   count ++;
   if(count >= 18)
   {
      if(*p == RED)
         *p = GREEN;
      else if(*p == GREEN)
         *p = BLUE;
      else if(*p == BLUE)
         *p = YELLOW;
      else
         *p = RED;
      count = 0;
   }
   outportb(0x20, 0x20);
}

void show(int x, int y, int num)
{
   char far *p = (char far *)0xB8000000 + (y*80+x)*2;
   char buf[10];
   int i = 0;
   sprintf(buf, "%04d", num);
   while(buf[i] != '\0')
   {
      *p++ = buf[i];
      *p++ = 0x07; /* black background, white foreground */
      i++;
   }
}

main()
{
   char far *p = (char far *) 0xB8000000;
   void interrupt (*old_8h)(void);
   int i;
   *p = 'K';
   *(p+1) = RED;
   old_8h = getvect(8);
   setvect(8, my_int8h);
   i = 0;
   do /* It's boring to do nothing here, so we show numbers */
   {  /* from 0000 to 9999 repeatedly */
```

```
52          show(20, 10, i); /* show i at (20, 10) */
53          i++;
54          if(i == 10000)
55             i = 0;
56       } while (bioskey(1)==0);
57       setvect(8, old_8h);
58    }
59
```

myint8h1.c

```c
#include <dos.h>
#define RED 0x0C
#define GREEN 0x0A
#define BLUE 0x09
#define YELLOW 0x0E
typedef void interrupt (*InterruptFunctionPointer)(void);

InterruptFunctionPointer old_8h;

void interrupt my_int8h(void)
{
   char far *p = (char far *) 0xB8000000 + 1;
   static int count = 0;
   count ++;
   if(count >= 18)
   {
      if(*p == RED)
         *p = GREEN;
      else if(*p == GREEN)
         *p = BLUE;
      else if(*p == BLUE)
         *p = YELLOW;
      else
         *p = RED;
      count = 0;
   }
   (*old_8h)();
   /* Here we remove outportb(0x20, 0x20),
      because (*old_8h)() has done it for us. */
}

void show(int x, int y, int num)
{
   char far *p = (char far *)0xB8000000 + (y*80+x)*2;
   char buf[10];
   int i = 0;
   sprintf(buf, "%04d", num);
   while(buf[i] != '\0')
   {
      *p++ = buf[i];
      *p++ = 0x07; /* black background, white foreground */
      i++;
   }
}

InterruptFunctionPointer getvect_myself(int n)
{
   return (InterruptFunctionPointer) ( *(unsigned long int far *)(n*4) );
   /* For n=8, its interrupt vector is located at 0x0000:0x0020 */
}

```

```
52 void setvect_myself(int n, InterruptFunctionPointer p)
53 {
54    disable(); /* disable interrupt so that there will be no occurrence of any
55                  interrupt when we are changing the interrupt vector */
56    *(unsigned long int far *)(n*4) = (unsigned long int)p;
57    enable();  /* enable interrupt */
58 }
59
60 main()
61 {
62    char far *p = (char far *) 0xB8000000;
63    int i;
64    *p = 'K';
65    *(p+1) = RED;
66    old_8h = getvect_myself(8);  /* getvect_myself() is similar to getvect() */
67    setvect_myself(8, my_int8h); /* setvect_myself() is similar to setvect() */
68    i = 0;
69    do /* It's boring to do nothing here, so we show numbers */
70    {  /* from 0000 to 9999 repeatedly */
71       show(20, 10, i); /* show i at (20, 10) */
72       i++;
73       if(i == 10000)
74          i = 0;
75    } while (bioskey(1)==0);
76    setvect(8, old_8h);
77 }
78
```

129 迷宫虫系列

1. 迷宫中的虫子(深度优先算法搜索目标) bugmaze.c
   http://blackwhite.8866.org/bhh/bugmaze.c
2. 迷宫中的虫子(A*算法搜索目标) bugastar.c
   http://blackwhite.8866.org/bhh/bugastar.c
3. 迷宫中的虫子(A*算法搜索目标，目标可移动，逮到目标后虫子会快乐地叫) bugeat.c
   http://blackwhite.8866.org/bhh/bugeat.c

130 关于setjmp的问题

问：

```c
#include "trans.h"
#include <stdio.h>
#include <stdlib.h>
#include <setjmp.h>

/* 用于存储setjmp状态信息的变量 */
jmp_buf restart;

int main()
{
int value;
Trans *transaction;

/* 确立希望longjmp调用之后执行流恢复执行的地点 */
value = setjmp( restart ); //对该句有疑问

/* 从longjmp返回后判断下一步执行什么 */
switch( setjmp(  restart ) ){
default:

/* 致命错误 */
fputs( "Fatal error.\n", stderr );
break;

case 1:

/* 小错误 */
fputs( "Invalid transaction.\n", stderr );
/* FALL THROUGH  继续执行 */

case 0:

/* 从setjmp返回的地点:  执行正常的处理 */
while( (transaction = get_trans()) != NULL)
process_trans( transaction );

}

write_data_to_file();

return value == 0 ? EXIT_SUCCESS : EXIT_FAILURE;

}
```

《c和指针》看标准函数库一章时看到这里，总觉得有问题，

value = setjmp(restart);

这一句按我觉得似乎是不应该有的，如果有这一句，longjmp将返回给value一个新的值，而在switch中再一次调用setjmp使用的并不是由longjmp返回的值。我觉得switch中使用的应该是value或者去掉上一条语句。

另外setjmp是不是一般由汇编实现，返回时并不清理本函数的堆栈?
小白能不能讲解下setjmp的具体实现，看书上讲的还是不太清晰。。。

答:

以前有同学问过setjmp及longjmp的问题，当时我自己编了一个演示程序如下:

```
#include <stdio.h>
#include <setjmp.h>
jmp_buf j;
int count=0;
void f(void)
{
   setjmp(j); /* 保存当前寄存器到j */
   count++;   /* longjmp()将回到此处 */
   if(count==1)
      return;
   while(count <= 3)
   {
      puts("longjmp triggered");
      count++;
   }
   exit(0);   /* 由于longjmp()跳到count++之前堆栈中留有残余数据没有弹出 */
              /* 所以，这里选择exit(0)强制终止程序比较安全，否则有可能在 */
              /* return时跳到一个不正确的地址从而导致非法操作 */
}
void g(void)
{
   longjmp(j, 1);
}
main()
{
   f();
   g();
}
```

一开始我也没有搞清楚这两个函数的含义，光是看它们的帮助仍不知其所以然，
后来我用IDA Pro把库函数给反编译出来才搞明白它们的意思。你给出的程序中，
我也感觉第2次调用setjmp有问题。另从以下反汇编代码中可以看出，setjmp()
永远返回0，longjmp()永远返回1，它们之间的返回值没有关系的。longjmp()
的任务只是返回到调用setjmp()的语句的下一个语句。
注：划线的观点有错误，具体在第4楼解释。

```
;
; +-------------------------------------------------------------------------+
; |     This file is generated by The Interactive Disassembler (IDA)        |
; |     Copyright (c) 2007 by DataRescue sa/nv, <ida@datarescue.com>        |
; | Licensed to: Mach EDV Dienstleistungen, Jan Mach, 1 user, adv, 11/2007  |
; +-------------------------------------------------------------------------+
;
; Input MD5   : DD34DB18397656511FB69D871A3DE4B4

; File Name   : SETJMP
; Format      : Object Module Format (OMF/Microsoft)
; Module name      : SETJMP


                .386
                .model flat

; ===========================================================================

; Segment type: Group
DGROUP          group _BSS

; ===========================================================================

; Segment type: Pure code
_TEXT           segment byte public 'CODE' use16
                assume cs:_TEXT
                assume es:nothing, ss:nothing, ds:DGROUP, fs:nothing, gs:nothing

; =============== S U B R O U T I N E =======================================

; Attributes: bp-based frame

                public _setjmp
_setjmp         proc near

arg_0           = word ptr  4

                push    bp
                mov     bp, sp
                push    si
                push    di
                push    ds
                pop     es
                assume es:DGROUP
                mov     dx, es
                mov     cx, di
                mov     di, [bp+arg_0]
                cld
                lea     ax, [bp+arg_0]
                stosw
```

```
                mov     ax, ss
                stosw
                pushf
                pop     ax
                stosw
                mov     ax, cs
                stosw
                mov     ax, [bp+2]
                stosw
                mov     ax, [bp+0]
                stosw
                mov     ax, cx
                stosw
                mov     ax, dx
                stosw
                mov     ax, si
                stosw
                mov     ax, ds
                stosw
                mov     es, dx
                xor     ax, ax
                jmp     short [noubb]$[/noubb]+2
                pop     di
                pop     si
                pop     bp
                retn
_setjmp         endp


; =============== S U B R O U T I N E =======================================

; Attributes: bp-based frame

                public _longjmp
_longjmp        proc far

arg_0           = word ptr  6

                push    bp
                mov     bp, sp
                push    si
                push    di
                mov     bx, [bp+arg_0]
                cmp     bx, 1
                adc     bx, 0
                mov     si, [bp+4]
                cld
                lodsw
                mov     ss, word ptr [si]
                mov     sp, ax
                lodsw
```

```
                lodsw
                push    ax
                lodsw
                push    ax
                lodsw
                push    ax
                lodsw
                mov     bp, ax
                lodsw
                mov     di, ax
                lodsw
                mov     es, ax
                assume es:nothing
                lodsw
                mov     ds, word ptr [si]
                mov     si, ax
                mov     ax, bx
                iret
_longjmp        endp

; ---------------------------------------------------------------------------
                pop     di
                pop     si
                pop     bp
                retn
; ---------------------------------------------------------------------------
                align 4
_TEXT           ends
```

131 给大家的C大程提供点灵感哈~<C/C++ RPG游戏实现技术细节>

- 1. http://file.cc98.org/uploadfile/2012/3/16/1540910544.rar
- 2. http://file.cc98.org/uploadfile/2012/3/16/15401398891.rar
- 3. http://file.cc98.org/uploadfile/2012/3/16/15401699333.rar
- 4. http://file.cc98.org/uploadfile/2012/3/16/1540589298.rar
  全都是有关DOS下游戏编程的,可能会穿插有一些汇编内容(没办法的事)如观察编译器汇编输出等,与<C游戏编程从入门到精通>的总体风格类似,但内容绝对原创.

全名: C/C++语言RPG游戏实现技术细节

这是和几个同学去年做大程时总结出来的,希望对大家有用啊~只是一些个人总结和体会,难免不会有什么错误,各位牛人还请高抬贵手啊~

不好意思中间差了一个,是由于后来发现里面的程序有重大失误,所以拿回去重新改了. 另外第四个由于现在还在写,所以没有完成,后面的内容会尽早补全的.
中间少掉的那个是介绍XMS扩展内存极其例程编写的,由于里面数据结构实在是太恶心了,所以我在最初写的时候犯了个更恶心的错误(返回临时变量的指针).

之后的内容还会有:(正在赶工中...)

VGA图形驱动(包括640*480 SVGA,别的地方已经很难找到这方面的资料了)
时间控制(硬时钟和垂直回扫,缓冲刷屏等等)
DOS下汉字的点阵显示及标准汉字库使用
有限状态机及战斗过程设计
BMP文件载入及显示(包括拉屏的实现)
.....

由于XMS例程是下面众多技术的基础,所以这个出了问题以后的内容就麻烦了
里面的很多内容都与软件逆向这门课有关,我最初也是从这里得到的灵感.例如旧不举例了,大家自己去发现吧.

另外里面的代码绝大部分都是经过我亲自实验的,在指定的编译环境下可以正常运行, 而且还有一些是个人原创的娱乐程序,其中比较得意的一个是DOS下实现了一个基本功能和WINDOWS API 的LoadLibrary() GetModuleHandle() FreeLibrary()类似的模块.虽说没什么实用性,但是作为娱乐程序还是不错的.

132 RPG游戏实现技术细节 第四章 模块动态载入

http://file.cc98.org/uploadfile/2012/3/22/14264481184.rar

第四章 模块动态载入

4.1 动态链接与DLL
4.1.1 对象输出
4.1.2 两种连接方式
4.1.3 运行时载入
4.1.4 绝对地址与相对地址
4.1.5 动态定位
4.2 COM文件格式
4.2.1 COM文件的储存结构与特点
4.2.2 汇编语言下COM文件生成
4.2.3 C语言下COM文件生成
4.3 连接器
4.3.1 函数名修饰
4.3.2 启动代码
4.3.3 连接策略
4.4 16位EXE文件格式
4.4.1 文件头
4.4.2 重定位表
4.4.3 DOS加载器

4.5 C语言实现EXE文件的动态载入
4.5.1 基本数据结构
4.5.2 初始化模块
4.5.3 载入实现模块
4.5.4 重定位
4.5.5 被载入模块的编写
4.5.6 函数输入定位
4.6 更多改进

133 关于单精度浮点型做参数的问题

问：

今天要对手工画的浮点加法器做功能验证，需要写一个程序来查看单精度浮点数的二进制内部结构，于是写了个C程序用printf函数查看，代码如下：

```c
#include <stdio.h>
#include <stdlib.h>

void main()
{
 float a = 12.34;

 printf("%lX\n",a);

 system("pause");
 return;
}
```

但是在运行时发现结果不对（输出80000000严重不符合IEEE 754标准），而且据我查看汇编输出的结果来看，造成这一问题的根本原因是编译器在通过堆栈传递单精度浮点参数时，使用了两条协处理器指令FLD和FSTP，问题就出在编译器为FLD指定的操作数是DWORD PTR，但为FSTP指定的长度为QWORD PTR（堆栈指针也按照QWORD进行调整），导致32位浮点数被扩展到了64位存入堆栈，从而printf的%lX格式限定符无法正确打印。在此就有一个问题，这样的长度传唤在C语言的标准和The C Programming Language中是无法查到的，而根据我用Borland，WATCOM和VC的测试结果来看大多又都有这样的特性，因此表示不理解。难道单精度浮点数做参数的时候也有类似于“整型提升”的问题？还是请白老师解答。

（注1：不是printf函数的问题）
（注2：如果换成双精度浮点数结果反倒是正确的，经验证此时参数的传递没有用协处理器）

Borland编译器的输出：

```
_TEXT   segment byte public 'CODE'
;       ?debug  L 4
_main   proc    near
        push    bp
        mov     bp,sp
        sub     sp,4
;       ?debug  L 6
        mov     dx,16709
        mov     ax,28836                                        mov     word ptr [bp-2],dx
        mov     word ptr [bp-4],ax      □
;       ?debug  L 8
        FLD     dword ptr [bp-4]
        sub     sp,8
        FSTP    qword ptr [bp-12]
        FWAIT
        push    ds
        mov     ax,offset DGROUP:s@
        push    ax
        call    near ptr _printf
        add     sp,12
;       ?debug  L 10
        push    ds
        mov     ax,offset DGROUP:s@+4
        push    ax
        call    near ptr _system
        add     sp,4
        jmp     short @1
@1:
;       ?debug  L 12
        mov     sp,bp
        pop     bp
        ret
_main   endp
_TEXT   ends
```

WATCOM编译器的输出：

```
1  void main()
2  0000                         main_:
3  0000    53                       push        ebx
4  0001    51                       push        ecx
5  0002    52                       push        edx
6  0003    56                       push        esi
7  0004    57                       push        edi
8  0005    55                       push        ebp
9  0006    89 E5                    mov         ebp,esp
10 0008    81 EC 04 00 00 00        sub         esp,0x00000004
11
12 {
13  float a = 12.34;
14 000E                         L$1:
15 000E    C7 45 FC A4 70 45 41     mov         dword ptr -0x4[ebp],0x414570a4
16
17
18  printf("%lX\n",a);
19 0015    D9 45 FC                 fld         dword ptr -0x4[ebp]
20 0018    83 EC 08                 sub         esp,0x00000008
21 001B    DD 1C 24                 fstp        qword ptr [esp]
22 001E    B8 00 00 00 00           mov         eax,offset L$4
23 0023    50                       push        eax
24 0024    E8 00 00 00 00           call        printf_
25 0029    83 C4 0C                 add         esp,0x0000000c
26
27  system("pause");
28 002C    B8 05 00 00 00           mov         eax,offset L$5
29 0031    E8 00 00 00 00           call        system_
30
31
32  return;
33 }
```

答：

> float to double
> short/char to int
> 所以printf处理到的参数都是int或double的
> ——印象中不定参数函数的参数都会有这种提升
> ps:来自cc98的**版纳野**同学

对的。对于不定参数的函数，由于只有实参类型没有形参类型，所以参数代入时按实参类型为准，但是char要提升为int，float要提升为double，也就是说此时要按C89老标准来，C99新标准不起作用。
因此，float类型的表达式既可以用%f也可以用%lf输出，

因为printf知道输出格式后面的小数不可能为float，只好把%f当作%lf来处理。同理，double类型的表达式既可以用%lf也可以用%f来输出。

如果实参、形参均为char时，是不会提升的。
C代码如下：

```
1  char f(char a)
2  {
3     return a+1;
4  }
5  main()
6  {
7     char x='A', y;
8     y = f(x);
9  }
```

```
_f: char f(char a)
  cs:01FA►55            push   bp
  cs:01FB 8BEC          mov    bp,sp
#CHAR#3:  return a+1;
  cs:01FD 8A4604        mov    al,[bp+04]; 接收的时候也只取低8位
  cs:0200 FEC0          inc    al
  cs:0202 EB00          jmp    #CHAR#4 (0204)
#CHAR#4: }
  cs:0204 5D            pop    bp
  cs:0205 C3            ret
_main: main()
  cs:0206►55            push   bp
  cs:0207 8BEC          mov    bp,sp
  cs:0209 83EC02        sub    sp,0002
#CHAR#7:  char x='A', y;
  cs:020C C646FE41      mov    byte ptr [bp-02],41
#CHAR#8:  y = f(x);
  cs:0210 FF76FE        push   word ptr [bp-02]; 注意这里的高8位并没有扩充为0
  cs:0213 E8E4FF        call   _f
  cs:0216 59            pop    cx
  cs:0217 8846FF        mov    [bp-01],al
#CHAR#9: }
  cs:021A 8BE5          mov    sp,bp
  cs:021C 5D            pop    bp
  cs:021D C3            ret
```

134 转义字符\ddd,\后面一定要3位吗?

Q:
转义字符\ddd,\后面一定要3位吗?

A:
不一定啊。比如'\0'就只有一位。
如果在双引号里面的转义字符，有一个贪心原则，
就是取尽可能多的位数，"\1234XY"，就会自动
取'\123'作为一个字符，而不是'\1'，也不是'\12'。

135 08试卷，查找数组中是否有相同元素，双重循环跳出条件

问：

输入n（2≤n≤5，程序不需要对此范围进行判断），再输入n个整数保存到数组a中，通过循环查找n个数中是否有重复的数，如果有则输出Yes，否则输出No。要求在循环过程中，任何两个数的比较次数不得超过1次（比如对a[0]与a[1]比较后接下去又对a[1]与a[0]比较是不符合要求的），并且要求一旦找到有数重复则立即结束循环。

```
#include <stdio.h>
 void main()
 {
    int a[5], i, j, n;
    scanf("%d", &n);
    for(i=0; i<n; i++)
       scanf("%d", &a[i]);
    for(i=0; i<=n-2; i++)
    {
       for(    (1)    ; j<=n-1; j++)
       {
          if(a[i] == a[j])
               (2)  ;
       }
       if(     (3)     )
          break;
    }
    if(    (4)     )
       puts("No");
    else
       puts("Yes");
 }
```

求问第（3）空填a[i] == a[j]为什么不给分

答：

因为j循环里面如果没有发生break，则j最终会等于n，于是a[j]会导致数组越界。

136 部分位求反

问：

invert(unsigned x,usigned p,unsigned n)
{
return ;
}
返回对x从p位（最右位为0位）起的n位求反，其余位不变。
求教！！如何用一个语句解决?

答：

第p位求反:

```
return x ^ (1<<p);
```

右边n位求反：

```
return x ^ ((1<<n)-1);
```

从p位开始连续n位求反：

```
return x ^ (((1<<n)-1) << p);
```

其中(1 << n)-1就是2的n次方-1。

137 若arr是char的二维数组，则printf("%s\n",arr);有错?

问：

```
int main()
{
    char arr[2][4];
    strcpy(arr[0],"you");
    strcpy(arr[1],"you");
    arr[0][3]='&';


    printf("%s\n",arr);
    printf("%s\n",arr[0]);
    printf("%c\n",*arr[0]);
    printf("%c\n",**arr);system("pause");
}
```

求问：为什么arr和arr【0】不是一个层次的（由三四行可知），但第一行第二行输出结果一样？arr是否是个指向数组的指针，而并非指向“you”首元素的地址？

答：

你的理解是正确的。

严格地讲，
printf("%s\n",arr); 是错的。
因为arr的类型为char (*)[4]，与%s格式要求的类型char *是不匹配的。
要是把printf("%s\n",arr); 换成puts(arr)就一定会有语法错误，因为puts要求参数为char *类型。
哪为什么printf不报错呢？这是因为printf()这个函数比较独特，它只有第1个参数的类型是明确的(具体类型为char *)，从第2个参数起不仅参数个数不确实，类型也是不确定的。因此在编译时，编译器无法检测第2个参数起各个参数的类型是否正确。
哪为什么printf("%s\n",arr); 能得到printf("%s\n",arr[0]); 一样的结果呢？因为arr的值即arr[0]行的地址碰巧与arr[0]即arr[0][0]的地址是相等的。

138 夹在函数之间的全局变量可以被该变量前面的函数访问吗?

问：

```
#include<stdio.h>
void fun(int x)
{
static int y;
...
return ;
}
int z;
void main()
{
int a,b;
fun(a);
...
}
```

在fun函数中可以使用的所有变量里有z么?

答：

没有z。
必须在#include<stdio.h>后面加
extern int z;才可以。

extern int z;的功能有点类似于函数原型，它并没有对z重新定义，
而是把z的类型提前告诉编译器。

139 用typedef定义函数指针

问：

typedef int (*PT)()
这样的东西存在吗?
用typedef定义完的数据类型可以用来定义函数吗?

答：

typedef int (*PT)()是函数指针类型，上课的时候没有讲。
这个东西不能用来定义一个函数，而是用来定义函数指针，例如：

```
typedef int (*PT)(int, int);
int f(int a, int b)
{
   return a+b;
}
main()
{
   PT p;
   int y;
   p = f; /*此处的f表示函数f()地址，不能写成&f，因为函数名已经表示函数的地址了 */
   y = (*p)(10, 20); /* 相当于y = f(10, 20); 其中*p就是函数f。
}
```

140 关于DosBoxTc的使用方法

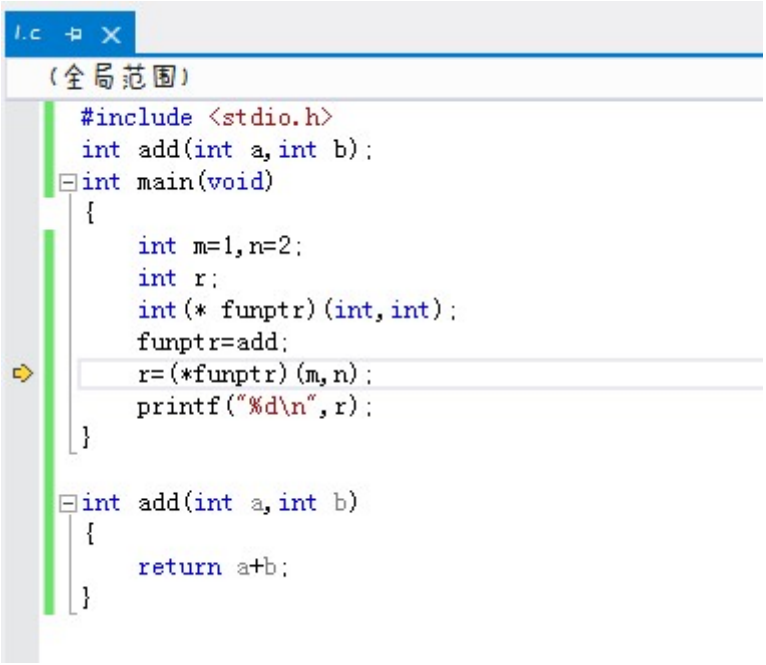
我今天特意找了一台win7的电脑试过，dosboxtc可以用的。
那些宣称不能用的同学是没有领会主页上的说明。
具体使用方法如下：
(1) 直接双击DosBox的快捷方式，它弹出的错误不用理它，按Esc忽略即可。
然后选菜单File->Load的时候，应该输入c:，就可以找到你存放在
dosboxtc\tc文件夹里面的C程序；
(2) 如果你已经用别的工具如EditPlus编好了.c文件，你把此文件拖给dosboxtc
的快捷方式，即可自动打开。
run完了之后，按alt+F5或Run->User screen查看结果。

141 老师，问一个有关于函数指针的问题~

问：

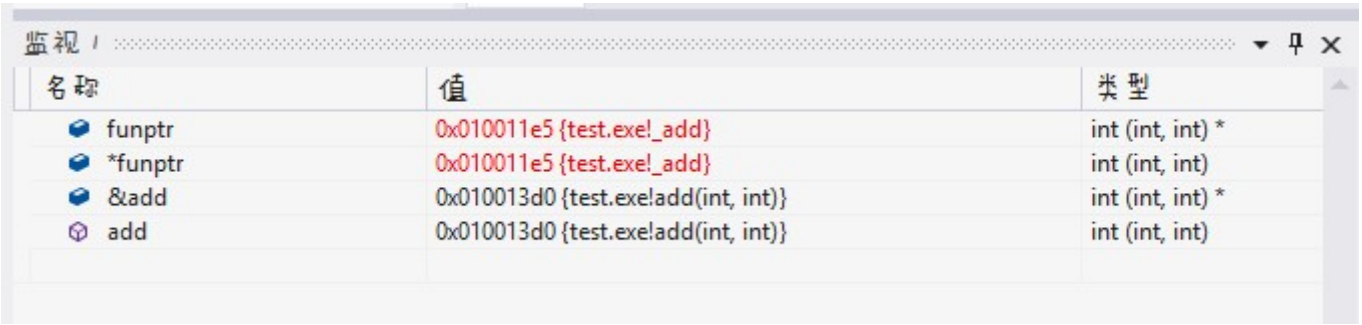
实在不是太搞得懂函数指针是一个怎么样神奇的存在

定义了一个函数add和一个函数指针funptr，将add赋给funptr

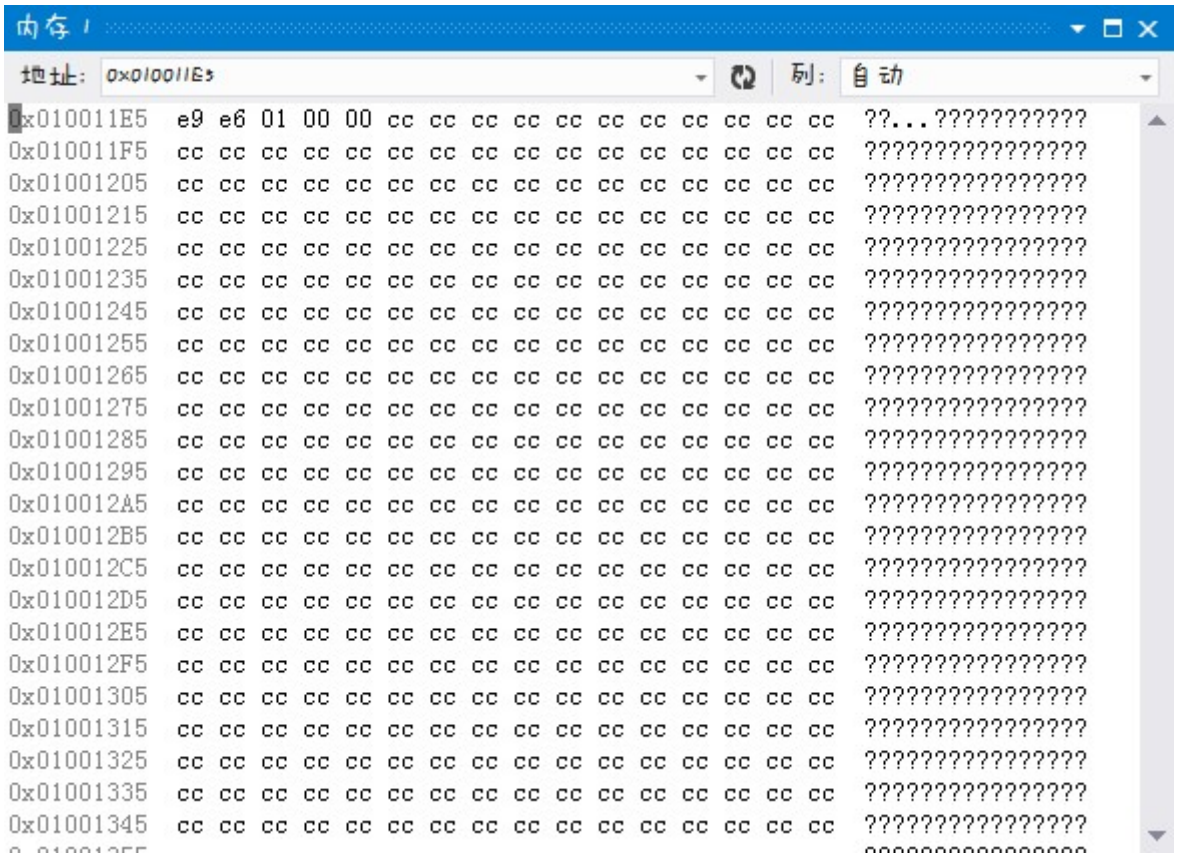


调试跟踪，发现*funptr和funptr，add和&add的值都是一样的，而且这两者之间居然还不一样

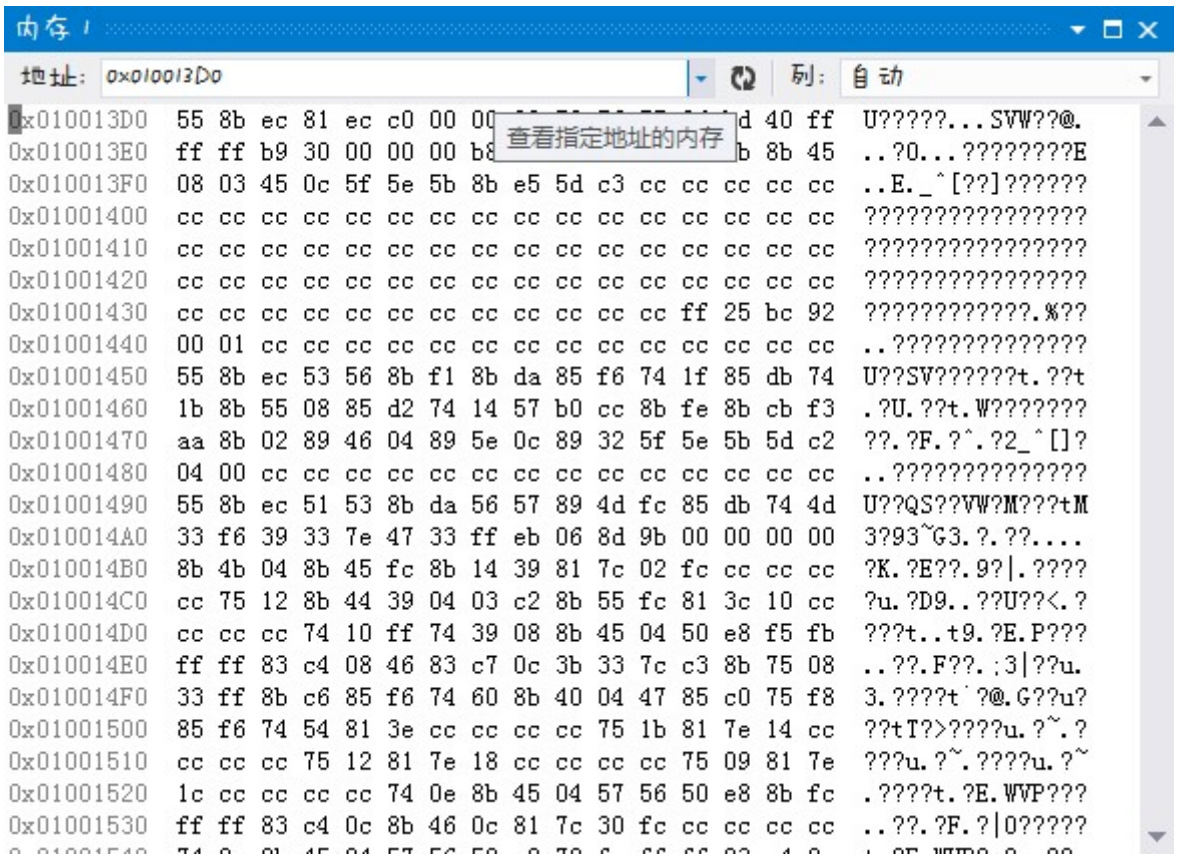
这个时候明明已经执行了funptr=add这一步了



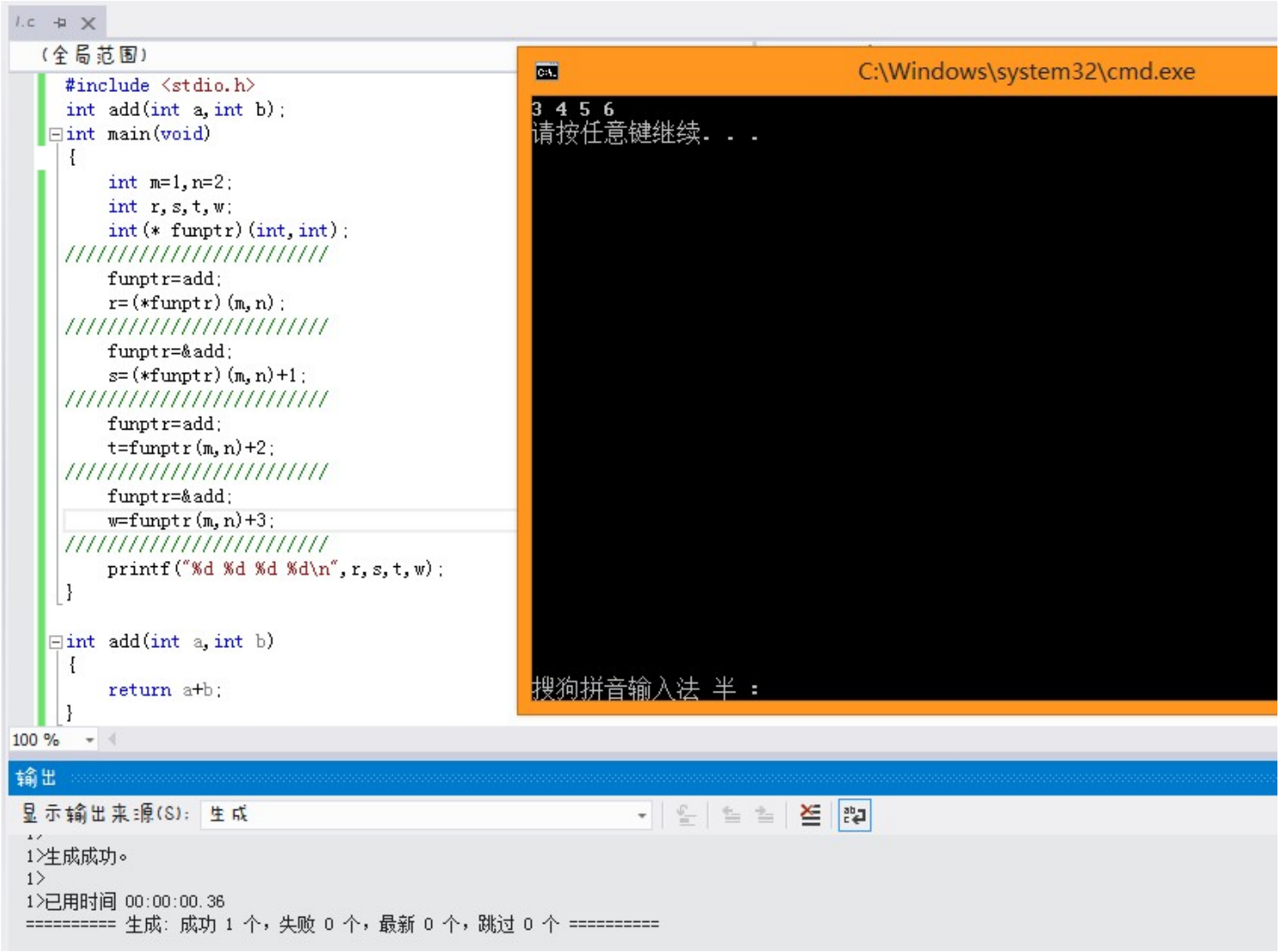
然后观察内存地址为0x010011E5（funptr的值）的值只有这么一点东西



再看地址为0x010013D0（add的值）处的内容，这里应该就是函数add的内容



再再再然后，把程序那两行换成这四种情况都可以彻底晕了



答：

函数名本身就已经表示该函数的地址，因此取函数地址时，是不应该在函数名前加&的，
即funaddr = add;正确用法，而funaddr = &add是错误用法，不过多数编译器能
容忍后面这种用法，并把它自动转化成前面这种用法；
当引用函数指针进行函数调用时，C语言中必须写成:
r = (*funaddr)(m, n);
*这样的形式，其中*funaddr表示函数指针funaddr指向的函数实体。
而C++中，除了上面这种形式外，也可以写成:
r = funaddr(m, n);

142 数组指针和指针数组为什么可以一样进行p[i][j]操作

问：

我写了两个短程序

```
main ()
{
  int a[2][3]={{1,2,3},{4,5,6}};
  int *p[2];
  int i,j;
  for (i=0;i<2;i++)
     p[i]=a[i];
  for (i=0;i<2;i++)
  {
      for(j=0;j<3;j++)
        printf("%d",p[i][j]);
      printf("\n");
  }
}
```

和

```
main ()
{
  int a[2][3]={{1,2,3},{4,5,6}};
  int (*p)[2];
  int i,j;
  p=&a[0];
  for (i=0;i<2;i++)
  {
    for(j=0;j<3;j++)
      printf("%d",p[i][j]);
    printf("\n");
  }
}
```

为什么两个的输出结果是一样的。在这种情况下他们可以通用吗？还有为什么第二个程序里，我把p=&a[0]改成p=&a也行呢，他们之间的含义有什么区别吗？谢谢老师

又有个问题 (float)10/2 不是该强制转成float吗 为什么是double呢

答：

两个明显是不一样的东西，前者的p是指针数组，而后者的p是数组指针；
前者的p[0]、p[1]是两个指针变量，而后者的p只是一个变量；
不过，按你的写法，无论前者p[i][j]还是后者p[i][j]都是正确的，它们都能达到表示二维数组元素的目的。
第2个程序里面，把p=&a[0]改成p=&a一定是错误的，只不过你在编译的时候忽略了警告。&a是一个指针，它指向了一个二维数组，即指向一个平面，而p按规定只能指向一行。要是改成p=a就对了。

> 又有个问题 (float)10/2 不是该强制转成float吗 为什么是double呢

这个是按照C89老标准，凡是float都一律要提升为double。
若按新标准C99来，则此表达式的类型仍旧为float。

143 [C程]关于feof()函数不能正常判断文件结束位置的探讨

多次实验表明，当文件指针已经移动到EOF位置即文件最末字节的右边那个位置，feof()并不能返回true。只有当你在EOF位置做一次读取动作，feof()才会返回true。

经测试，DOS下的TC、Windows下的VC、Linux下的GCC中的feof()函数都有上述特征。

假定有一个文本文件abc.txt，长度为3个字节，里面仅包含字母'A','B','C'，则以下程序中的while循环一共会执行4次，分别读出'A', 'B', 'C', -1即EOF:

```
#include <stdio.h>
main()
{
   FILE *fp;
   unsigned char c;
   long int position;
   fp = fopen("abc.txt", "r");
   position = ftell(fp);
   printf("position=%d\n", position);
   while(!feof(fp))
   {
      c = fgetc(fp);
      position = ftell(fp);
      printf("c=%02X, position=%d\n", c, position);
   }
   fclose(fp);
}
```

程序的输出结果如下：

```
position=0
c=41, position=1
c=42, position=2
c=43, position=3
c=FF, position=3
```

根据程序的输出结果可以推断，当程序读取字母'C'(16进制为43)后，文件指针已经移动到EOF位置即position=3，但此时while却并不结束，feof(fp)仍旧返回true，于是再次进入循环fgetc()就会读取一个并不存在的东西，此时fgetc()会返回-1表示读取失败，并且文件指针仍旧停留在EOF位置即position=3，并不会再向右挪动一格。

所以，我们不应该把-1理解成feof()读到的内容，因为若-1是内容，
那文件指针就应该向右移动一格变成4。

当第4次循环让fgetc()在EOF位置做了一个读的动作后，feof(fp)才
返回true，但此时我们已经多读取了一个垃圾数据。

如何避免这多出来的1次循环?
方法1：用fscanf()代替fgetc()做读取工作，格式采用"%c "(注意%c后面有一个空格)。

```c
#include <stdio.h>
main()
{
   FILE *fp;
   unsigned char c;
   long int position;
   fp = fopen("abc.txt", "r");
   position = ftell(fp);
   printf("position=%d\n", position);
   while(!feof(fp))
   {
      /* c = fgetc(fp); */
      fscanf(fp, "%c ", &c);
      position = ftell(fp);
      printf("c=%02X, position=%d\n", c, position);
   }
   fclose(fp);
}
```

以上程序在VC、Watcom C++、Linux GCC下都能得到正确结果，如下所示:

```
position=0
c=41, position=1
c=42, position=2
c=43, position=3
```

但TC 2.0输出结果不正常，字母'C'没有输出，却输出了-1:

```
position=0
c=41, position=1
c=42, position=2
c=FF, position=3
```

方法2: 把循环条件改成(c=fgetc(fp)) != EOF

```c
#include <stdio.h>
main()
{
   FILE *fp;
   char c;
   long int position;
   fp = fopen("abc.txt", "r");
   position = ftell(fp);
   printf("position=%d\n", position);
   while((c=fgetc(fp)) != EOF)
   {
      position = ftell(fp);
      printf("c=%02X, position=%d\n", c, position);
   }
   fclose(fp);
}
```

方法3:用ftell()求出文件的长度，用长度控制循环次数

```c
#include <stdio.h>
main()
{
   FILE *fp;
   char c;
   long int position, len;
   fp = fopen("abc.txt", "r");
   fseek(fp, 0, SEEK_END);
   len = ftell(fp);
   fseek(fp, 0, SEEK_SET);
   position = ftell(fp);
   printf("position=%d\n", position);
   while(len != 0)
   {
      c = fgetc(fp);
      position = ftell(fp);
      printf("c=%02X, position=%d\n", c, position);
      len--;
   }
   fclose(fp);
}
```

144 win8.1系统运行vc6.0的解决方法

在win8.1环境下，vc6.0无法运行，在网上找到了解决方法，实测可行，现在来分享。

把安装目录下的"MSDEV.EXE",改个名字比如"MSDEV1.EXE"..."MSDEV2.EXE"...."MSDEVx.EXE",
然后再右键,兼容模式选择"WINXP SP2"或者WINXPSP3都可以.把"以管理员身份运行"也勾上,再运行.

第一次报错,没关系,关闭,再运行一次就OK,以后都OK了.
若不行,在改改名称,再试验,总有一个能行的.我的改为"MSDEV9.EXE"

来源网站http://bbs.ithome.com/thread-569555-1-1.html
ps:感谢来自cc98的**这真讽刺**同学的分享

145 [C大程]如何用010Editor搜索源代码中的函数?

用010Editor工具可以轻松搞定某个函数在哪些.c文件中被引用,
步骤如下:

1. 主页下载010Editor:
   http://blackwhite.8866.org/bhh/010Editor.rar
2. 安装好之后，用安装包内的keygen打补丁再生成注册码激活软件
3. 点菜单Search->Find in Files
   Type选择ASCII String
   Value输入函数名如putimage
   Find in选择目标文件夹如D:\DosBoxWc\Watcom\project
   File Types输入*.c
   Include Subdirectories前面打勾
   Match Case(完全匹配大小写) 按需要决定前面是否打勾
   Match Whole Word Only(匹配整个单词) 按需要决定前面是否打勾
4. 点按钮"Find All"

146 [C大程]如何查库函数帮助信息?

1. 主页下载DosBoxTc，解压缩，双击里面的dosboxtc.exe，输入函数名，按Ctrl+F1
   http://blackwhite.8866.org/bhh/DosboxTc.rar
2. 主页下载函数速查软件，输入函数名查询:
   http://blackwhite.8866.org/bhh/function.exe
3. 打开头文件graphics.h查看函数注释:
   DosBoxWc\Watcom\h\graphics.h

147 [C大程]中断函数内切换到用户堆栈以便调用库函数的演示

中断服务函数如int_9h()在调用某些库函数如printf()时会出现问题,
要么没有结果，要么程序异常退出。究其原因是因为中断服务函数内
的堆栈是系统堆栈，而不是用户堆栈，导致堆栈段寄存器SS != DS,
从而造成某些库函数在引用内存变量时计算地址有错。
现针对此问题专门写了一个演示程序stkshift.c，请各位同学按程序

内的注释复制相应代码到自己的程序中。
http://blackwhite.8866.org/bhh/stkshift.rar

148 关于printf()函数参数从右到左方向传递的问题

问：

下面一个程序，在printf函数中，按照老师讲的逗号表达式从右向左进行参数传递，但为什么有时候右边的变量的值会被左边的操作影响？而那些逻辑、关系表达式的值却不变？
自己编了几个简单程序试验了几次，还是不清楚。猜想逻辑表达式、关系表达式的值是不是不会像一般的变量、常量一样保存在内存中？可它们是怎么来的啊？
求解答...

```
#include <stdio.h>
int main(void)
{
    int x=5, y=5, z=0;
    printf("%d %d %d %d %d\n", x==0 ,z=1 ,(z==1)||(x=0), x==0, z);
    printf("%d\n", z);
    printf("%d %d\n", y++, y);
}
```

输出结果：
1 1 0 0 1（为什么右边的z能是1，而（z==1）|| (x=0)却不是真呢？）
1
5 6

答：

我在TC里跑了一下，结果跟你不一样：
1 1 0 0 0
1
5 5

另，printf()里面的参数不能叫逗号表达式，
逗号表达式一般都是用()括起来的整体。
而且逗号表达式内部各项的求值顺序是从左到右的。

int a=2, b=3, c;
printf("%d %d", b, (b++, c=a+b));
这里的 (b++, c=a+b) 才是逗号表达式。
dev-c++是C++的语法，不是C语言编译器，因此它有可能不遵守C语言的规范。
不用理会它的输出结果。

149 老师，该怎么理解这里的a和&a大小一样?

问1:

```
#include<stdio.h>
int main()
{
        int a[4]={1,2,3,4};
        printf("%d %d",a,&a);
}
```

&a是不是就是数组的地址，然后就是数组首元素的地址，即a?

答1:

&a可以理解成一维数组a的地址，可以把a当作二维数组的一行来看。

问2:

```
#include<stdio.h>
int main()
{
        int a[4]={1,2,3,4};
        printf("%d %d %d",a,&a,&(&a));
}
```

老师，那为什么又不可以将a当作三维数组的，这样写就会报错了。。由此我又想到了下面几个问题。。
问题一：是不是对于一个变量，取地址的操作只能进行一次（和*操作不同）？
问题二：对于一个数组名取地址得到的就是它本身的地址吗?
问题三：对于一个指针变量（指向一维数组），我发现指针变量的地址就是紧接着它所指向的数组的末尾，为什么会这样，麻烦老师讲一下编译器是如何分配地址的。

答2:

> 首先明确一点，我们在写int a[4]={1,2,3,4};的时候，实际内存中只有四个sizeof(int)的空间分配给了这个数组。
> 问题一：取地址&只能对内存中的变量进行操作。对于一个内存中的变量，你取地址之后的结果并不会放到内存之中，因此你再去取地址是非法的。
> （在汇编语言中会知道，获得变量的地址，这个地址会被放到寄存器中
> 问题二：我个人的理解是，一维数组名本身是指针，指向数组的第一个元素，也就是第一个元素的地址，对数组名取地址之后，得到的还是个指针，指向的是整个数组空间，也就是整个数组空间的地址。这两者是不同的，可以输出(a+1)和(&a+1)看看结果理解一下就知道了。
> 整块数组空间的地址和第一个元素的地址的值相同的，这也是为什么a和&a相等，但是它们的类型是

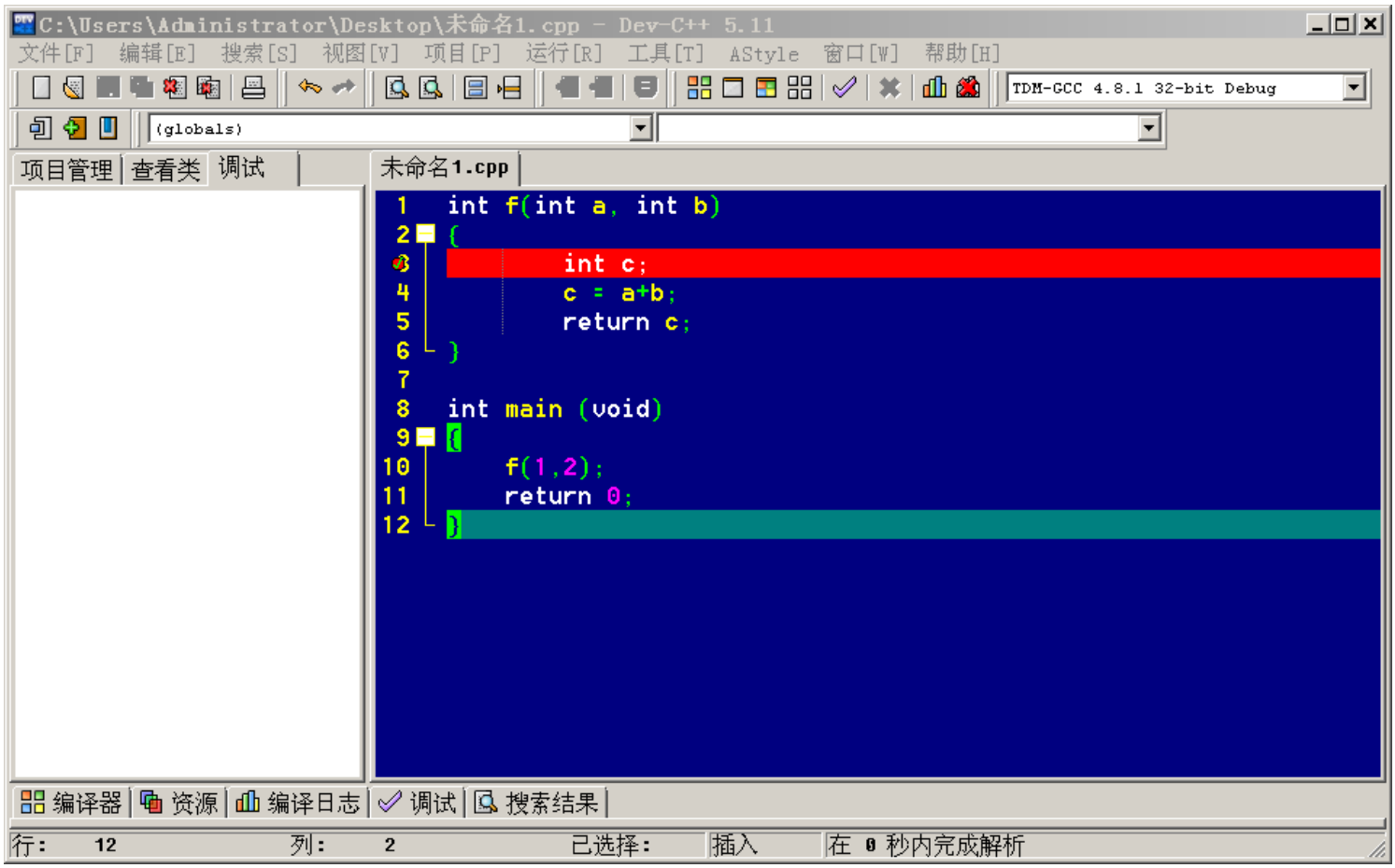
> 不相同的
> （ a的类型是int * ， 而&a的类型是int (*)[4]，也就是二维数组中的行指针
> 问题三：莫非说的是
>
> ```
> int a[3] = {1, 2, 3};
> int *p = a;
> printf("%x %x\n", a, &p);
> ```
>
> 但是我测试的结果是5e125c1c 5e125c10也就是a紧跟着p，考虑到局部变量是在堆栈中分配的，所以先分配的是a后分配的p。
> 这个涉及到内存分配的问题，对于局部变量，它的内存分配是连续的，所以p的地址和数组是紧接着的。对这个有兴趣需要了解一点汇编的知识，可以去选小白的汇编语言那门课，会有深入的理解。
> ps:感谢来自cc98的**lxlxwana**同学的解析

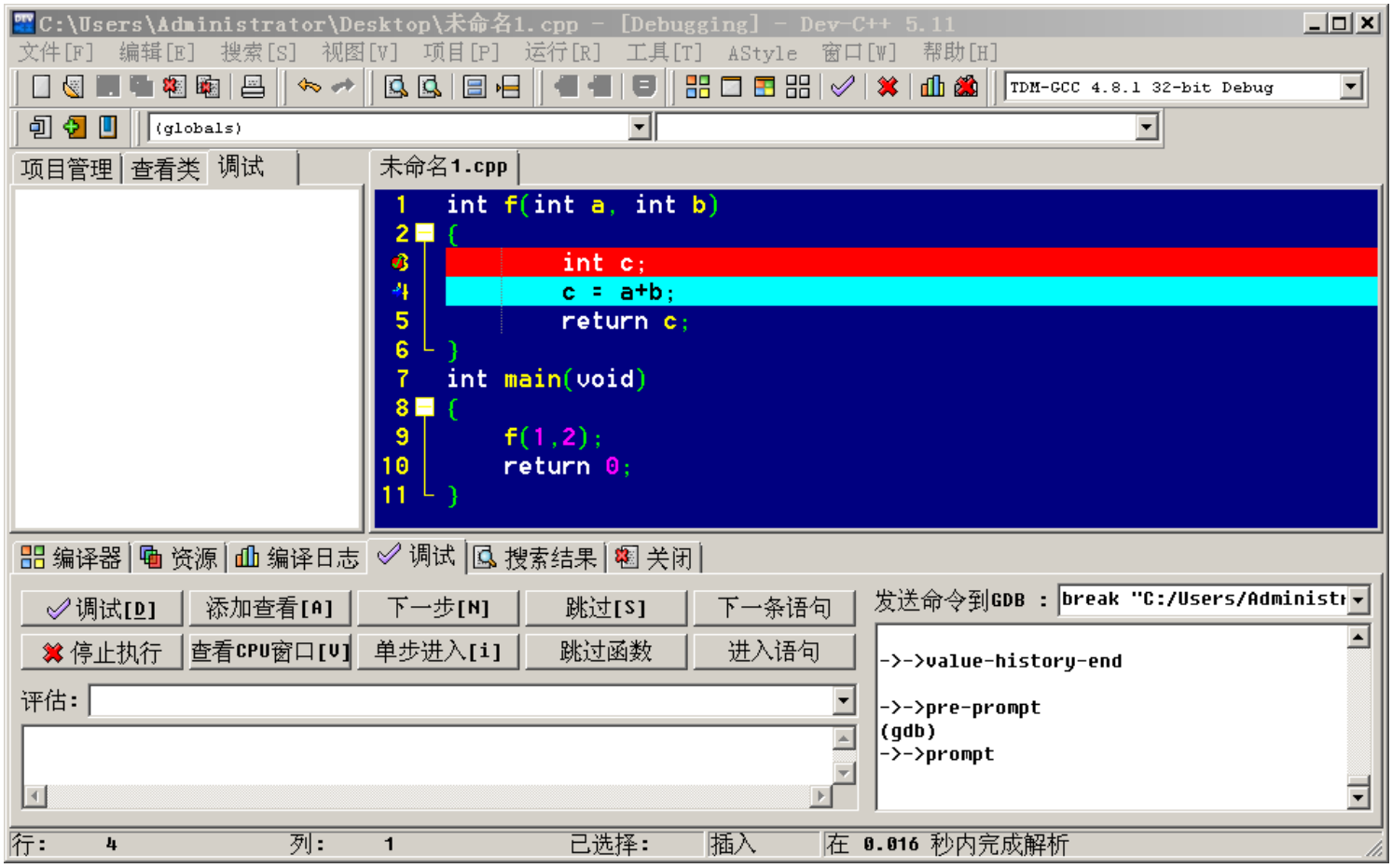
150 用dev-cpp进行C语言调试的方法（汇编与条件断点）

方法：

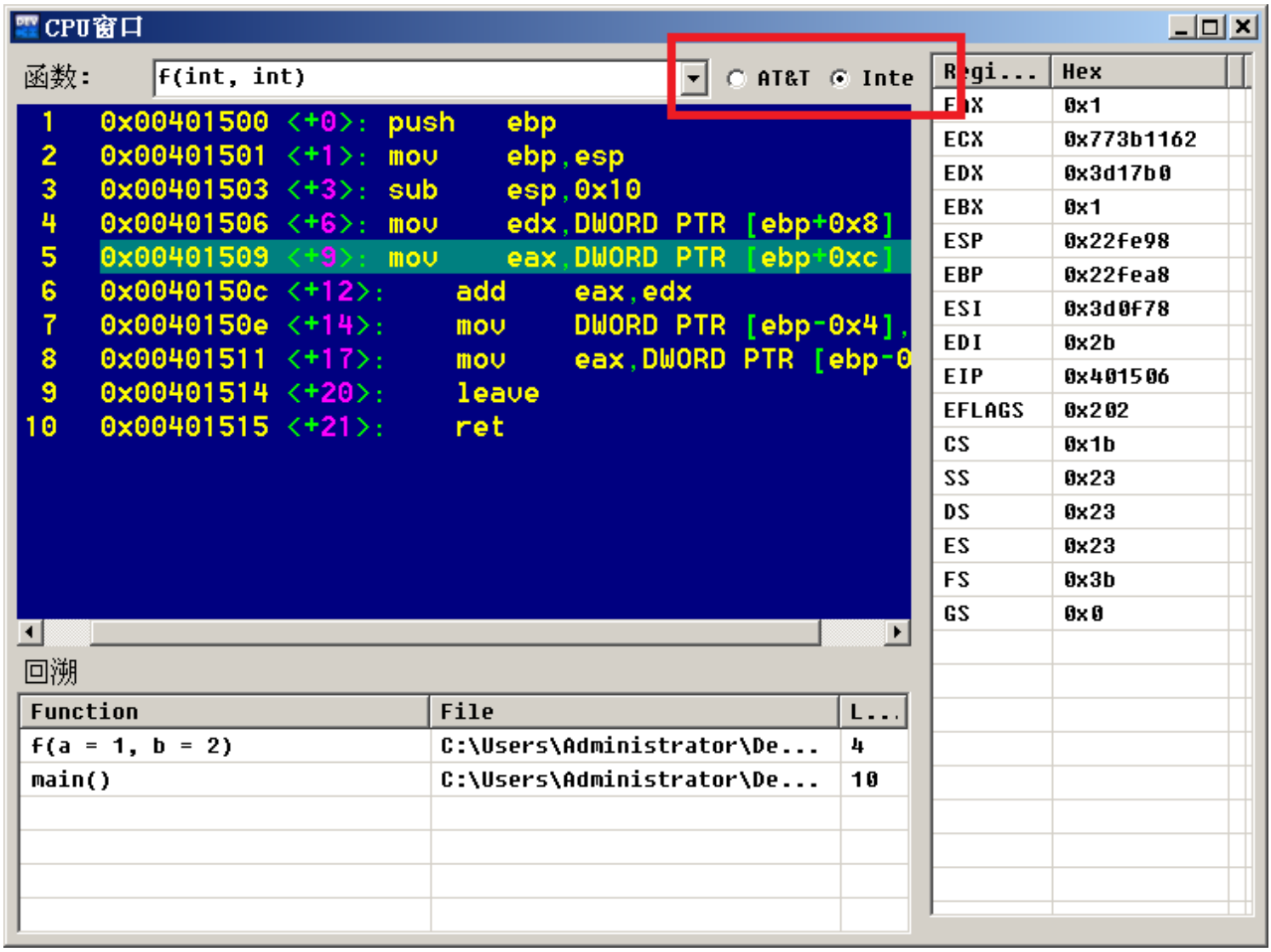
用dev-cpp进行C语言调试的方法（汇编与条件断点）
dev-cpp是一个小巧简单、方便快捷的IDE。个人十分喜欢用（当然，在大型工程、代码补全等方面还有欠缺）
今天考完试，正没事干，就来讲一讲如何用dev-cpp进行汇编

首先，写出任意的一段程序，单击行号即可设置断点，进行调试（想必这个大家都知道……）


C:\Users\Administrator\Desktop\未命名1.cpp - Dev-C++ 5.11
文件[F] 编辑[E] 搜索[S] 视图[V] 项目[P] 运行[R] 工具[T] AStyle 窗口[W] 帮助[H]
TDM-GCC 4.8.1 32-bit Debug
(globals)
项目管理 查看类 调试
未命名1.cpp
1 int f(int a, int b)
2 {
3 int c;
4 c = a+b;
5 return c;
6 }
7
8 int main (void)
9 {
10 f(1,2);
11 return 0;
12 }
编译器 资源 编译日志 调试 搜索结果
行: 12 列: 2 已选择: 插入 在 0 秒内完成解析

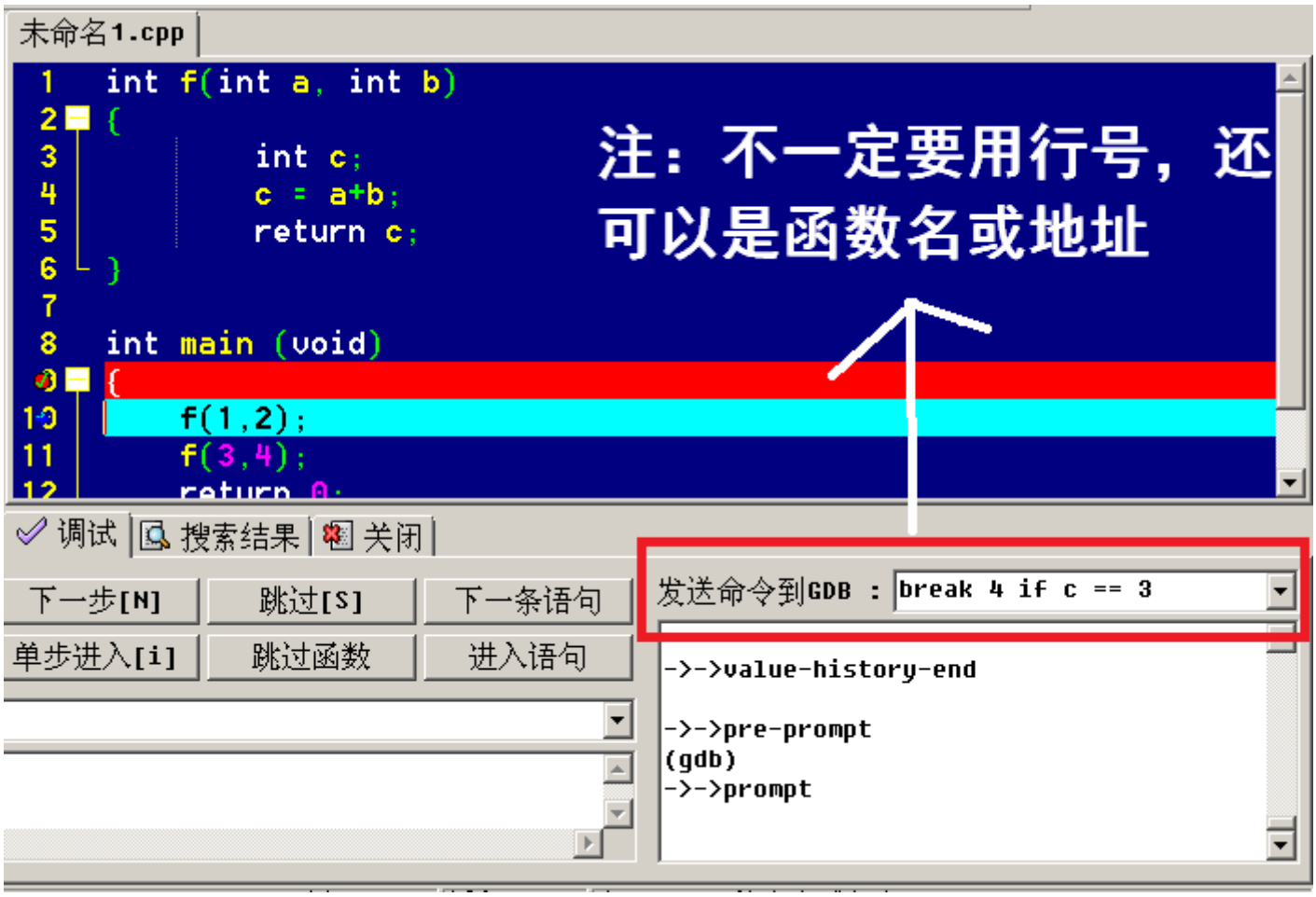

C:\Users\Administrator\Desktop\未命名1.cpp - [Debugging] - Dev-C++ 5.11
文件[F] 编辑[E] 搜索[S] 视图[V] 项目[P] 运行[R] 工具[T] AStyle 窗口[W] 帮助[H]
TDM-GCC 4.8.1 32-bit Debug
(globals)
项目管理 查看类 调试
未命名1.cpp
1 int f(int a, int b)
2 {
3 int c;
4 c = a+b;
5 return c;
6 }
7 int main(void)
8 {
9 f(1,2);
10 return 0;
11 }
编译器 资源 编译日志 调试 搜索结果 关闭
调试[D] 添加查看[A] 下一步[N] 跳过[S] 下一条语句
停止执行 查看CPU窗口[V] 单步进入[i] 跳过函数 进入语句
发送命令到GDB : break "C:/Users/Administr
评估:
->->value-history-end
->->pre-prompt
(gdb)
->->prompt
行: 4 列: 1 已选择: 插入 在 0.016 秒内完成解析

点击调试，再点击下面的“查看CPU窗口”，就可以查看当前函数的汇编指令了
嫌AT&T格式看起来不习惯，还可以选中”Inte”来查看intel格式的汇编。



调试栏中的“下一条语句”“进入语句”就是用在汇编上的。点击“下一条语句”后，会单步执行一条汇编语句（但不会进入call调用的函数，而是直接到call的下一句），点击“进入语句”，就可以进入call调用函数。

上面就是汇编相关的，下面来讲条件断点
什么是条件断点呢？假如你在调试中发现了一个bug，有时候某变量的值会出现异常值0。但是，你不知道这个变量是什么时候变为0的，一步一步地调试很麻烦。这时，你可以设置条件断点，当这个变量为0 时触发断点，让程序暂停。
devcpp的调试是基于gdb的，gdb有条件断点的功能，但devcpp没有做出相应的图形化界面，这就需要我们在“发送命令到gdb”中手动写命令……
比如，我们有如下程序，要让f函数中的c为3时触发断点。

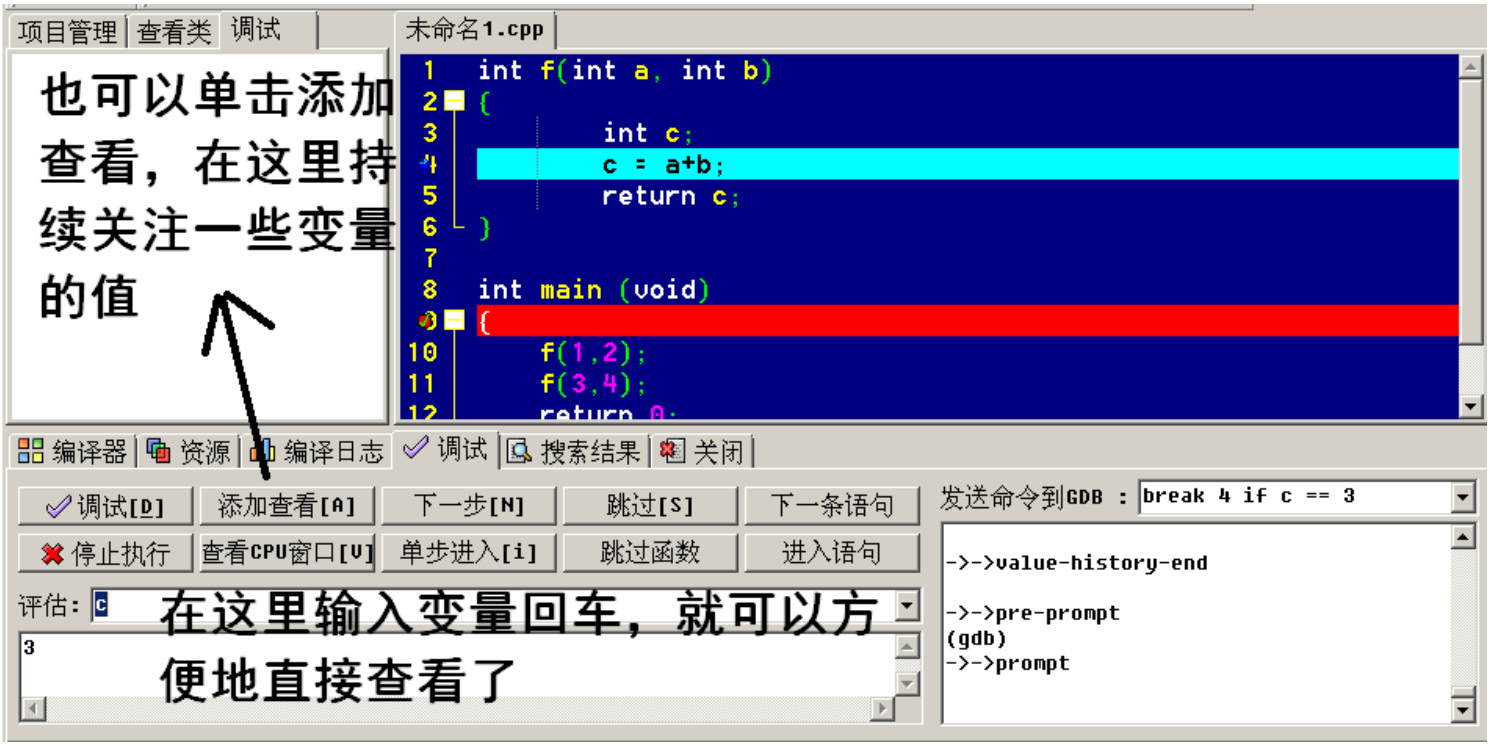


在“发送命令到gdb”中输入break 4 if c == 3并回车，这样，一个条件断点就设置好了

单击“跳过”（即继续运行，直到下一个断点触发）

可以看到，程序在第4行停下了。

在“评估”窗口中输入“c”并回车，就可以直接查看变量c的值，可以看到，c确实为3



不止是变量，DEVCPP可以直接查看表达式和函数的值，也是直接在这里输入你要看的表达式（如a+b）和函数（ 如f(1,2) ）就可以了。再次点击“跳过”，由于下一次调用是f(3,4);，c并不为3，所以，断点没有触发，程序运行到结束。

需要注意，每次点击调试，都要发送命令到gdb设置条件断点……不过，还好devcpp会把我们输入过的命令保存在下拉框中，只要选中回车就行了
了解GDB的相关命令，还可以有更多的用法。GDB是一个很强大的调试工具，在linux下，可以大显身手。
ps:感谢来自cc98的**五色旗**同学的分享